

# Automating All Things YANG, All the time - Programmability and Automation 101

with Cisco IOS XE

**CISCO** Live !

Jeremy Cohoe  
Technical Marketing Engineer @jeremycohoe

Ashil Parekh  
Product Management  
@Ashil34571977

Story DeWeese  
Technical Marketing Engineer  
@storydeweese

# Webex App

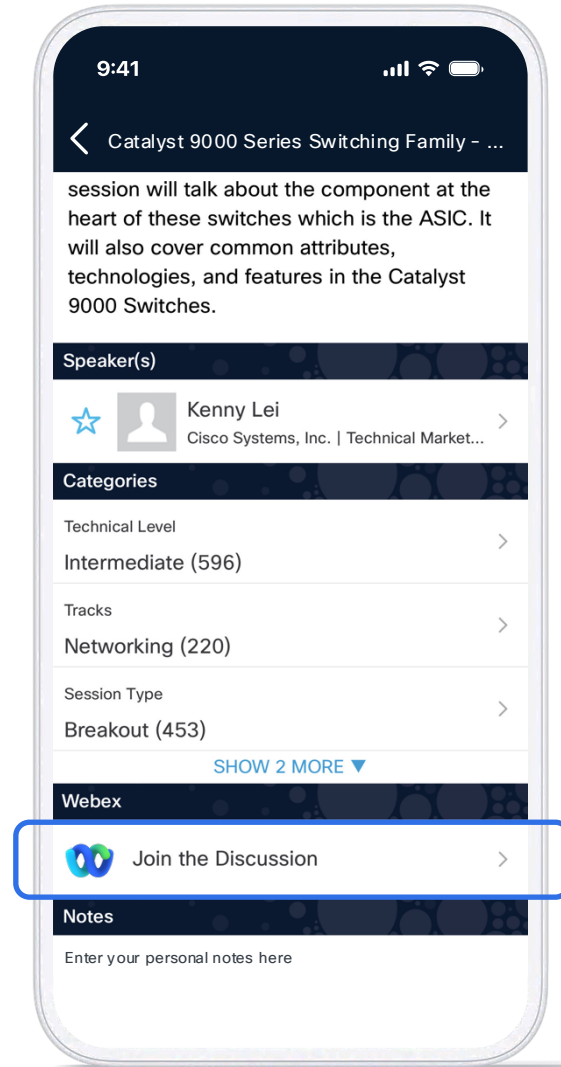
## Questions?

Use Webex App to chat with the speaker after the session

## How

- 1 Find this session in the Cisco Events App
- 2 Click “Join the Discussion”
- 3 Install the Webex App or go directly to the Webex space
- 4 Enter messages/questions in the Webex space

**Webex spaces will be moderated by the speaker until February 27, 2026.**



# Who We Are

Story DeWeese

Technical  
Marketing  
Engineer



Jeremy Cohoe

Technical  
Marketing  
Engineer



Ashil Parekh

Product  
Management



## Who you are:

1. Public sector
2. Financial
3. Education
4. Defense
5. MSP/Self
6. Others

We see “web scalers” as leaders in Programmability & Automation adoption  
Closely followed by: Financial, Education, Defense, Government, MSP... etc

# Agenda

8:45 – 9:55: Introduction / YANG & API / Tooling

5 mins – Story – Intro session

25 mins Ashil – Why we are here

50 mins – Jeremy – Intro to the API's & Tooling

10:05 – 10:30: Device Onboarding (SZTP)

5 mins – Ashil

20 mins – Story

10:30 – 11:00: Break – coffee and light snacks

11:00 – 11:35: Configuration Management

10 mins – Ashil

25 mins – Story

11:35 – 12:05: Monitoring and Reporting

5 mins – Ashil

10 mins – Jeremy – Intro to telemetry

15 mins – Story – Sustainability Outcomes

12:05 – 12:30: Optimization and Troubleshooting

25 mins – Story/Jeremy GuestShell/AppHosting

12:30 – 1:15: Conclusion, Resources and Questions

45 mins – all

- 01 Intro to Programmability
- 02 API's, YANG & Tooling
- 03 Onboarding (Day0)
- 04 Configuration (Day1)
- 05 Monitoring (Day2)
- 06 Optimizations (DayN)
- 07 Resources (everyday)
- 08 Closing & Q&A



# Cisco Webex App Technical Seminars

## Questions?

Use Cisco Webex App to chat with the speaker after the session

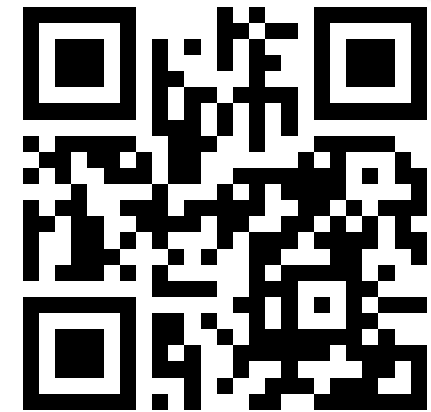
- Please ask any questions or share any comments you have within the Webex Space – We're monitoring it now !
- **We may have time for Q&A towards the end of the session**
- **We are happy to discuss with you 1:1 after the seminar is over as well**

## How

- 1 If you have not been added to the Webex space for this session, scan the QR code.
- 2 If you are unable to join the Webex space, it could be due to your businesses Webex org security policy. Please try again using a different email, such as Gmail or Yahoo.
- 3 Still having issues? Please visit the Webex Help Desk located at Registration.

**Webex spaces will be moderated by the speaker until June 13, 2025.**

**CISCO** Live !



TECOPS-2314

# Intro to Programmability





- Fewer Sites
- Less Endpoints
- Simple Network Architecture

# AI-Driven NetOps: The Opportunity is Real—But So Are the Prerequisites

## The Momentum: From Curiosity to Commitment

AI driven network management has progressed from **“Maybe I’ll use it”** to **“Must have”** [1]

**55% of leaders strongly agree:** AI-driven NetOps delivers better business outcomes [1]

**93% of organizations** are actively investing in AI-driven network management [1]

## The Reality: Is Your Network AI-Ready?

**No structured telemetry → no reliable AI** [2]

**Leaders recommend a phased approach:** first ensure your network can produce quality data, then layer in AI and automation [3]

**No APIs → AI can recommend, but it can't automate**



# The Total Economic Impact™ Of Cisco Meraki

Cost Savings And Business Benefits Enabled By Cisco Meraki Cloud-Managed Networking

A FORRESTER TOTAL ECONOMIC IMPACT™ STUDY COMMISSIONED BY CISCO, MAY 2025

## Key Findings

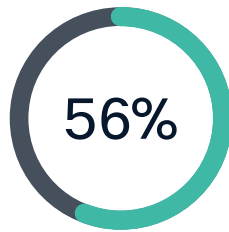
**Quantified benefits.** Three-year, risk-adjusted present value (PV) quantified benefits for the composite organization include:

- **A 40% reduction in network downtime.** Cisco Meraki cloud-based network management and centralized visibility allows the composite organization to proactively detect and address connectivity issues before they cause disruptions. As a result, the composite organization avoids lost revenue and business interruption, especially for digital-first or customer-facing operations. This saves it \$1.7 million over three years.
- **A 90% reduction in time spent on firmware upgrades.** With Meraki cloud-managed networking in a centralized dashboard, IT teams at the composite organization remotely schedule and automate firmware updates across devices, eliminating time-consuming manual upgrade processes. Devices self-install and revert to prior versions if needed, reducing downtime, avoiding after-hours work, and freeing up IT staff. This results in \$838,000 worth of resource savings over three years for the composite.
- **A 90% reduction in time spent on a configuration change.** Configuration changes, such as adjusting virtual local area networks (VLANs), security rules, or service set identifiers (SSIDs), are completed once in a centralized dashboard and pushed to all relevant devices. Compared to the composite's legacy processes, which required manual command-line interface (CLI) entry, Cisco Meraki allows the composite organization to execute changes three times faster by lower-cost staff. As a result, it saves \$1.5 million over three years.
- **Savings of \$1.4 million over three years from reduced network support.** The composite sees 80% fewer network support tickets and 80% faster resolution times due to real-time diagnostics, remote troubleshooting tools, and centralized telemetry from Cisco cloud-managed networking. Additionally, instead of dispatching onsite technicians, IT teams resolve many issues remotely, reducing labor and travel costs.
- **End-user productivity gains worth \$1.7 million over three years from reduced network issues.** Fewer disruptions mean less time spent by end users reporting network problems, waiting for resolution, or working around connectivity issues. On average, each resolved ticket avoids 0.75 hours of disruption for 15 impacted users at the composite organization.
- **Cost savings from retiring legacy solutions of \$134,000 over three years.** By consolidating functions into the integrated Cisco Meraki platform, the composite organization retires local controllers, legacy security appliances, and older switching gear. It also eliminates redundant monitoring tools and virtual private network (VPN) concentrators.

↑ Back to

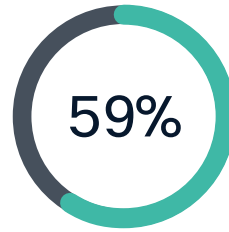
Reference – <https://tei.forrester.com/go/cisco/merakinetworking/?lang=en-us>

# From Networks to Business Impact: The Automation Advantages



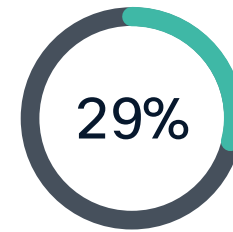
## Improved Operational Efficiency

More than 80% organizations were able to achieve process standardization, reduce Human errors and improve Mean Time to Resolution (MTTR) - [1]



## Operating Cost Reduction

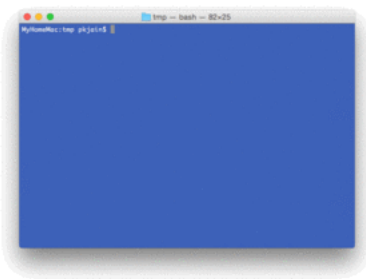
More than 80% organizations were able reduce network outages and improve security and compliance - [1]



## Faster Service Deployment

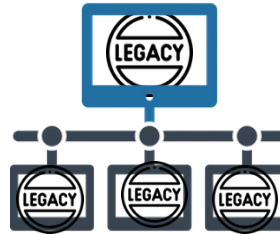
95% Respondents were able to improve the expedite the service deployment time and align with their business transformation goals - [1]

# State of Network Automation Adoption



## Manual Network Management

65% activities performed manually which causes 80% of outage and 22% of data breach[1]



## Legacy Network Infrastructure

46% IT Leaders say that Legacy systems are difficult to automate [2]



## Lack of Skills

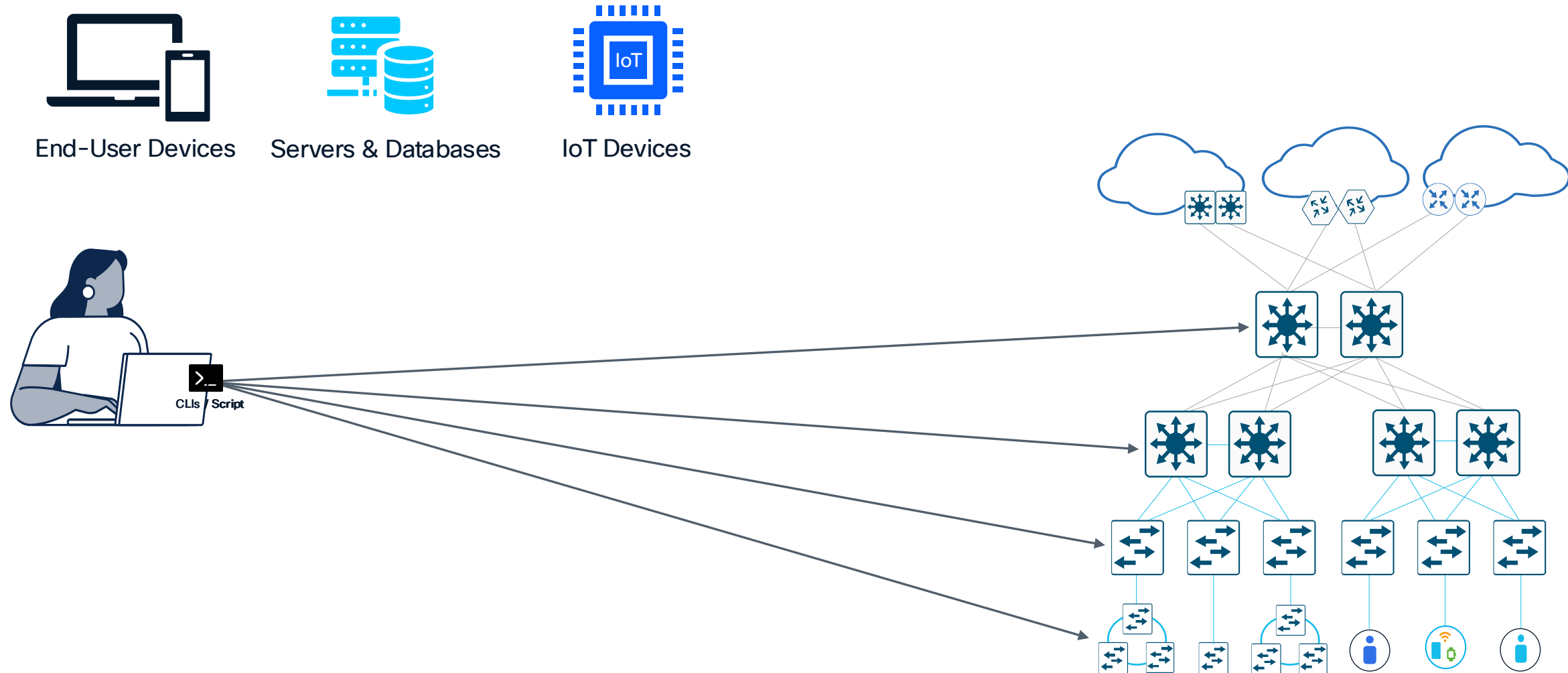
42% of IT Leaders claim their organizations lack network automation skills [2]

[1] - Enterprise Network Automation, Why do you need it? - Deloitte

[2] - Global Network Automation Report - IDC

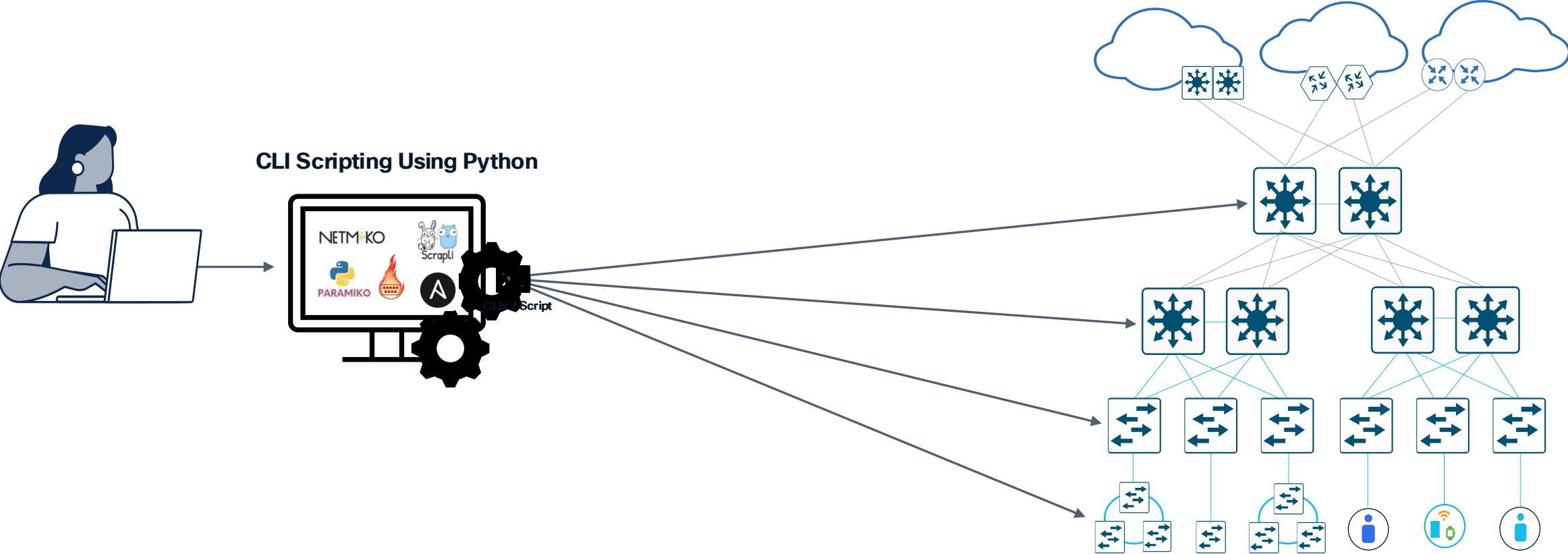


# If It Feels Repetitive, It's Not Automation



# The First Generation of Network Automation:

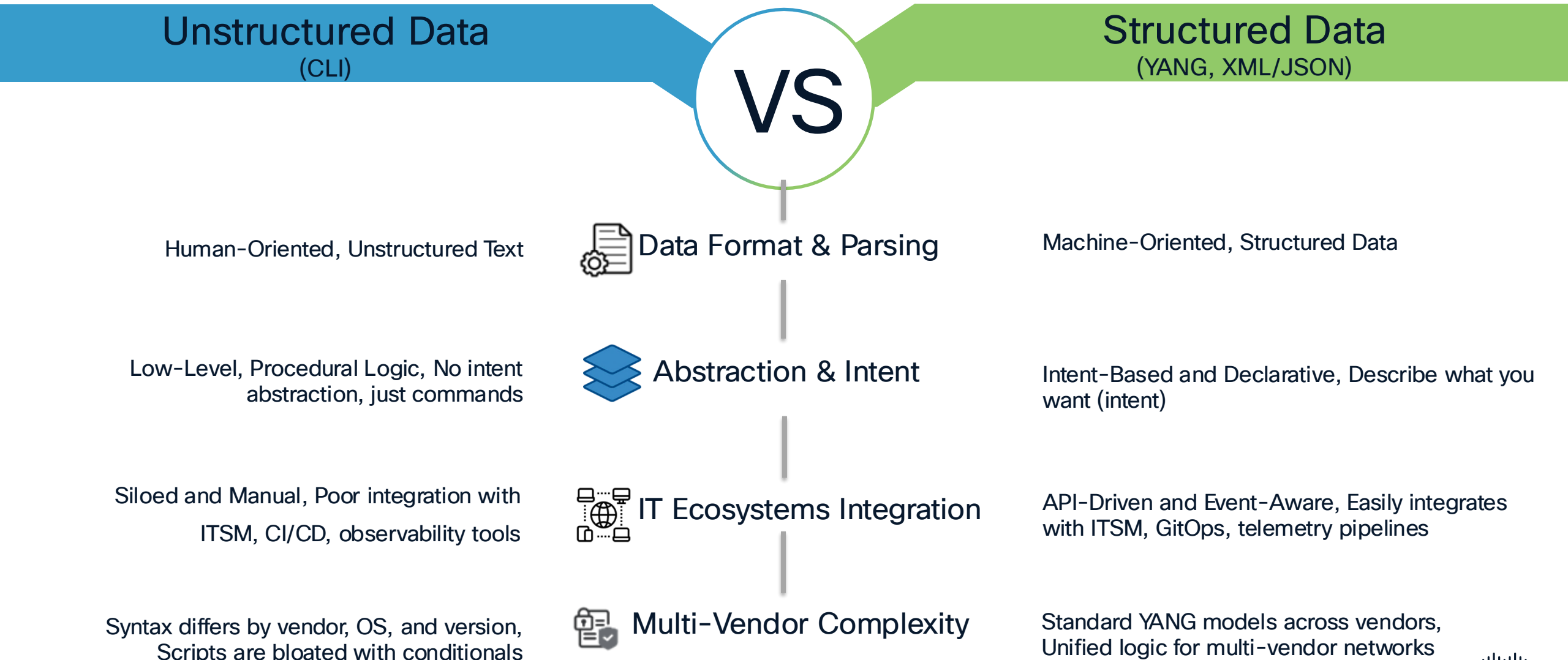
Sheets, Scripts, and Struggles



# Structured Data: The Fuel for Reliable Automation

| Feature / Capability       | Structured Data (YANG, XML/JSON)                               | Unstructured Data (CLI)                                |
|----------------------------|----------------------------------------------------------------|--------------------------------------------------------|
| Designed For               | Machine-to-machine communication                               | Human-to-machine interaction                           |
| Data Format                | Structured (XML/JSON following a schema)                       | Free-form text, tables, headings                       |
| Parsing & Automation       | Easy to parse programmatically                                 | Requires fragile screen-scraping                       |
| Validation                 | Schema-based (YANG) validation before commit                   | No built-in validation; errors caught only at runtime  |
| Error Handling             | Structured errors (e.g., <rpc-error> with tags and messages)   | Textual errors, inconsistent messages across vendors   |
| Transactions               | Supports atomic commits (all-or-nothing changes)               | Commands executed line-by-line; no rollback capability |
| Consistency Across Vendors | Standard models (OpenConfig, IETF) enable multi-vendor support | Vendor-specific CLI syntax and outputs                 |
| Change Safety              | Supports rollback, confirmed commits                           | No native rollback; must script it manually            |
| Monitoring Integration     | Works with model-driven telemetry (push-based)                 | Relies on CLI polling or SNMP                          |
| Scalability                | Efficient for managing 100s or 1000s of devices at once        | Slow, sequential; complex to scale manually            |
| Extensibility              | Models can evolve with versioning and modularization           | Hard to adapt; even minor format changes break scripts |
| Feedback Loop              | Clean input/output model for closed-loop automation            | No structured input/output contract                    |

# Structured Data: The Fuel for Reliable Automation



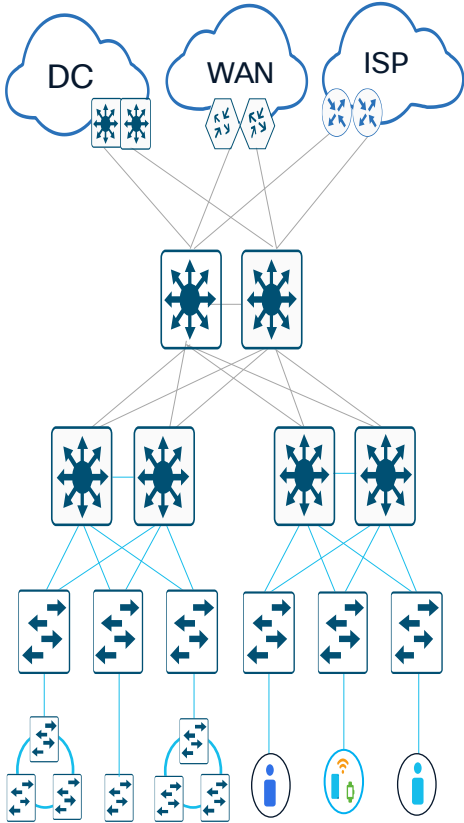


# What's the Difference?

Tools

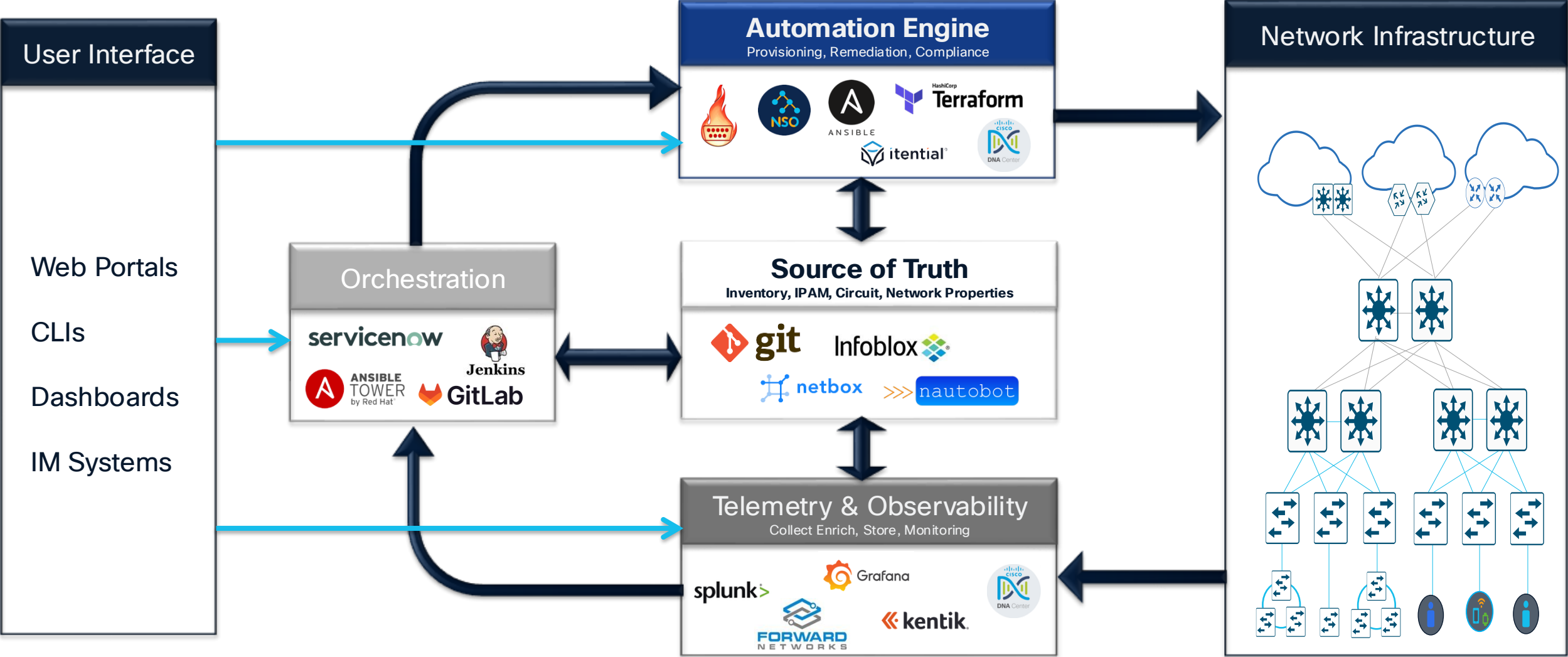
Protocol Stack

| Open Source or Infrastructure Automation Tools |                   | Mgmt Protocol                       | Data Models                  | Resources | Data Modeling Language | Encoding                | Transport           |
|------------------------------------------------|-------------------|-------------------------------------|------------------------------|-----------|------------------------|-------------------------|---------------------|
| Using                                          | Unstructured Data | Vendor Specific                     | -                            | CLI       | -                      | -                       | SSH                 |
|                                                | Structured Data   | NETCONF<br>RESTCONF<br>gNMI<br>gRPC | Vendor<br>IETF<br>OpenConfig | xPaths    | YANG                   | XML<br>JSON<br>PROTOBUF | SSH<br>HTTPS<br>TLS |



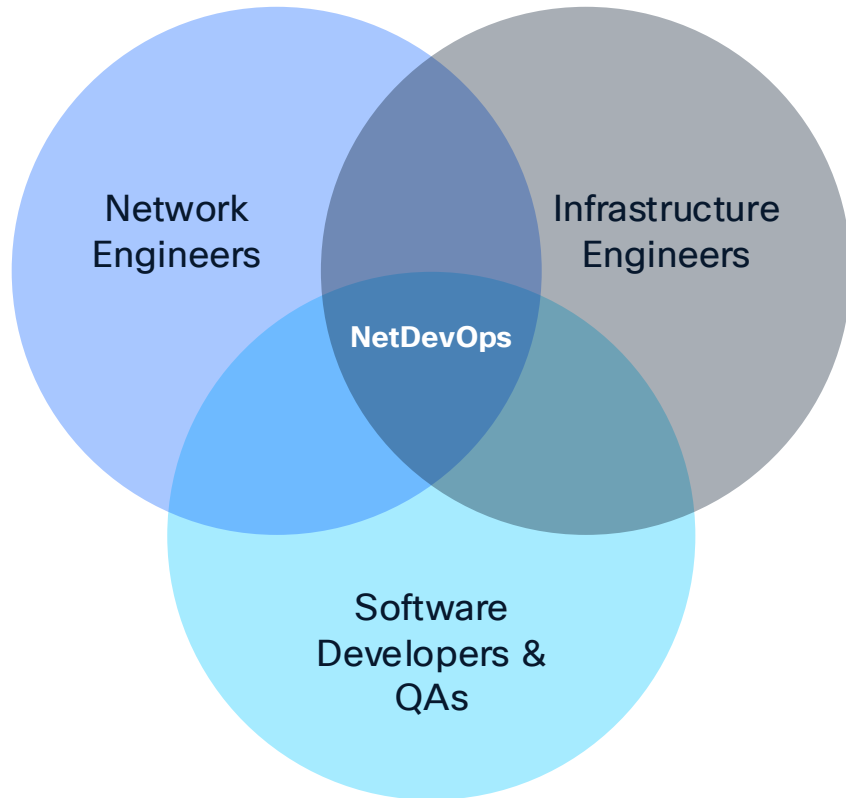
# Modern Network Automation Architecture

Microsoft VS Code  
Github Copilot  
Anthropic Sonnet 4.5



# From CLI to CI/CD: Enter NetDevOps

You can't fully realize the benefits of network automation unless you embrace NetDevOps.



Infrastructure as Code



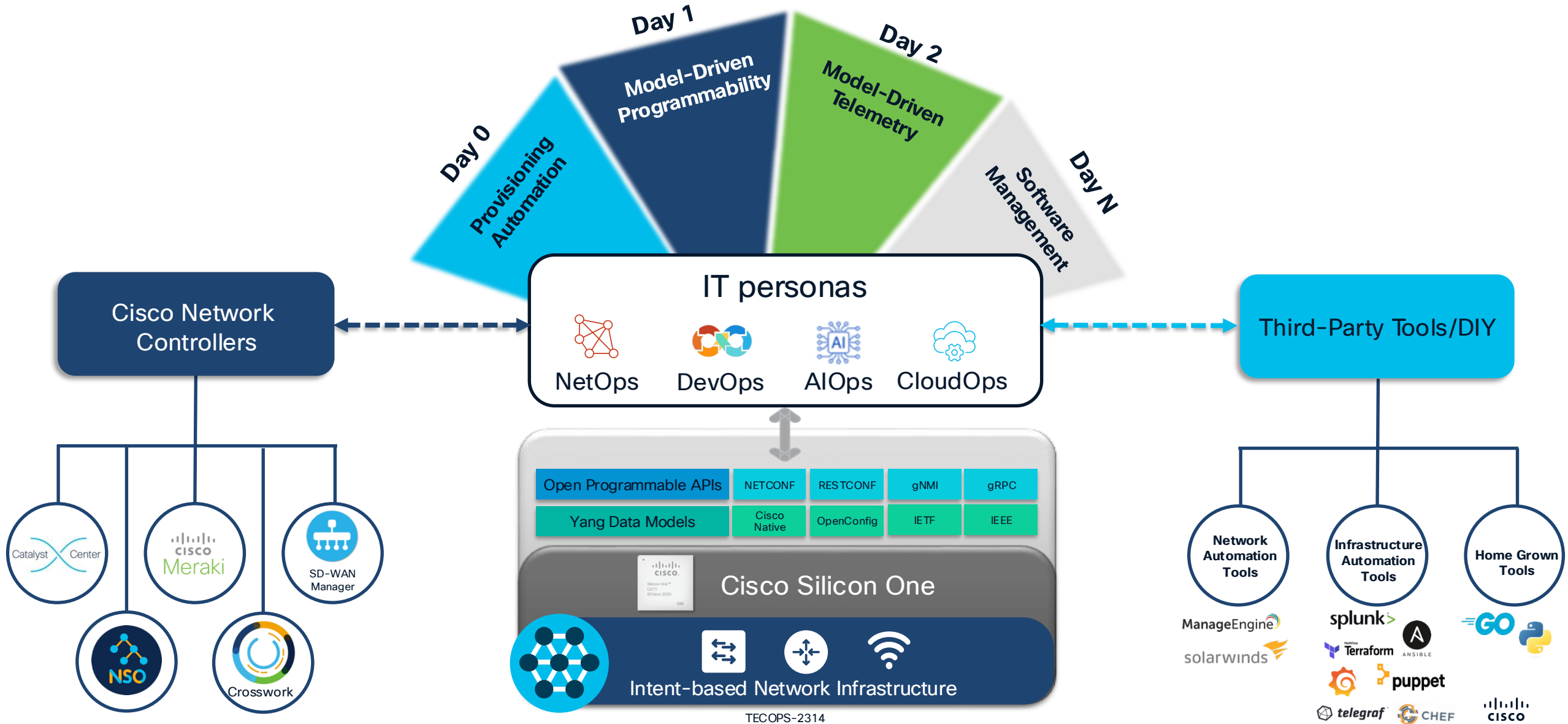
Version Control



CI/CD Pipelines

# Industry's most open network OS

Receptive to a wide range of management and orchestration strategies





# **YANG**

## **Lets talk... about data models**



# About Data Models

YANG = Yet Another Next Generation

*“A Data-Model Explicitly and precisely defines Data Structure, Syntax and Semantics”*



The YANG modules are included inside of Cisco IOS XE

They can be ‘exported’ from the software

They can be browsed using online/offline tools

## Interface Model definition

```
ietf-interfaces@2014-05-08.yang
/*
 * Configuration data nodes
 */
container interfaces {
  description
    "Interface configuration parameters.";
  list interface {
    key "name";
    description
  leaf name {
    type string;
  }
  leaf description {
    type string;
  }
  leaf type {
    type identityref {
      base interface-type;
    }
    mandatory true;
  }
  leaf enabled {
    type boolean;
    default "true";
  }
}
```

# Human vs Machine Interfaces

## CLI

```
router bgp 100
  bgp log-neighbor-changes
  neighbor 13.13.13.1 remote-as 100
  neighbor 13.13.13.1 update-source Loopback10
  neighbor 14.14.14.1 remote-as 100
  neighbor 14.14.14.1 update-source Loopback11
  neighbor 15.15.15.1 remote-as 100
  neighbor 101.1.1.1 remote-as 4200123003
  neighbor 172.26.244.141 remote-as 100
!
iox
ip forward-protocol nd
ip http server
ip http secure-server
!
ip route 10.10.1.0 255.255.255.0 Loopback12
ip route 13.13.13.0 255.255.255.0 172.26.244.141
ip route 14.14.14.0 255.255.255.0 172.26.244.141
ip route 15.15.15.0 255.255.255.0 172.26.244.141
ip route 16.16.16.0 255.255.255.0 172.26.244.141
ip route 21.21.21.0 255.255.255.0 Loopback12
ip route 100.101.102.0 255.255.255.0 Loopback12
ip route 123.22.11.0 255.255.255.0 Loopback12
ip route vrf Mgmt-vrf 0.0.0.0 0.0.0.0 172.26.244.1
ip tacacs source-interface GigabitEthernet1/0/24
ip ssh server algorithm encryption aes128-ctr aes192-ctr aes256-ctr
ip ssh client algorithm encryption aes128-ctr aes192-ctr aes256-ctr
```



Human-Oriented Interface

## YANG Models

```
list interface {
  key "name";
  unique "type location";

  leaf name {
    type string;
    reference
      "RFC 2863: The Interfaces Group MIB - ifName";
  }

  leaf description {
    type string;
  }

  ...
  container statistics {
    config false;
    leaf discontinuity-time {
      type yang:date-and-time;
    }

    leaf in-octets {
      type yang:counter64;
      reference
        "RFC 2863: The Interfaces Group MIB - ifHCInOctets";
    }

    ...
  }
}
```



Machine-Oriented Interface

# Feature Configuration via CLI or YANG

## via CLI

```
telemetry ietf subscription 101
  encoding encode-kvgpb
  filter xpath /memory-ios-xe-oper:memory-
statistics/memory-statistic
  stream yang-push
  update-policy periodic 6000
  source-vrf Mgmt-intf
  receiver ip address 10.10.1.45 57555
  protocol grpc-tcp
```

## via YANG Data Model

```
"mdt-config-data": {
  "mdt-subscription": [ {
    "subscription-id": "101",
    "base": {
      "stream": "yang-push",
      "encoding": "encode-kvgpb",
      "period": "6000",
      "xpath": "/memory-ios-xe-oper:memory-
statistics/memory-statistic"
    }
  }
  "mdt-receivers": {
    "address": "10.10.1.45"
    "port": "57555" }
  }
]
```

# YANG Models Example

## YANG Models

```
ietf-interfaces@2014-05-08.yang
/*
 * Configuration data nodes
 */
container interfaces {
  description
    "Interface configuration parameters.";
  list interface {
    key "name";
    description
      "Interface configuration parameters.";
    leaf name {
      type string;
    }
    leaf description {
      type string;
    }
    leaf type {
      type identityref {
        base interface-type;
      }
      mandatory true;
    }
    leaf enabled {
      type boolean;
      default "true";
    }
  }
}
```

## Data



Gig 1/0/1  
"CL rocks!"  
enabled



## XML Payload

```
interface.xml — MyPythonScripts
1 <config>
2   <interfaces
  xmlns="urn:ietf:params:xml:ns:yang:ietf-int
  erfaces">
3     <interface>
4       <name>GigabitEthernet0/0</name>
5       <description>CL Rocks!</description>
6       <enabled>true</enabled>
7     </interface>
8   </interfaces>
9 </config>
```

YANG Models → Data Models defined using the YANG language

# YANG Configuration Model Example

```
container ip {  
  list vrf {  
    description  
      "Configure an IP VPN Routing/Forwarding  
      instance";  
  
    leaf name {  
      type string;  
    }  
  
    leaf rd {  
      description  
        "Specify Route Distinguisher";  
      type rd-type;  
    }  
  }  
}
```

YANG

```
<ip>  
  <vrf>  
    <name>vrf_red</name>  
    <rd>65000:1</rd>  
  </vrf>  
  <vrf>  
    <name>vrf_green</name>  
    <rd>65000:2</rd>  
  </vrf>  
</ip>
```

XML

```
ip vrf vrf_red  
  rd 65001:1  
ip vrf vrf_green  
  rd 65001:2
```

CLI



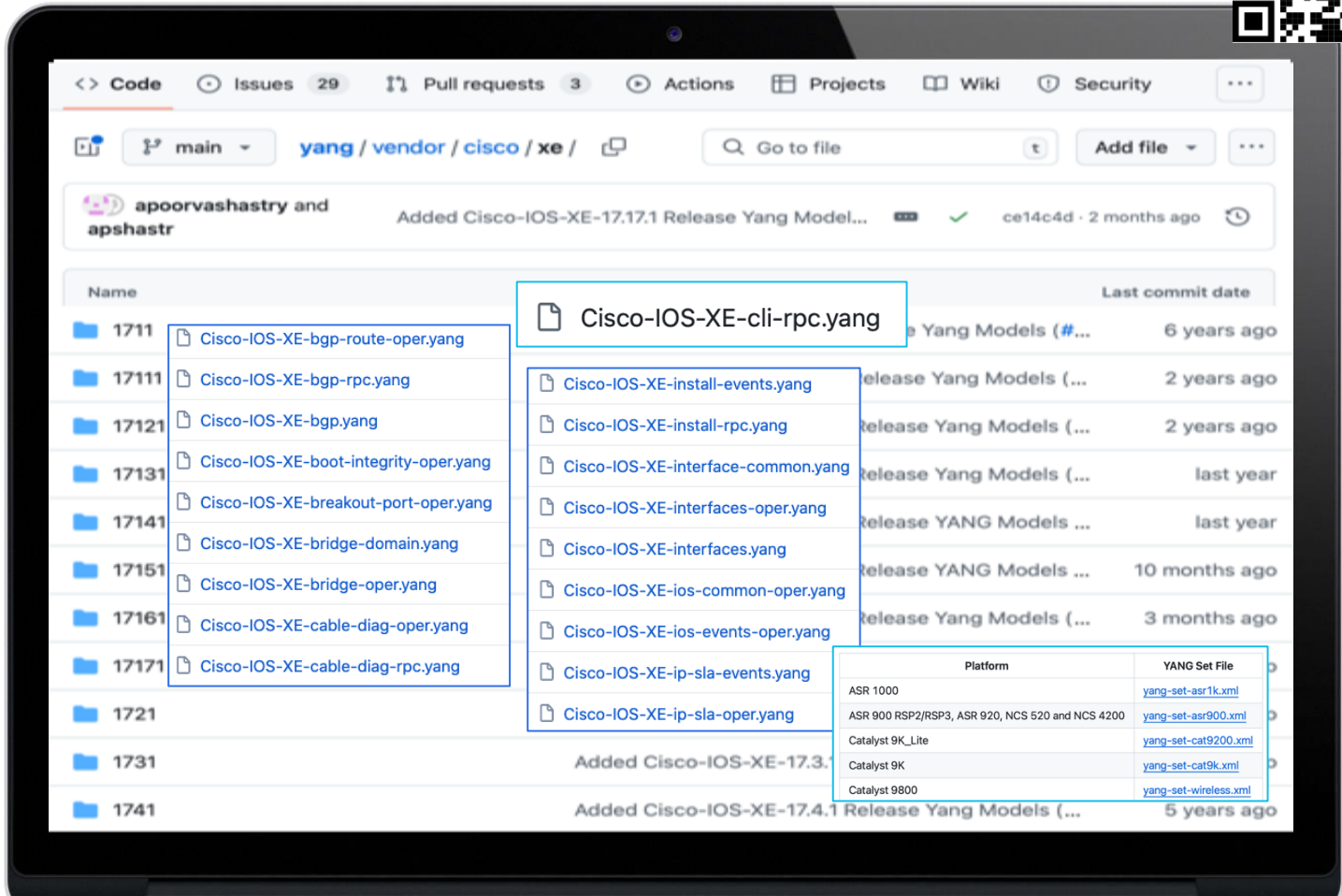
# Cisco IOS XE - YANG model API documentation



- 1. There are 10 types of data models including config, oper, actions, and deviations
- 2. RFC 7950 YANG data modelling language are the API definitions for IOS XE
- 3. The YANG modules are available for download from the API and are also published on Github.com
- 4. Notable modules are listed below for the running-config, feature oper, actions and event notifications
- 5. XML “set” file list which modules are supported on the various hardware platforms

|    | TYPE                       | YANG module name.yang                     | Description                                          |
|----|----------------------------|-------------------------------------------|------------------------------------------------------|
| 1  | Native configuration model | Cisco-IOS-XE-native                       | running-config                                       |
| 2  | Operation                  | Cisco-IOS-XE-(feature)-oper               | Feature operational data                             |
| 3  | Configuration              | Cisco-IOS-XE-(feature)-cfg                | Feature configuration, independent of "native" model |
| 4  | Events                     | Cisco-IOS-XE-(feature)-events             | Telemetry Events that can be triggered               |
| 5  | RPC                        | Cisco-IOS-XE-(feature)-rpc                | Actions that can be performed                        |
| 6  | Deviation                  | Cisco-IOS-XE-(feature)-deviation          | Device implementation deviation from module          |
| 7  | Types                      | Cisco-IOS-XE-(feature)-types              | Types – Imported by other modules                    |
| 8  | Obsolete                   | Cisco-IOS-XE-(feature)-obsolete           | Obsolete should not be implemented                   |
| 9  | Common                     | Cisco-IOS-XE-(feature)-common             | Common – Imported by other modules                   |
| 10 | Abstractions               | OpenConfig-(feature) & Cisco-evpn-service | abstraction for EVPN, OpenConfig config & oper       |

Since 17.10, YANG 1.1 modules are not advertised in NETCONF hello/capabilities but are available by reading the ietf-yang-library module



The YANG models are available for download directly from the running IOS XE device’s NETCONF, RESTCONF, or gNMI API, and from: <https://github.com/YangModels/yang/tree/main/vendor/cisco/xr>  
Find the model for your usecase: <https://www.yangcatalog.org>



# Cisco IOS XE – YANG model innovations

Deep dive into YANG models and FAQ, Common Models, etc

<https://github.com/jeremycohoe/cisco-ios-xe-yang-model-innovations>

## YANG Model Innovations

© 2024 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

- 1 Introduction & Overview  
Model Types Overview  
Summaries per Release
- 2 Config and Oper Models
- 3 PIN Specific  
PIN Common  
    WNCD, PUBD  
PIN Switching  
PIN Routing  
PIN Wireless
- 4 Model Abstractions  
EVPN Service Model  
OpenConfig  
IETF
- 5 Custom Cisco  
EXEC Actions  
Telemetry & Telemetry Events  
CLI RPC
- 6 Release Deep Dive  
17.13, 17.14, 17.15

# Cisco IOS XE – OpenAPI Swagger from YANG



“Vibe Coded” using VSCode + Copilot + Sonnet4.5: We wrote the “PRD” product requirements which essentially stated: “Take the YANG modules, sort them into categories, create yang tree, add example data aligned to YANG, then create Swagger API for all possible operations”

A ~2-week project, but after only the first 2 hours it was clear how helpful and useful this project was going to be.

<https://jeremycohoe.github.io/cisco-ios-xe-openapi-swagger/>

Cisco IOS-XE 17.18.1 - Operational State APIs

Real-time operational data via RESTCONF - Read-only GET operations for monitoring and telemetry

HomeOperationalRPCConfigNativeOpenConfigIETFMIBEventsOtherTree Browser

Operational Modules

Search modules...

199 Modules2,652 Paths2,652 Operations

aaaTree

aclTree

app-cflowdTree

app-hostingTree

appqoe-httpTree

appqoeTree

appqoe-servTree

appqoe-sslproxyTree

appqoe-tcpproxyTree

arpTree

aws-cwTree

Cisco-IOS-XE-aaa-operThis module contains a collection of YANG definitions for AAA operational data. Copyright (c) 2017-2019, 2021-2023 by Cisco Systems, Inc. All rights reserved.

GET /data/Cisco-IOS-XE-aaa-oper:aaa-dataGet aaa-data

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-radius-glo

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-radius-pro

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-radius-sta

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-radius-sta port}Get aaa-radius-stats-item

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-tacacs-sta

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-tacacs-staGet aaa-tacacs-stats-item

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-ldap-count

GET /data/Cisco-IOS-XE-aaa-oper:aaa-data/aaa-ldap-countGet aaa-ldap-counters-item

Example ValueSchema

```
{  "install_rpc:install": {    "vpn": "GigabitEthernet1/0/1",    "uuid": "example-string",    "one-shot": "example-string",    "reloadfast": "example-string",    "issu": "example-string",    "download-timeout": "example-string",    "xfsu": "example-string",    "scheduled-start": "example-string",    "scheduled-end": "example-string",    "path": "flash:/config.txt",    "name": "example-string",    "controller-mode-options": {      "vpn": "GigabitEthernet1/0/1"    }  }}
```

GET /data/Cisco-IOS-XE-native:native/hostnameGet hostname

PUT /data/Cisco-IOS-XE-native:native/hostnameSet hostname

PATCH /data/Cisco-IOS-XE-native:native/hostnameUpdate hostname

DELETE /data/Cisco-IOS-XE-native:native/hostnameDelete hostname

Operational Data

Real-time device state and statistics. Consolidated into 16 categories with production-realistic examples.

199 modules16 categories2,652 paths3 quick-starts

Browse Operational →

RPC Operations

Remote procedure calls and device actions. Organized by operation type (network, wireless, security, system, etc.).

58 modules9 categories290 operations

Browse RPC →

Events

Event notifications and telemetry streams. Organized by event type (interfaces, routing, security, platform, etc.).

128 modules10 categories715 notifications

Browse Events →

Native Config

Cisco IOS-XE native configuration model. Organized into 18 categories with production examples and 3 quick-starts.

28 modules18 categories5,267 paths3 quick-starts

Browse Native →

Configuration (CFG)

Device configuration management with full CRUD operations.

39 modules612 paths1,992 ops

Browse Config →

OpenConfig

Vendor-neutral network configuration standards.

41 modules772 paths2,880 ops

Browse OpenConfig →

IETF Standards

RFC-compliant IETF YANG models.

21 modules592 paths1,664 ops

Browse IETF →

MIB Translations

SNMP MIB to YANG translations.

147 modules4,272 paths4,272 ops

Browse MIB →

Other Models

Additional and miscellaneous models.

8 modules287 paths636 ops

Browse Other →





# API

# Programmable Interfaces

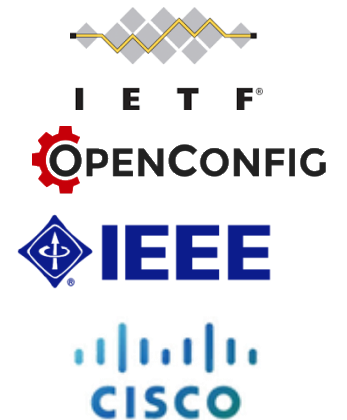
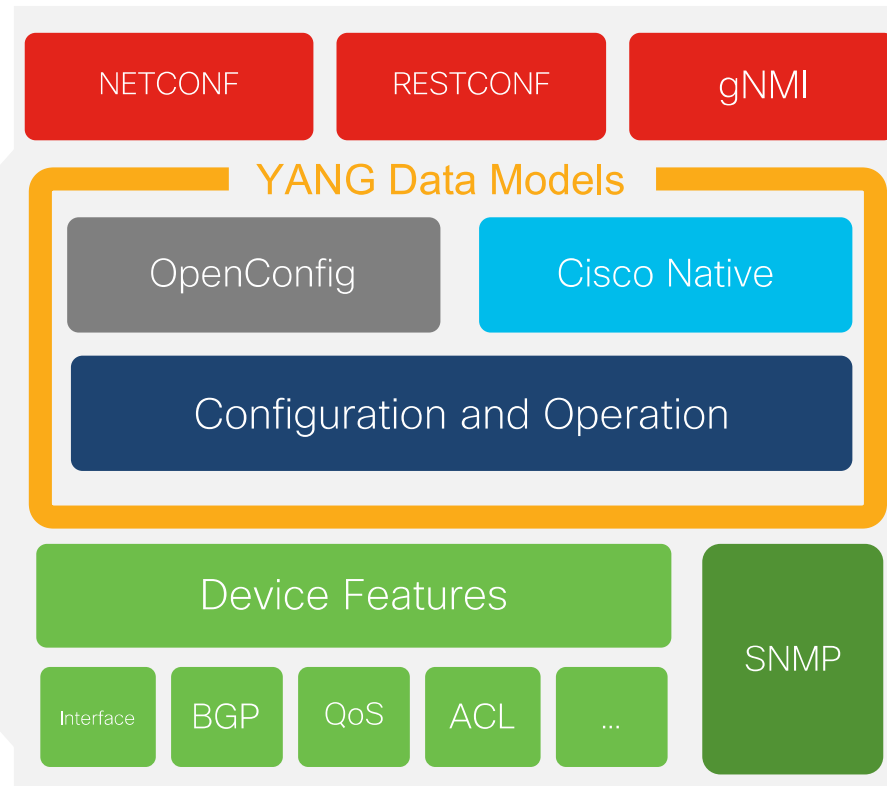
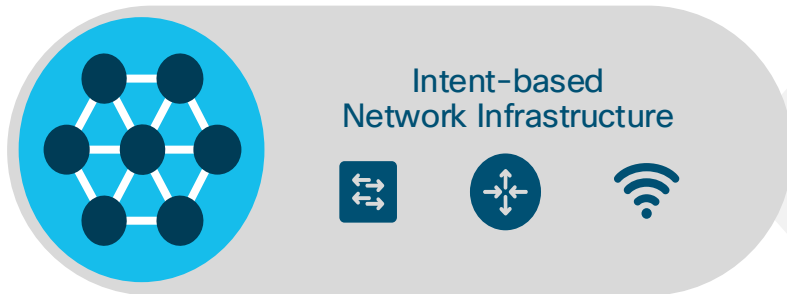
CLI

SNMP

WebUI

The NETCONF, RESTCONF and gNMI are programmatic interfaces that provide **additional** methods for interfacing with the IOS XE device – Just like the CLI, SNMP, and WebUI is used for configuration changes and operational metrics so can the programmatic interfaces of NETCONF, RESTCONF and gNMI

YANG data models define the data that is available for configuration and streaming telemetry





# API Operations

| NETCONF                              | RESTCONF   | gNMI         | gRPC      |
|--------------------------------------|------------|--------------|-----------|
| <get-config>, <get>                  | GET        | GET          | -         |
| <edit-config><br>operation="create"  | PUT, PATCH | SET = update | -         |
| <edit-config><br>operation="replace" | POST       | SET= replace | -         |
| <edit-config><br>operation="delete"  | DELETE     | SET = delete | -         |
| <establish-subscription>             | -          | SUBSCRIBE    | YANG push |

# Model Driven Programmability Interface Comparison

| Protocol Benefits  | NETCONF                                                      | RESTCONF                                                           | gNMI                                                                             |
|--------------------|--------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Minimum IOS XE     | 16.6 (2017)                                                  | 16.7 (2017)                                                        | 16.8 (2018)                                                                      |
| Default Port       | 830                                                          | 443                                                                | 9339                                                                             |
| Operations         | <get>, <get-config>, <edit-config>, <establish-subscription> | GET, POST, PUT, PATCH, DELETE                                      | GET, SET, SUBSCRIBE                                                              |
| Encoding           | XML                                                          | XML or JSON                                                        | RFC7951JSON_IETF + Proto                                                         |
| Security           | SSH + PKI certificate or password                            | HTTPS user/pass                                                    | mTLS certificate with user authentication                                        |
| Transport Protocol | SSH                                                          | HTTPS                                                              | HTTP/2                                                                           |
| Tooling            | YANG Suite, ncclient, Netconf-console                        | YANG Suite, Postman, python, curl                                  | YANG Suite, gnmic, gnmi_cli                                                      |
| Content            | YANG                                                         | YANG                                                               | YANG + Protobuf                                                                  |
| Benefits/Pros      | Mature API with candidate datastores, validation, rollback   | REST is very common API used across industry, well know operations | Single secure API for config + Telemetry                                         |
| Caveats/Cons       | XML can be slow & difficult to use                           | No telemetry support, limited datastore support                    | “New” API, inconsistent implementations across vendors, Go lang, no rollback etc |

# Multiple API's, same YANG data: the choice is yours

Request Interfaces Data

Reply from Device

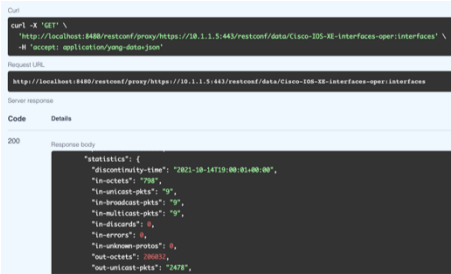
NETCONF

```
<nc:rpc xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:af602e50-6490-4b84-abff-5461a8e37502">
  <nc:get>
    <nc:filter>
      <interfaces xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-interfaces-oper"/>
    </nc:filter>
  </nc:get>
</nc:rpc>

<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
  xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:af602e50-6490-4b84-abff-5461a8e37502">
  <data>
    <interfaces xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-interfaces-oper">
      <interface>
        <name>GigabitEthernet0/0</name>
        <interface-type>iana-iftype-ethernet-csmacd</interface-type>
        <admin-status>if-state-up</admin-status>
        <oper-status>if-oper-state-ready</oper-status>
        <last-change>2025-06-04T16:48:26.731+00:00</last-change>
        <if-in-dex>1</if-in-dex>
        <phys-address>00:50:56:bf:77:ea</phys-address>
        <speed>1000000000</speed>
        <statistics>...</statistics>
      </interface>
    </interfaces>
  </data>
</rpc-reply>
```

RESTCONF

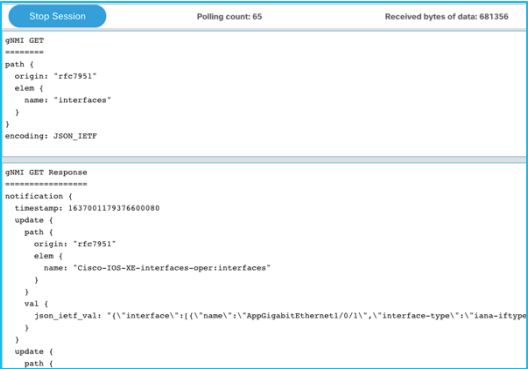
```
curl -X 'GET' \
'http://0.0.0.0:8480/restconf/proxy/https://devnetsandboxiosxc9k.cisco.com:443/restconf/data/Cisco-IOS-XE-interfaces-oper:interfaces/interface' \
-H 'accept: application/yang-data+json'
```



```
{
  "Cisco-IOS-XE-interfaces-oper:interface": [
    {
      "name": "GigabitEthernet0/0",
      "interface-type": "iana-iftype-ethernet-csmacd",
      "admin-status": "if-state-up",
      "oper-status": "if-oper-state-ready",
      "last-change": "2025-06-04T16:48:26.49+00:00",
      "if-index": 1,
      "phys-address": "00:50:56:bf:77:ea",
      "speed": "1000000000",
      "statistics": {
        ...
      }
    }
  ]
}
```

gNMI

```
{
  "path": [
    {
      "origin": "rfc7951",
      "elem": [
        {
          "name": "Cisco-IOS-XE-interfaces-oper:interfaces"
        },
        {
          "name": "interface"
        }
      ]
    },
    {
      "encoding": "JSON_IETF"
    }
  ]
}
```



CLI

show interfaces

```
CAT9k_A0# show interfaces
Vlan1 is up, line protocol is down , Autostate B
Hardware is Ethernet SVI, address is 0050.56bf.77ea
MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
  reliability 255/255, txload 1/255, rxload 1/255
Encapsulation ARPA, loopback not set
Keepalive not supported
ARP type: ARPA, ARP Timeout 04:00:00
Last input never, output never, output hang never
Last clearing of "show interface" counters never
Input queue: 0/375/0/0 (size/max/drops/flushes)
Queueing strategy: fifo
Output queue: 0/40 (size/max)
5 minute input rate 0 bits/sec, 0 packets/sec
5 minute output rate 0 bits/sec, 0 packets/sec
  0 packets input, 0 bytes, 0 no buffer
    Received 0 broadcasts (0 IP multicasts)
  0 runs, 0 giants, 0 throttles
  0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
  0 packets output, 0 bytes, 0 underruns
  Output 0 broadcasts (0 IP multicasts)
  0 output errors, 2 interface resets
  0 unknown protocol drops
  0 output buffer failures, 0 output buffers
```



# CLI to YANG: show run format netconf/restconf

1. This CLI addition to “show run | format” brings additional visibility into the YANG modelled configuration
2. NETCONF with XML and JSON with RESTCONF for Cisco-IOS-XE native models only
3. **Easily convert CLI into YANG** to re-use in tooling, scripts, and automation and orchestration systems

show run | format netconf-xml

show run | format restconf-json

```
C9300#show run | format netconf-xml
<config xmlns="http://tail-f.com/ns/config/1.0">
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
    <version>17.7</version>
    <memory>
      <free>
        <low-watermark>
          <processor>131752</processor>
        </low-watermark>
      </free>
    </memory>
  </native>
</config>
```

```
C9300#show run | format restconf-json
{
  "data": {
    "Cisco-IOS-XE-native:native": {
      "version": "17.7",
      "memory": {
        "free": {
          "low-watermark": {
            "processor": 131923
          }
        }
      }
    }
  }
}
```

```
C9300#
C9300#show run | i netconf-yang
netconf-yang
C9300#
```

- Requires netconf-yang Data Model Interfaces to be enabled
- CLIs with corresponding native YANG and modeled in show run are returned
- No support for “show run all” or additional parameters
- NETCONF “Get-Config” RPC can also get used to get the YANG modelled configuration

# About NETCONF (1/3)

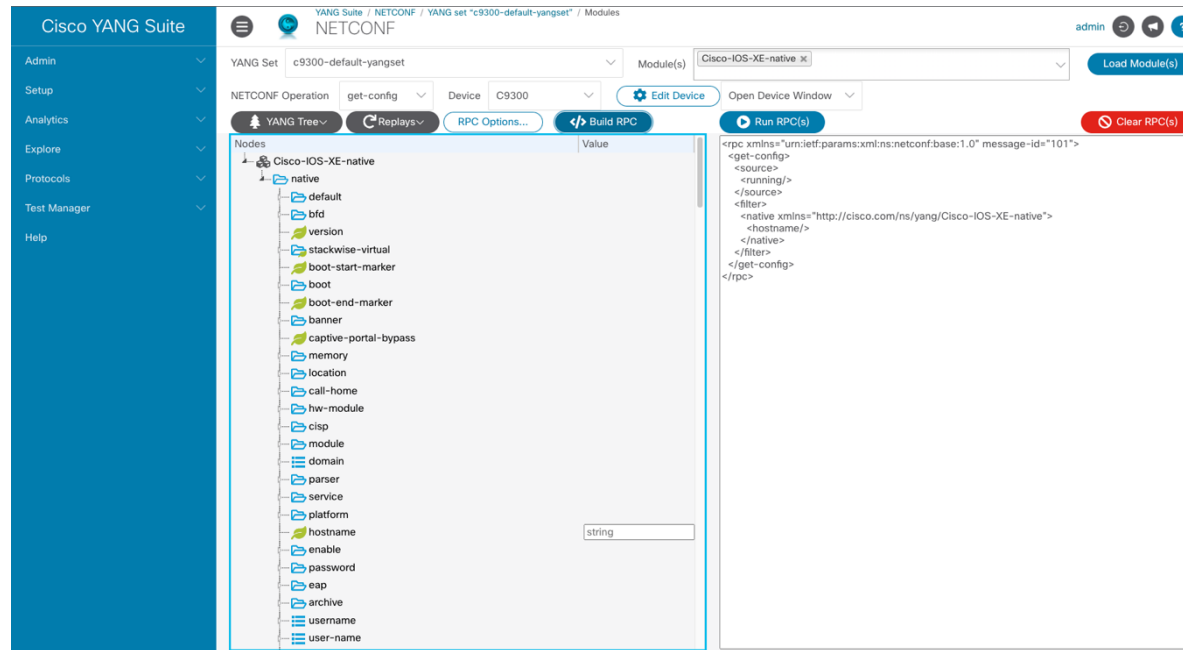
- NETCONF provides a programmatic interface based on standard mechanisms for accessing configuration data, state data, data-model-specific Remote Procedure Call (RPC) operations and events, defined in the YANG model.
- NETCONF uses XML formatting for machine-friendly data transfers and supports the operations described below.
- The YANG Suite NETCONF plugin provides options to execute and visualize the YANG data model.

**Attend the ACR Session on Wednesday to learn more!**

**1:45 PM – 2:45 PM**

BRKDEV-2188

## Atomic Config Replace on Catalyst



## Why use NETCONF?

- Candidate datastore
- Easily convert CLI to YANG using format | Netconf-xml
- NETCONF Access Control Model (NACM)
- Commonly used with Ansible
- Confirm Commit
- Mature RFC standard

## NETCONF RFC Standards

NETCONF is a protocol defined by the IETF to install, manipulate, and delete the configuration of network devices.

The diagram illustrates the evolution of NETCONF and the sequence of operations for a configuration change.

**Evolution of NETCONF:**

- V1.0:**
  - [RFC 4741](#)
  - Base NETCONF Protocol
- V1.1:**
  - [RFC 6241](#)
  - Base NETCONF Protocol
  - [RFC 6242](#)
  - NETCONF over SSH
- Extensions:**
  - [RFC 5277](#)
  - Notifications
  - [RFC 5717](#)
  - Partial Locking
  - [RFC 6243](#)
  - With defaults
  - [RFC 6020](#)
  - YANG

**Sequence of Operations:**

- Transactional** (represented by a circular arrow icon):
  - Either all configuration is applied or nothing
  - Avoids inconsistent state
  - Both at Single Device and Network-wide level
- Error Management** (represented by a crossed-out 'X' icon):
  - OK or error code
- Capability Exchange** (represented by a double-headed arrow icon):
  - `ssh -p 830 admin@10.1.1.5 -s netconf`
- Models Download from a Device** (represented by a download arrow icon):

At the bottom, a timeline shows the years **2006**, **2010**, and **2011** with arrows indicating the progression of time. Below the timeline is the URL: <https://tools.ietf.org/html/rfc6241>.

## NETCONF Operations

| Main Operations                    | Description                                                                   |
|------------------------------------|-------------------------------------------------------------------------------|
| <get> (close to 'show ?')          | Retrieve running configuration and device state information                   |
| <get-config> (close to 'show run') | Retrieve all or part of specified configuration datastore                     |
| <edit-config> (close to 'conf t')  | Loads all or part of a configuration to the specified configuration datastore |
| Other Operations                   | Description                                                                   |
| <copy-config>                      | Replace an entire configuration datastore with another                        |
| <delete-config>                    | Delete a configuration datastore                                              |
| <commit>                           | Copy candidate datastore to running datastore                                 |
| <lock> / <unlock>                  | Lock or unlock the entire configuration datastore system                      |
| <close-session>                    | Graceful termination of NETCONF session                                       |
| <kill-session>                     | Forced termination of NETCONF session                                         |



# About RESTCONF (2/3)

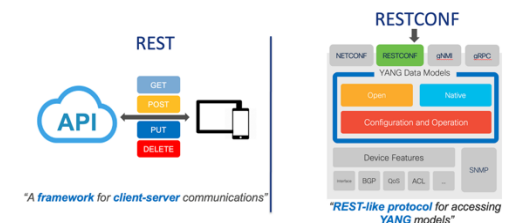
- RESTCONF provides a programmatic interface based on standard mechanisms for accessing configuration data, state data, data-model-specific Remote Procedure Call (RPC) operations and events, defined in the YANG model.
- The YANG Suite RESTCONF plugin provides Swagger UI and execution visualization of the YANG data model.
- Use YANG Suite or Postman tool

## Why use RESTCONF?

- REST-like API
  - Easy to use with edit & get operations
  - Swagger documentation
- Commonly used with cURL commands, JSON-formatting and Python scripting
- Terraform enablement

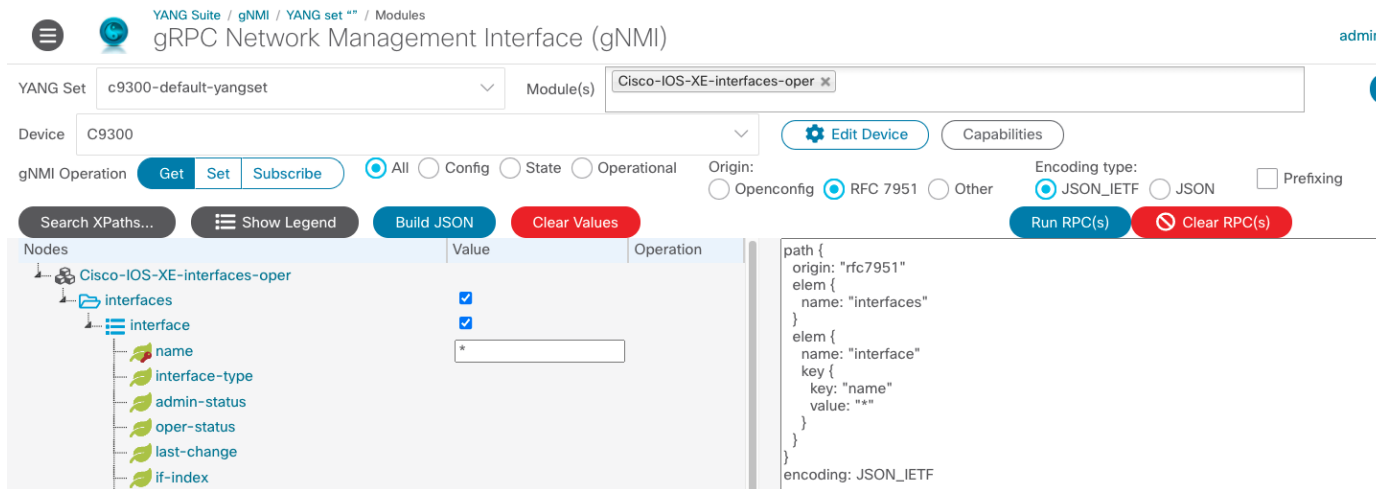
The screenshot shows the YANG Suite RESTCONF interface. On the left, there are input fields for 'Select a YANG set:' (c9300-default-yangset), 'Select a device:' (C9300), 'Select YANG module(s):' (Cisco-IOS-XE-interfaces-oper), and 'Select depth limit:' (No limit). Below these are buttons for 'Load module(s)', 'Generate API(s)', and 'Show API(s)'. A tree view on the left shows the selected module 'Cisco-IOS-XE-interfaces-oper' expanded to show 'interfaces', 'interface', 'name', and 'interface-type'. On the right, the 'OpenAPI v3.0.3' specification is displayed, showing the host destination and a list of endpoints for the 'default' server. The endpoints include PATCH, PUT, POST, GET, and DELETE operations for various YANG data nodes.

REST vs RESTCONF: not the same!



# About gNMI (3/3)

gNMI provides a programmatic interface based on standard mechanisms for accessing configuration data, state data, data-model-specific Remote Procedure Call (RPC) operations and events, defined in the YANG model. gNMI supports GET, SET and SUBSCRIBE operations. The YANG Suite gNMI plugin provides execution and visualization of the YANG data model.



## Why use gNMI?

- gNMI API supports config management through the “SET” operation and includes a “Subscribe” for model-driven telemetry. It also has some features for TLS certificate management, OS version upgrades, and factory-reset API's
- gNMI is quickly becoming the standard API used across network devices because it supports device configuration and telemetry all while its encoding mechanism is just faster and more efficient

There are 450+ “actions” in YANG that are modelled for NETCONF/RESTCONF operations, but **NOT** for gNMI !

TLS certificate management, OS version upgrades, factory reset = Supported

450+ other actions: “clear”, reset, save, copy, ping, etc = NOT Supported

# So which API does Cisco recommend ?

It depends! On the usecase, tooling maturity, NetDevOps investment, the complexity of the problem, etc  
This is a brief introduction... API's will be covered in more detail after Day0/Onboarding

| API      | Pros                                                                                                                                                                                                                                                                                                                                                                                                                | Cons                                                                                                                                                                                                                                                                    |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NETCONF  | <ul style="list-style-type: none"><li>• Candidate datastore capability: Option to work with Running datastore or candidate datastore (a sandbox-like environment for testing and validation on-box before pushing code to the running config)</li><li>• Supports config, retrieving data and telemetry</li><li>• Telemetry data comes faster than SNMP</li><li>• Confirm Commit</li><li>• Standards-based</li></ul> | <ul style="list-style-type: none"><li>• XML is slower than gNMI / gRPC</li><li>• Machine friendly (not human friendly)</li><li>• Requires installing libraries for ease of use (ncclient)</li></ul>                                                                     |
| RESTCONF | <ul style="list-style-type: none"><li>• Works like other REST-based APIs with standard operations like GET, PUT, POST, etc</li><li>• No additional libraries to install for use (REST is widely supported)</li><li>• Standards-based</li><li>• Swagger documentation for use of use</li></ul>                                                                                                                       | <ul style="list-style-type: none"><li>• No telemetry support. If RESTCONF is the only option, aggressive polling through GET requests can be used in place of telemetry</li><li>• HTML/JavaScript is commonly used for REST, which is slower than gNMI / gRPC</li></ul> |
| gNMI     | <ul style="list-style-type: none"><li>• Capabilities for GET, SET, and Subscribe operations</li><li>• Telemetry data comes faster than other APIs or SNMP because gRPC uses protobuf encoding which is more efficient and faster to serialize &amp; deserialize the data</li></ul>                                                                                                                                  | <ul style="list-style-type: none"><li>• Non-standards-based (rather it is operator-led), meaning YANG not always at parity (actions,events,on-change,etc don't work)</li><li>• Machine friendly (not human friendly)</li></ul>                                          |
| gRPC     | <ul style="list-style-type: none"><li>• Fast Push-based telemetry</li></ul>                                                                                                                                                                                                                                                                                                                                         | <ul style="list-style-type: none"><li>• Only telemetry support, meaning no device config or single gets (use gNMI)</li></ul>                                                                                                                                            |

So which API to use ? We see all of these interfaces used depending on requirements, complexity, etc

NETCONF: Best for config management. RESTCONF: Best for developers and ease of use. gNMI: If multi-OS, includes telemetry



# Tooling

YANG Suite

Ansible

Terraform

# Empower your NetOps with Cisco YANG Suite

## Best in class tooling for open programmability

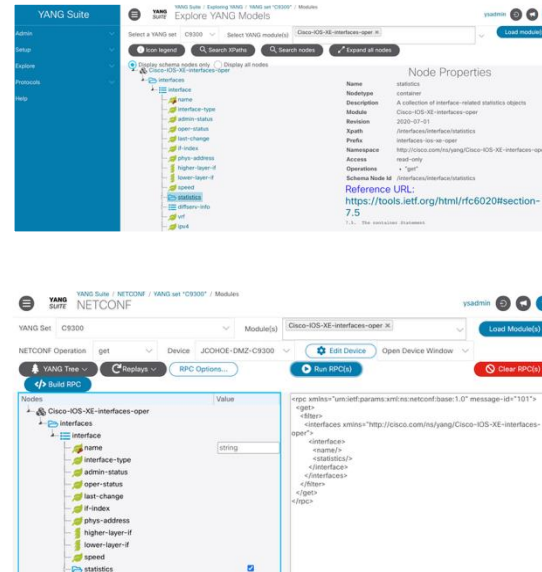
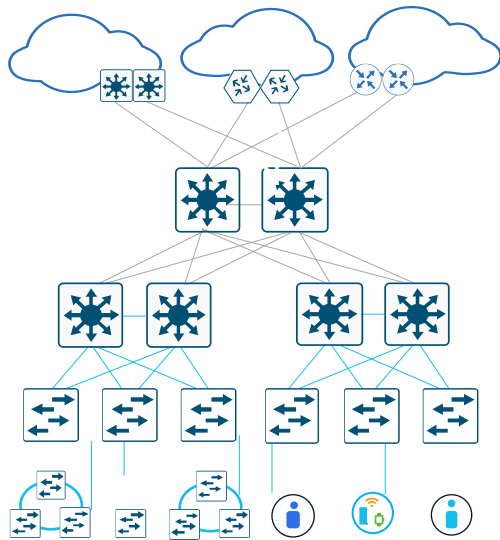
Free and publicly available!

### Complex Network Operations

### Cisco YANG Suite

### Customer Benefits

### Use Cases



**Simplified migration from CLI and SNMP to YANG**



**Toolkit to integrate with existing workflows**



**Guided workflow for all things YANG**



**API Testing and Validation Environment for NETCONF, RESTCONF, gNMI & gRPC**

**Legacy Protocol Migration**

**Device Automation**

**Network Monitoring**

**Compliance and Coverage**

Get hands-on using the new learning lab!

<https://developer.cisco.com/learning/labs/intro-yangsuite/>

Docker container innovation 1 container



<https://developer.cisco.com/yangsuite>

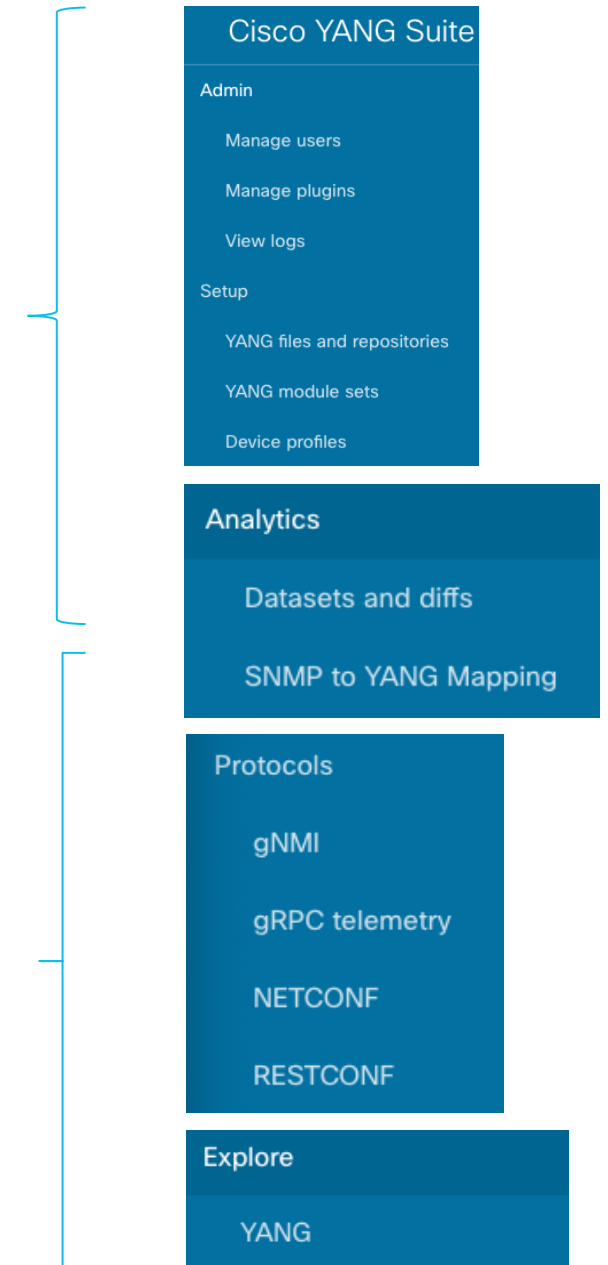
<https://github.com/CiscoDevNet/yangsuite>

## Simplify your device programmability journey with Cisco YANG Suite



# What's Included in YANG Suite

- Initial Release:
  - Plugin and YANG File Manager, Datasets and diffs
  - Device Manager
  - NETCONF (Python), gRPC Telemetry
  - Docker install support with HTTPS
- Second Release:
  - RESTCONF
  - gNMI
  - Python Integrations
- Third Release:
  - gRPC Telemetry with TLS Support
  - **SNMP OID to YANG Xpath Mapping**
  - Ansible Integrations
  - Pip install support



# YANG Suite Learning Lab 2.0 now available



- Learning Lab 2.0 includes interactive guide **and** YANG Suite tool running and available for use
  - Get hands on with YANG Suite using the Introduction to YANG Suite Learning Lab
  - Complete all sections or focus on the features that are most relevant to you!
- <https://devnetapps.cisco.com/learning/labs/intro-yangsuite/introduction-launching-yang-suite/>
  - Credentials for YANG Suite: developer:developer



The screenshot displays the Cisco DevNet Learning Labs Center interface for the 'Introduction to YANG Suite' lab. The sidebar on the left lists 11 steps: 1. Introduction: Launching YANG Suite, 2. Launch the YANG Suite server and connect with your web browser, 3. Configure YANG Suite to connect to a device, 4. Download YANG models from the device, 5. Explore YANG models, 6. Build and send NETCONF messages, 7. Build and send RESTCONF messages, 8. Build and send gNMI messages, 9. Create a gRPC telemetry subscription, 10. SNMP Mapping, 11. Summary. The main content area shows the first step, 'Launch the YANG Suite server and connect with your web browser', with instructions to issue the command 'echo \$DEVENV\_APP\_8080\_URL' in the terminal. A terminal window shows the command and its output: 'https://app-8080-328538164133236256.devnet-stg-us-west-1.devnetcloud.com'. A YANG Suite NETCONF interface is also visible, showing a tree structure for 'Cisco-IOS-XE-interfaces-oper' with nodes like 'interfaces', 'interface', 'name', 'interface-type', 'admin-status', 'oper-status', 'last-change', 'if-index', 'phys-address', 'higher-layer-if', 'lower-layer-if', 'speed', and 'statistics'.

## What's included:

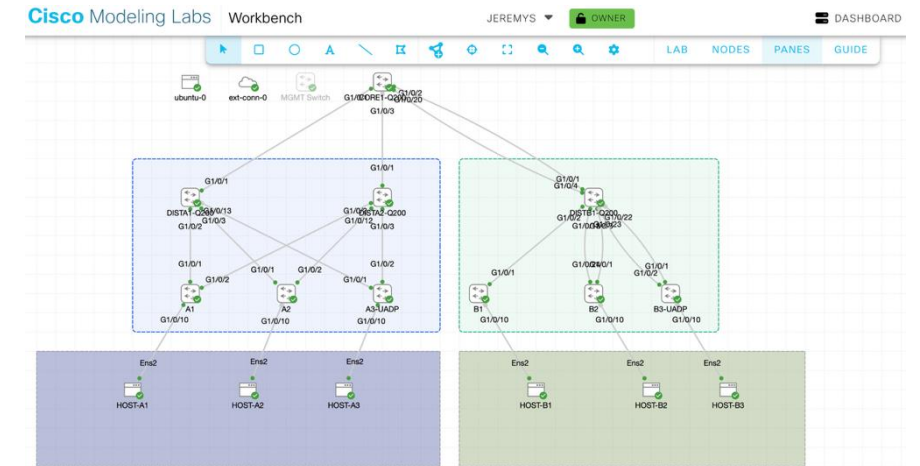
- Access to YANG Suite
- Tutorials on getting started
  - Connect a device to YANG Suite
  - Download YANG models from device
- Explore YANG models
- NETCONF: GET/SET/SUBSCRIBE
  - Generate Ansible & Python
- RESTCONF: GET/SET/SUBSCRIBE
  - Generate Ansible & Python
- gNMI: GET/SET/Subscribe
  - Generate Ansible
- gRPC Telemetry receiver
- SNMP -> YANG mapping
- Datasets & Diffs

<https://developer.cisco.com/yangsuite>  
<https://github.com/CiscoDevNet/yangsuite>

1. YANG API Testing and Validation Environment
2. Construct and test YANG based APIs with NETCONF, RESTCONF, gRPC and gNMI
3. IOS XE / IOS XR / NX OS platforms

# AI tooling introduction

- We have been using AI (VSCode) for Network Management, to build tooling, to operate tooling, etc
- We are not in the production network... but we have high confidence with the tooling in the lab (Specifically CML and C9KV Virtual Switch)
- We also have a secure AI infrastructure to use through Cisco approved for “Highly Confidential” classification data
- So we do not hesitate to ask the LLM to “Use SSH with credentials (username/password) to login to the Switch at IP 10.1.1.1 and troubleshoot any problems” or “Configure EVPN per the design guide” or....
- **Example Prompt:** Read the Cisco Enterprise Campus validated design for 3-tier architecture, and implement it on the following switches: access01-04, core01, dist01-02. Use best practices whenever possible. Provide a simplified readme of what is configured where and how. Validate all services and provide a report with proof showing results.
- We also see success with multiple LLM’s: Describe the problem and requirements to ChatGPT. Ask for a “**handoff prompt**” to share with another LLM that has additional context and details required to implement



SESSION\_HANDOFF\_PROMPT — MERAKI-MANAGED CAMPUS (PARALLEL PROJECT)

## GOAL

Design and implement a Meraki Dashboard-managed campus network using:

- 1x C9500 as CORE
- 1x C9500 as DISTRIBUTION
- 3x C9300 as ACCESS

The design must follow Cisco/Meraki best practices and be suitable for WAN c

## SCOPE

This is a separate project from a classic three-tier campus lab.

This design prioritizes operational simplicity, Meraki Dashboard management,

## HARD CONSTRAINTS

- All devices are managed via Meraki Dashboard (cloud-managed Catalyst)
- Configuration is Dashboard-driven (profiles, networks, VLANs), NOT CLI-fir
- WAN connectivity is required
- Single core, single distribution (acceptable SPOFs)
- Scale is small/medium site (not fabric, not SDA)

## ARCHITECTURE DECISIONS (LOCKED)

### 1. TOPOLOGY

Physical:

[ WAN / ISP ]

# AI Tooling example: Ansible + gNMI

- Ansible with gNMI has previously been limited to the OpenConfig module ecosystem
- Leveraging the AI tooling ecosystem we have built the integration to support the entire YANG module ecosystem
- Prompt: Review the existing ansible-gnmi plugin as part of the nokia.grpc collection, review the YANG RFC, review the gNMI specifications, review the IOS XE Programmability guide chapter gNMI – work to build an integration that supports the entire Cisco YANG modules for Native config, feature config, and all available YANG modules and operations.
- Result: <https://github.com/jeremycohoe/ansible-gnmi>
- Detailed documentation and test cases, validation completed by the AI
- Soon to be published into the CiscoDevNet Ansible collection

### gNMI + Ansible

Requirements

1. Install Ansible
2. Install the required collection

```
ansible-galaxy collection install nokia.grpc
```

```
- name: gNMI playbook
gather_facts: false
hosts: MY_HOST_NAME

collections:
- nokia.grpc

tasks:
- name: gNMI RPC
  gnmi_config:
    update:
      - path: system/config/hostname
        val: set-by-ys-gnmi-ansible

register: testout
- name: dump test output
  debug:
    msg: '{{ testout.output }}'
```

```
auto@pod19-xelab:~/ansible/YANGSuite-ansible$ ansible-galaxy collection install nokia.grpc
Process install dependency map
Starting collection install process
Installing 'nokia.grpc:1.0.2' to '/home/auto/.ansible/collections/ansible_collections/nokia/grpc'
auto@pod19-xelab:~/ansible/YANGSuite-ansible$
```

Note: the ansible gnmi integration works only with OpenConfig model

#### Quick Reference

| Use Case            | Model Type   | Origin  | Path Example                                       | Playbook                       |
|---------------------|--------------|---------|----------------------------------------------------|--------------------------------|
| Get hostname        | OpenConfig   | -       | /system/config/hostname                            | test_device.yml                |
| Get hostname        | Cisco Native | rfc7951 | /Cisco-IOS-XE-native:native/hostname               | test_set_hostname_native.yml   |
| Get memory          | Cisco Native | rfc7951 | /Cisco-IOS-XE-memory-oper:memory-statistics        | test_memory.yml                |
| Get NTP             | OpenConfig   | -       | /system/ntp                                        | test_ntp_config.yml            |
| Get environment     | Cisco Native | rfc7951 | /Cisco-IOS-XE-environment-oper:environment-sensors | test_environmental_sensors.yml |
| Get SNMP MIB        | SNMP         | rfc7951 | /IF-MIB:IF-MIB                                     | test_snmp_mib.yml              |
| Get IETF interfaces | IETF         | rfc7951 | /ietf-interfaces:interfaces                        | test_ietf_interfaces.yml       |
| Set hostname        | OpenConfig   | -       | /system/config/hostname                            | test_set_hostname.yml          |
| Set hostname        | Cisco Native | rfc7951 | /Cisco-IOS-XE-native:native/hostname               | test_set_hostname_native.yml   |

# Ansible with Cisco Catalyst & IOS XE

Ansible has full support for configuration management of Cisco IOS XE and Catalyst 9000 using plugins for CLI, SNMP, and for the YANG based API's including NETCONF, RESTCONF, and gNMI.

Integrations with Cisco IOS XE & Catalyst 9000:

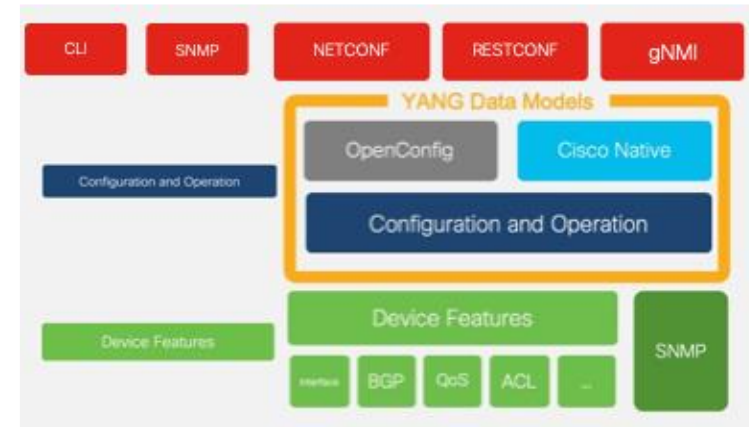
**2 CLI modules** are most common:

|             |              |
|-------------|--------------|
| ios_config  | - config CLI |
| ios_command | - show CLI   |

Various CLI feature modules have been created to support specific usecases including: bgp, l2\_interface, ntp, vlan, vrf, etc

The programmatic API interfaces of **NETCONF/RESTCONF/gNMI** are also supported with a variety of modules and playbooks

DevNet has code samples, learning labs, and sandbox



## Ios

- ios\_banner – Manage multiline banners on Cisco IOS devices
- ios\_bgp – Configure global BGP protocol settings on Cisco IOS

## Netconf

- netconf\_config – netconf device configuration
- netconf\_get – Fetch configuration/state data from NETCONF enabled network devices

## Restconf

- restconf\_config – Handles create, update, read and delete of configuration data on RESTCONF enabled devices
- restconf\_get – Fetch configuration/state data from RESTCONF enabled devices

<https://github.com/ansible-collections/cisco.ios>

[https://docs.ansible.com/ansible/latest/collections/ansible/netcommon/netconf\\_config\\_module.html](https://docs.ansible.com/ansible/latest/collections/ansible/netcommon/netconf_config_module.html)



# Ansible Resources

<https://youtu.be/KIJBE9DVMDA>

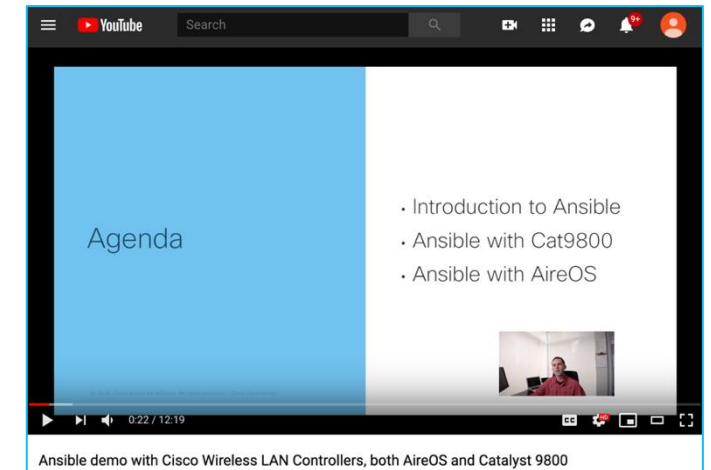
## DevNet:

DevOps video course

<https://developer.cisco.com/video/net-prog-basics/05-netdevops>

Configuration Management

<https://developer.cisco.com/docs/ios-xe/#configuration-management-quick-start-guide>



## Ansible examples on GitHub:

<https://github.com/jeremycohoe/ansible-config-samples>



GitHub



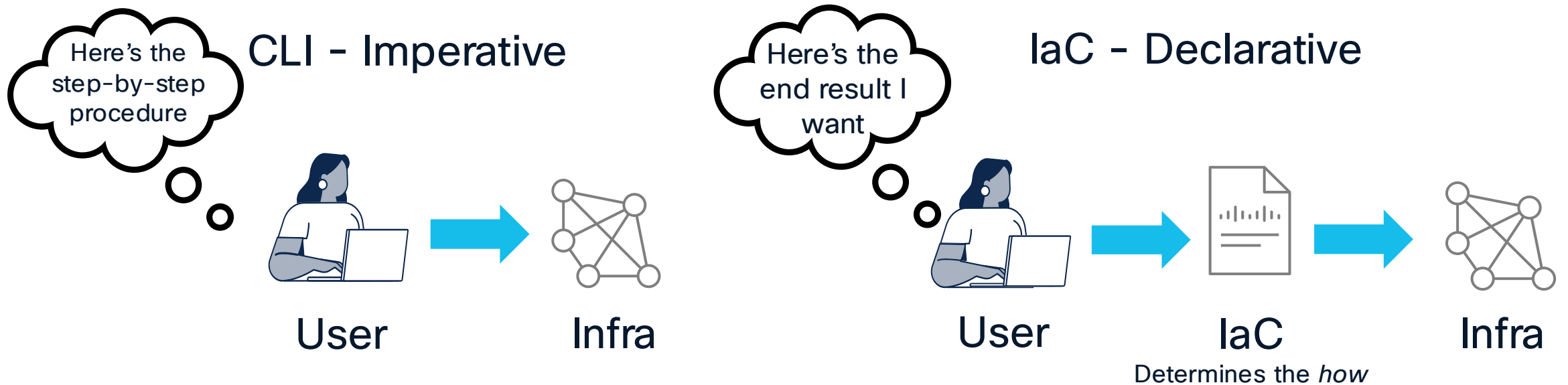
ANSIBLE

## Ansible Documentation for Network Automation:

<https://docs.ansible.com/ansible/latest/network/index.html>

# What's Infrastructure as Code (IaC)?

The *imperative* CLI approach manages configuration using in a step-by-step procedure, while the *declarative* IaC manages configuration through the end state



Learn more about IaC here:

<https://developer.cisco.com/iac/#:~:text=Adopting%20Infrastructure%20as%20Code%20allows,data%20center%20to%20the%20edge>

# Terraform with Cisco Catalyst & IOS XE

The 1 "iosxe" terraform provider manages configuration declaratively across 19 features using 98 resources & data sources  
There are 2 additional resources for imperative configuration using both CLI and YANG  
The imperative resources take YANG modelled inputs and the Cisco-IOS-XE-cli-rpc.YANG is used to abstract CLI over RESTCONF

**Terraform interfaces with the IOS XE RESTCONF API**

NETCONF | RESTCONF | gNMI

YANG Data Models

OpenConfig | Cisco Native

Configuration and Operation

Device Features

Interface | BGP | QoS | ACL | ... | SNMP

**Declarative support**

iosxe provider

- > Guides
- > AAA
- > BFD
- > BGP
- > CTS
- > Crypto
- > EVPN
- > General
- > Interface
- > MDT
- > MPLS
- > Management
- > Multicast
- > OSPF
- > QoS
- > Routing
- > Switching
- > System
- > VRF

**Imperative support**

General

- Resources
  - iosxe\_cli
  - iosxe\_restconf
  - iosxe\_save\_config
- Data Sources
  - iosxe\_restconf

iosxe\_cli

```
resource "iosxe_cli" "example" {
  cli = <<-EOT
  interface Loopback123
  description configured-via-restconf-cli
  EOT
}
```

**CLI RPC YANG Model**

Cisco-IOS-XE-cli-rpc

config-ios-cli-rpc

input

- config-clis
- timeout
- error-patterns

output

- result
- lineno
- error-code
- error-string
- error-tag

**iosxe\_restconf**

```
resource "iosxe_restconf" "nested_list" {
  path = "Cisco-IOS-XE-native:native/ip"
  attributes = {
    source-route = "true"
  }
  lists = [{
    name = "vrf"
    key = "name"
    items = [
      {
        name = "VRF1"
      }
    ]
  }
}]
```

Documentation on registry <https://registry.terraform.io/providers/CiscoDevNet/iosxe/latest>

Source code in Cisco DevNet GitHub repo: <https://github.com/CiscoDevNet/terraform-provider-iosxe/>

RESTCONF guide [https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b\\_1715\\_programmability\\_cg/m\\_1715\\_prog\\_restconf.html](https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b_1715_programmability_cg/m_1715_prog_restconf.html)

CLI RPC YANG API <https://github.com/YangModels/yang/blob/main/vendor/cisco/xe/17121/Cisco-IOS-XE-cli-rpc.yang>

# Terraform example for Model Driven Telemetry

Enable gRPC Dial-Out telemetry subscriptions for the POE and Basic Device Monitoring use cases

```
auto@pod22-xelab:~/mdt_example$ terraform plan -var-file="var.tfvars"
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# iosxe_mdt_subscription.cpu_subs["100"] will be created
+ resource "iosxe_mdt_subscription" "cpu_subs" {
+   encoding          = "encode-kvgpb"
+   filter_xpath      = "/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization/five-seconds"
+   id                = (known after apply)
+   receivers         = [
+     {
+       address = "10.1.1.3"
+       port    = 57500
+       protocol = "grpc-tcp"
+     },
+   ],
+   stream          = "yang-push"
+   subscription_id  = 100
+   update_policy_periodic = 3000
+ }

# iosxe_mdt_subscription.cpu_subs["101"] will be created
+ resource "iosxe_mdt_subscription" "cpu_subs" {
+   encoding          = "encode-kvgpb"
+   filter_xpath      = "/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization/one-minute"
+   id                = (known after apply)
+   receivers         = [
+     {
+       address = "10.1.1.3"
+       port    = 57500
+       protocol = "grpc-tcp"
+     },
+   ],
+   stream          = "yang-push"
+   subscription_id  = 101
+   update_policy_periodic = 3000
+ }
```

## Lab Guide Steps:

1. Introduction
2. IOS XE CLI pre-req
3. Terraform Install & headers, device variables
4. Variables and config for MDT subscription
5. Terraform configuration for device monitoring
6. TF Workflow: init, plan, apply & destroy
7. Validation with TF and CLI
8. Conclusion

## iosxe\_mdt\_subscription (Resource)

This resource can manage the MDT Subscription configuration.

### Example Usage

```
resource "iosxe_mdt_subscription" "example" {
  subscription_id = 101
  stream         = "yang-notif-native"
  encoding       = "encode-kvgpb"
  source_vrf     = "Mgmt-vrf"
  source_address = "1.2.3.4"
  update_policy_on_change = true
  filter_xpath   = "/ios-events-ios-xe-oper"
  receivers = [
    {
      address = "5.6.7.8"
      port    = 57600
      protocol = "grpc-tcp"
    }
  ]
}
```

[https://registry.terraform.io/providers/CiscoDevNet/iosxe/latest/docs/resources/mdt\\_subscription](https://registry.terraform.io/providers/CiscoDevNet/iosxe/latest/docs/resources/mdt_subscription)

[https://registry.terraform.io/providers/CiscoDevNet/iosxe/latest/docs/data-sources/mdt\\_subscription](https://registry.terraform.io/providers/CiscoDevNet/iosxe/latest/docs/data-sources/mdt_subscription)

<https://github.com/jeremycohoe/cisco-ios-xe-panda-lab-terraform>

# The power of Terraform's hosts variable file

Similar to Ansible the grouping of hosts and variables can be added to Terraform for management at scale

```
locals {
  core_switches = {
    "core-sw-01.example.com" = {
      ip          = "10.10.0.1"
      device_type = "cisco_ios"
      ansible_user = "netops"
    }
    "core-sw-02.example.com" = {
      ip          = "10.10.0.2"
      device_type = "cisco_ios"
      ansible_user = "netops"
    }
  }

  access_switches = {
    "access-sw-01.example.com" = {
      ip          = "10.10.1.1"
      device_type = "cisco_ios"
      ansible_user = "netops"
    }
    "access-sw-02.example.com" = {
      ip          = "10.10.1.2"
      device_type = "cisco_ios"
      ansible_user = "netops"
    }
  }
}
```

```
routers = {
  "router-edge-01.example.com" = {
    ip          = "10.10.4.1"
    device_type = "cisco_ios"
    ansible_user = "netops"
  }
  "router-edge-02.example.com" = {
    ip          = "10.10.4.2"
    device_type = "cisco_ios"
    ansible_user = "netops"
  }
  "router-dist-01.example.com" = {
    ip          = "10.10.5.1"
    device_type = "cisco_ios"
    ansible_user = "netops"
  }
  "router-dist-02.example.com" = {
    ip          = "10.10.5.2"
    device_type = "cisco_ios"
    ansible_user = "netops"
  }
}
```

```
firewalls = {
  "fw-edge-01.example.com" = {
    ip          = "10.10.2.1"
    device_type = "fortinet"
    ansible_user = "fwadmin"
  }
  "fw-edge-02.example.com" = {
    ip          = "10.10.2.2"
    device_type = "fortinet"
    ansible_user = "fwadmin"
  }
}

loadbalancers = {
  "lb-01.example.com" = {
    ip          = "10.10.3.1"
    device_type = "f5"
    ansible_user = "lbadmin"
  }
  "lb-02.example.com" = {
    ip          = "10.10.3.2"
    device_type = "f5"
    ansible_user = "lbadmin"
  }
}
```



# They can coexist... it's not an either/or story



**Terraform**

Manage infrastructure as code

<https://developer.hashicorp.com/terraform>

- Terraform keeps state locally
- Terraform knows what is configured vs desired end-state
- Terraform can automatically destroy/recreate resources



**Red Hat**  
Ansible Automation  
Platform

<https://www.redhat.com/en/ansible-collaborative>

- Ansible mutates the infrastructure
- Ansible does not keep state (mostly)
- Terraform can call Ansible to perform tasks post resource deployment (e.g., on VMs deployed)

# Tooling Comparison

This is not a complete list!

Other common tools: Postman(REST), gNMIc, Python, GoLang, <https://httpie.io>

| Tool                | YANG Suite                                                                             | Ansible                                | Terraform                            |
|---------------------|----------------------------------------------------------------------------------------|----------------------------------------|--------------------------------------|
| Use Case            | To understand how YANG models work<br>To help build a payload                          | Task-based tooling                     | Infrastructure-as-Code (IaC) tooling |
| Supported Protocols | NETCONF, RESTCONF, gNMI, gRPC (and more capabilities such as diffs, SNMP to YANG, etc) | NETCONF, RESTCONF                      | RESTCONF                             |
| Benefits            | Getting started to understand YANG models and build payloads                           | Imperative – procedural (step-by-step) | Declarative                          |
| Interface           | GUI web tool                                                                           | CLI / YAML files                       | CLI / HCL (similar to JSON) files    |

# Device Onboarding

I just received 5000 switches. How can I onboard them reliably, efficiently and at scale?





# Why It's Time for a Change?

## Limitations of Manual Network Provisioning



Time Consuming &  
Error prone



Limited Scalability with  
High Operational Costs



Security Risks

AI Generated Image  
Prompt: A real person in a server room, surrounded by racks of switches and servers. The person has connected their laptop to one of the racks with a wire and is intently working on their laptop. The server room is well-lit with a cool, blue-toned lighting, reflecting off the metallic surfaces of the servers. The person is focused, with a look of concentration on their face, as they navigate through their laptop. The background shows rows of servers, with blinking lights indicating active connections and data transfer.

# Day – 0 Network Automation

## Zero-Touch Provisioning

ZTP Server



15%

Organizations identified a supply chain compromise as the source of a data breach [1]

\$4.8 Million

Is the global average cost of a data breach, increased by 10% compared to the previous year [1]



AI Generated Image  
Prompt: A detailed illustration of an enterprise network, featuring a large main campus building at the center, surrounded by only 4 to 5 smaller branch office buildings. The network connections between the main campus and branch offices are clearly depicted, with lines representing data flow. The main campus is modern and sleek, with glass windows and a well-maintained exterior. The branch offices are smaller but share a similar architectural style. The background includes a subtle cityscape, with a clear blue sky and a few scattered clouds.

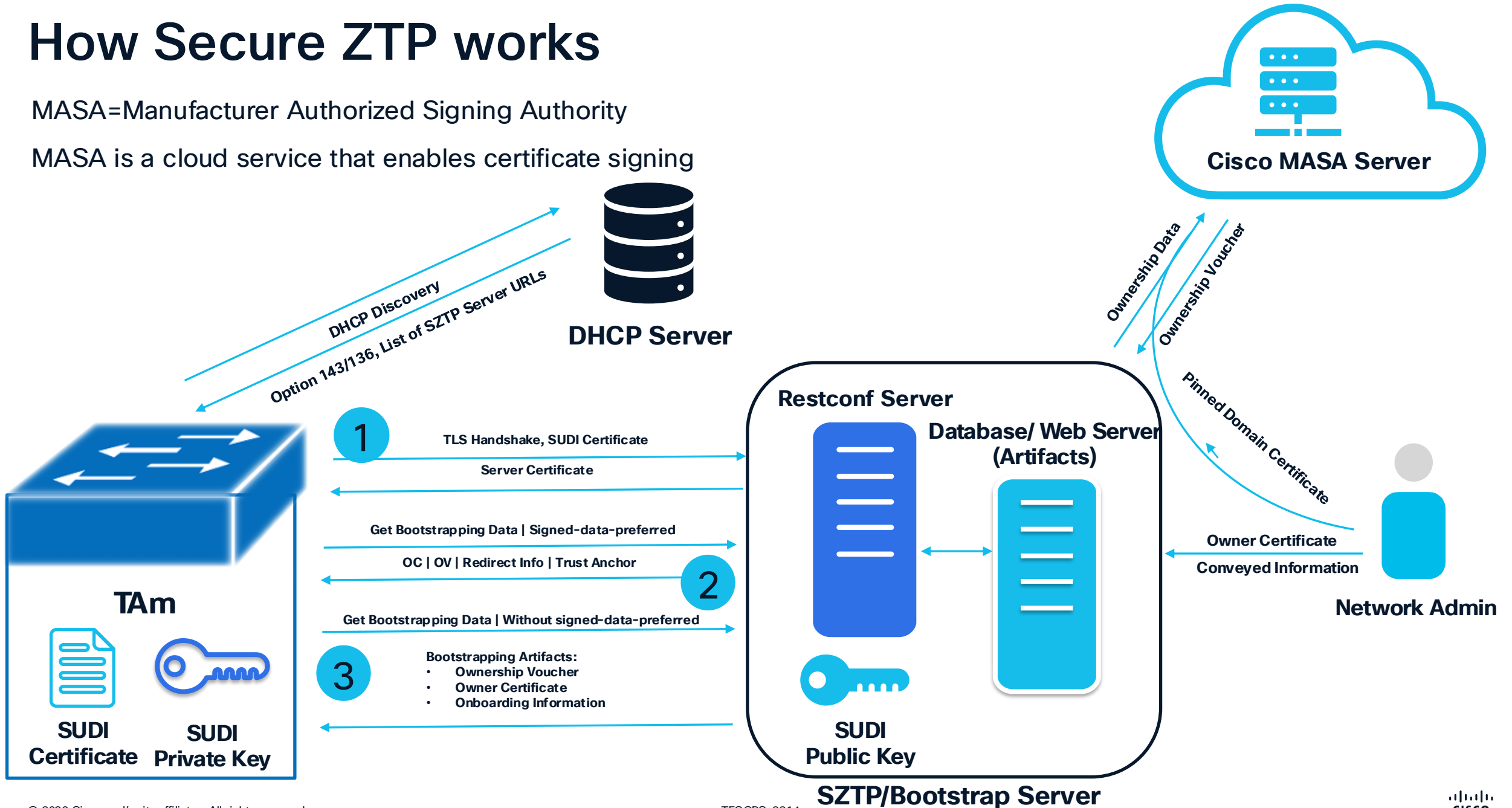
[1] - Cost of a Data Breach Report 2023/24 - IBM



# How Secure ZTP works

MASA=Manufacturer Authorized Signing Authority

MASA is a cloud service that enables certificate signing



# Manual Provisioning vs ZTP vs SZTP

|                           | Manual Provisioning                                                                   | ZTP                                                                                   | Secure - ZTP                                                                          |
|---------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Provisioning Automation   |    |    |    |
| Configuration Consistency |    |    |    |
| Scalability               |    |    |    |
| End to End Security       |   |   |   |
| Audit Logs                |  |  |  |
| Easy Image Upgrade        |  |  |  |

# Learn more about Secure Zero Touch Provisioning

## SZTP Blog



Developer  
Building Trust from the Ground Up: The Role of Secure ZTP in Zero Trust Networks  
6 min read  
Ashil Parekh

## Cisco SZTP Guide

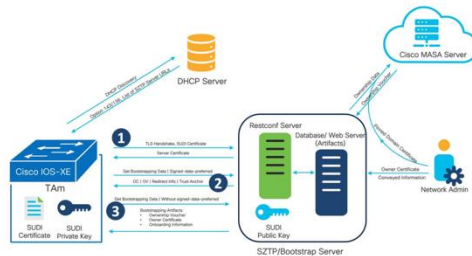


## ZTP guide for hands-on lab



### How does Secure ZTP work?

Secure ZTP employs three-step validation, including device validation, server validation, and artifact validation, to securely onboard the device. The diagram provided below illustrates the various steps involved in the device onboarding and provisioning process within a secure ZTP framework. Let's take a closer look at each of these steps:



<https://blogs.cisco.com/developer/secureztp01>

### Secure Zero Touch Provisioning (Secure ZTP or SZTP) with Cisco Catalyst Guide

#### Secure Boot

We use secure boot all the time when onboarding new devices. The goal is to ensure that the software installed on our devices is trusted and has not been tampered with or manipulated by an outside adversary. Ultimately, we want to securely onboard devices into our network. The biggest benefit of SZTP is onboarding devices securely. Additionally, a major benefit is that once the SZTP components are properly prepped, a non-technical person can easily rack and stack the device to efficiently bring the device online.

#### Secure Zero Touch Provisioning Use cases

1. Onboarding devices that have network access and can reach an external server for bootstrapping data
2. Onboard devices are in an air-gapped network where the bootstrapping server requires all bootstrapping data to be within that network

<https://github.com/sdeweese/sztp>

### Module: Zero Touch Provisioning

In this module, you will verify and confirm the prerequisites for Zero Touch Provisioning (ZTP), the feature of IOS XE on the Catalyst 9300 switch. At the end of this module, you will issue the 'write erase' command, reload the switch, and watch as the ZTP process completes and the switch is configured programmatically and automatically.

What is ZTP? When a device that supports Zero-Touch Provisioning boots up, and does not find the startup configuration (during initial installation), the device enters the Zero-Touch Provisioning mode. The device searches for an IP from a DHCP server and bootstraps itself by enabling the Guest Shell. The device then obtains the IP address or URL of an HTTP/FTP server, and downloads a Python script from an server to configure the device.

<https://github.com/sdeweese/CLUS22-LTROPS-1836-programmability-and-automation/blob/main/ZTP2.md>

## ZTP Script repository

<https://github.com/jeremycohoe/IOSXE-Zero-Touch-Provisioning>

# ZTP Case Study 1: Combined XE & NX Onboarding

Goal: Onboard Cisco IOS XE switches using ZTP and Nexus devices using POAP



<https://github.com/jeremycohoe/IOSXE-Zero-Touch-Provisioning/blob/main/c9300l/combined-simple.py>



2500+ C9300-L  
Switches



Combined IOS XE &  
Nexus Onboarding script

## Script Highlights:

- ZTP/POAP script feature:
  - Apply custom CLI's (enable 3<sup>rd</sup> party optics)
  - Using NETCONF, upgrade device to latest specified code version, reboot if needed
  - Copy completed pre-generated config into running config
  - Use Option81 to learn hostname, hostname set from Source of Truth with pre-generated config available

```
IOSXE-Zero-Touch-Provisioning / c9300l / combined-simple.py
jeremycohoe Create combined-simple.py

Code Blame 247 lines (215 loc) · 6.86 KB

1  #!/usr/bin/env python3
2  """
3  Unified Day0 Onboarding Script for Cisco Nexus (POAP) and Catalyst XE (ZTP)
4  This script auto-detects the device type and applies the appropriate configuration.
5  """
6
7  import sys
8  import os
9  import json
10 import re
11 import time
12
13 # Try to import device-specific modules
14 try:
15     import cli # Available on IOS-XE ZTP
16     DEVICE_TYPE = "IOS-XE"
17 except ImportError:
18     try:
19         from cisco import system # Available on NX-OS POAP
20         DEVICE_TYPE = "NX-OS"
21     except ImportError:
22         # Fallback detection method
23         DEVICE_TYPE = None
24
25
```



# ZTP Case Study 2: C9200CX XE Onboarding

Goal: Onboard Cisco IOS XE C9200CX switches using ZTP for image upgrades



3000+ C9200CX  
Switches



Script leverages NETCONF  
RPC for image upgrades

## Script Highlights:

- Copies new code version to flash
- Copies pre-generated config into running from http, saves config
- Using EEM, install the latest specified code version (if already at this version, install will not proceed)
- Reboot, if needed, as part of install
- Leverages NETCONF RPC to trigger install operation

```
IOSXE-Zero-Touch-Provisioning / code-upgrade / ztp-upgrade-netconf.py
jeremycchoe Rename zip-upgrade-netconf.py to ztp-upgrade-netconf.py
Code Blame 699 lines (568 loc) · 23.5 KB
1  #!/usr/bin/env python3
2  """
3  ZTP NETCONF Software Upgrade Script for Cisco Catalyst 9000 Series
4  """
5
6  Description:
7  This script performs automated software upgrades using NETCONF RPCs
8  during Zero-Touch Provisioning (ZTP) on Cisco Catalyst 9000 switches.
9
10 The script will:
11 - Verify current IOS XE version supports NETCONF (17.6+)
12 - Download target image from HTTP server if not present
13 - Verify MD5 checksum of the image
14 - Execute NETCONF install RPC to trigger upgrade
15 - Device will automatically reload and apply new image
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55 # =====
56 # CONFIGURATION - MODIFY THESE VALUES FOR YOUR ENVIRONMENT
57 # =====
58
59 # Target software version and image details
60 TARGET_VERSION = '17.18.01'
61 TARGET_IMAGE = 'usbflash1:/cat9k_iosxe.17.18.01.SPA.bin'
62 TARGET_IMAGE_MD5 = '3d95fc4dffa160507e927612ce38a7ee'
63
64 # HTTP server for image downloads
65 HTTP_SERVER = '10.1.1.3' # Your HTTP server IP
66 HTTP_PATH = '/' # Path to image file on server
67
68 # NETCONF connection parameters (localhost via guestshell - No need to change these defaults)
69 NETCONF_HOST = '127.0.0.1'
70 NETCONF_PORT = 830
71 NETCONF_USER = 'guestshell'
72 NETCONF_PASS = '' # SSH key authentication used
73 NETCONF_KEY = '/home/guestshell/.ssh/id_rsa_netconf'
74
```

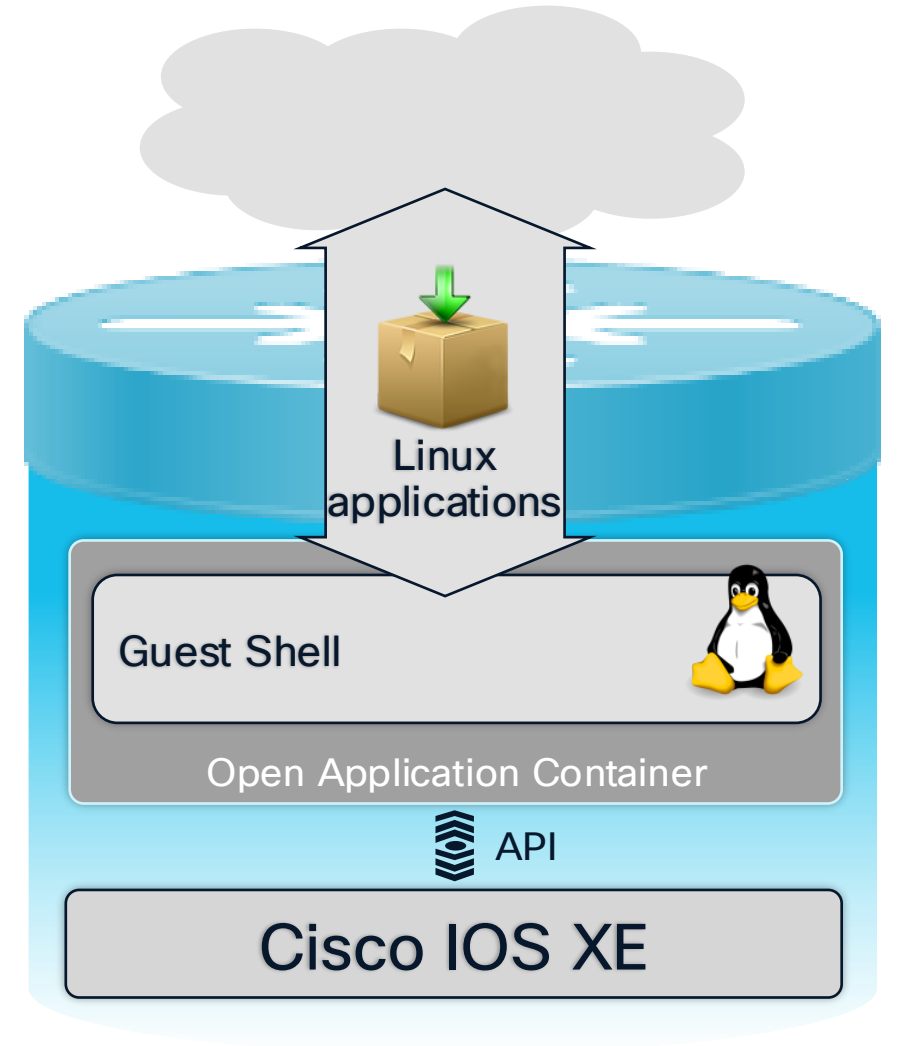


<https://github.com/jeremycchoe/IOSXE-Zero-Touch-Provisioning/blob/main/c9300I/combined-simple.py>

# Guest Shell & EEM

# Linux Shell Environment on Cisco IOS XE devices

1. Fault/resource isolation
2. Secure Linux shell environment
3. Python interpreter, Bash shell
4. PIP Package Manager
5. Integrated with ZTP and EEM
6. Disabled by default
7. Enabled then disabled during ZTP





# EEM: Embedded Event Manager

On-box capabilities for automation

Can run Python, Bash/Guestshell apps, NETCONF/YANG scripts, etc

Can take data from IOS XE and publish to the telemetry subsystems

```
conf t
event manager applet log_ios_version_boot
  event syslog pattern "SYS-5-RESTART"
  action 1.0 cli command "enable"
  action 2.0 cli command "show version | include ^Cisco IOS XE Software"
  action 3.0 syslog msg "Version: $_cli_result"
end
```

```
enable
configure terminal
event manager applet catchall
event cli pattern ".*" sync no skip no
action 1 syslog msg "$_cli_msg"
end
```

Simple exmples: <https://github.com/jeremycohoe/cisco-catalyst-eem-examples>

Log all CLI's to Syslog: <https://github.com/jeremycohoe/eem-catch-all>

Advanced Usecases: <https://github.com/AirManic/Cisco-IOS-EEM-script-playground>

[https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b\\_1715\\_programmability\\_cg/m\\_1715\\_prog\\_eem\\_python.html](https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b_1715_programmability_cg/m_1715_prog_eem_python.html)

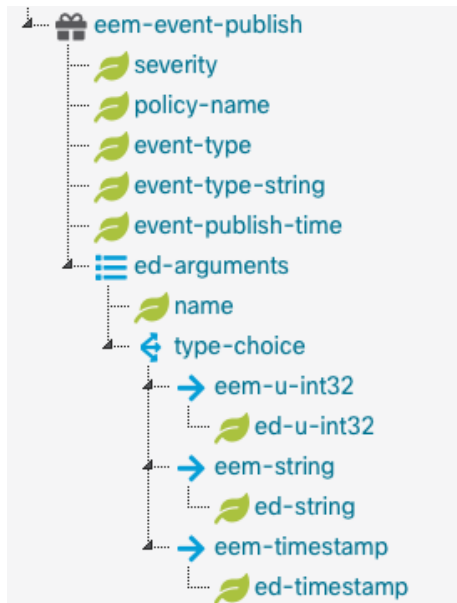
- EEM config part of Cisco Native YANG module EEM section
- The oper module contains the Embedded Event Manager (EEM) operational data

```
jcohoe-c9300#sh run | s event manager
event manager applet catchall
event cli pattern ".*" sync no skip no
action 1 syslog msg "$_cli_msg"
```

```
{,
"event": {
  "Cisco-IOS-XE-eem:manager": {
    "applet": [
      {
        "name": "catchall",
        "event": {
          "cli": {
            "pattern": ".*",
            "sync": "no",
            "skip": "no"
          }
        }
      }
    ],
    "action-config": {
      "action": [
        {
          "name": "1",
          "syslog-option": {
            "msg": "$_cli_msg"
          }
        }
      ]
    }
  }
}
```

|                                                                                                          |                    |                                                      |
|----------------------------------------------------------------------------------------------------------|--------------------|------------------------------------------------------|
|  Cisco-IOS-XE-eem-oper | <b>Name</b>        | eem-oper-data                                        |
|  eem-oper-data         | <b>Nodetype</b>    | container                                            |
|  eem-policy-info      | <b>Description</b> | This is top level container for EEM operational data |
|  index                | <b>Module</b>      | Cisco-IOS-XE-eem-oper                                |
|  name                 | <b>Revision</b>    | 2023-11-01                                           |
|  event-type           | <b>Xpath</b>       | /eem-oper-data                                       |
|  class                | <b>Prefix</b>      | eem-ios-xe-oper                                      |
|  type                 | <b>Namespace</b>   | http://cisco.com/ns/yang/Cisco-IOS-XE-eem-oper       |
|  trap                 | <b>Name</b>        | class                                                |
|  time                 | <b>Nodetype</b>    | leaf                                                 |
|                                                                                                          | <b>Datatype</b>    | eem-ios-xe-oper:eem-policy-class                     |
|                                                                                                          | <b>Basetype</b>    | enumeration                                          |
|                                                                                                          | <b>Description</b> | Policy class of embedded event manager policy        |
|                                                                                                          | <b>Module</b>      | Cisco-IOS-XE-eem-oper                                |
|                                                                                                          | <b>Revision</b>    | 2023-11-01                                           |
|                                                                                                          | <b>Xpath</b>       | /eem-oper-data/eem-policy-info/class                 |
|                                                                                                          | <b>Prefix</b>      | eem-ios-xe-oper                                      |
|                                                                                                          | <b>Namespace</b>   | http://cisco.com/ns/yang/Cisco-IOS-XE-eem-oper       |





Usecase: Capture the CLI's that are sent from the VTY's and (1) send to syslog and (2) export to telemetry

1. The EEM Event Publish capability is part of the Cisco-IOS-XE-ios-events-oper:YANG module for on-change telemetry notifications
2. This feature exports the event specific data of the EEM policy using YANG notification to the external telemetry collector
3. The variables exported in "key:value" pair format for the external telemetry collector to use
4. Example: If the EEM applet script detects certain percentage packet loss on an interface, then a custom message can be added to notify the loss

```
no telemetry ietf subscription 12123
telemetry ietf subscription 12123
encoding encode-kvgpb
filter xpath /ios-events-ios-xe-oper:eem-event-publish
source-address 10.85.134.65
stream yang-notif-native
update-policy on-change
receiver ip address 10.85.134.66 57500 protocol grpc-tcp
```

```
Cisco-IOS-XE-ios-events-oper:eem-event-publish,
host=jcohoe-ubuntu,
path=Cisco-IOS-XE-ios-events-oper:eem-event-publish,
source=jcohoe-c9300,
subscription=12123, collector_timestamp=17272021233690
```

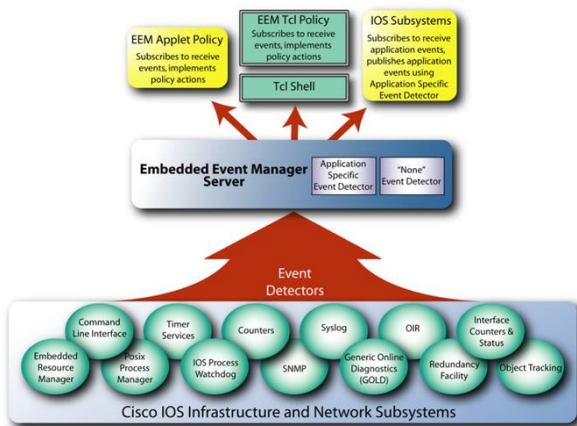
```
severity="major",
policy_name="catchall",
event_type="eem-ed-cli",
event_type_string="cli",
event_publish_time="2024-09-24T18:22:03.366000+00:00",
ed_arguments/name="_cli_msg",
ed_arguments/ed_string="show clock"
```

```
jcohoe-c9300#sh clock
11:22:03.367 PDST Tue Sep 24 2024
jcohoe-c9300#
```

CLI:  
Switch(config-applet)#action 1.0 export-to-telemetry  
EEM TCL action:  
action\_export\_to\_telemetry  
EEM python action:  
action\_export\_to\_telemetry()

```
event manager applet catchall
event cli pattern ".*" sync no skip no
action 1 syslog msg "$_cli_msg"
action 2 export-to-telemetry $_cli_msg
```

```
,ed_arguments/ed_string="enable " 172720235
,ed_arguments/ed_string="terminal length 0"
,ed_arguments/ed_string="terminal width 0"
,ed_arguments/ed_string="show license udi "
,ed_arguments/ed_string="disable " 17272024
,ed_arguments/ed_string="logout " 172720248
```



## **Break time**

**Feel free to stretch, grab a coffee or snack**

## **Resuming at 11:00 with**

**Device Configuration, Device Monitoring,  
Device Optimization & Resources**



# Device Configuration

How can I configure all  
5000 new switches reliably,  
efficiently and at scale?



# Why It's Time for a Change?

- The Complexity and Struggles of CLI-Based Automation



Creating  
Configurations That  
Just Work



Error Handling &  
Rollback



Scalability & Operational  
Efficiency

# Why It's Time for a Change?

## Creating Configurations That Just Work

1. Declarative Config Management



Terraform

2. Two-Phase Commit



Verify before apply

## Error Handling & Rollback

1. Atomic Config Replace



Transactional Config Change

2. Multiple Rollback Options



Instant Rollback  
Post Deployment Rollback

## Scalability & Operational Efficiency

1. Programmable Interfaces



APIs & Yang Models

2. Infrastructure as Code





# Introducing Atomic Configuration Replace

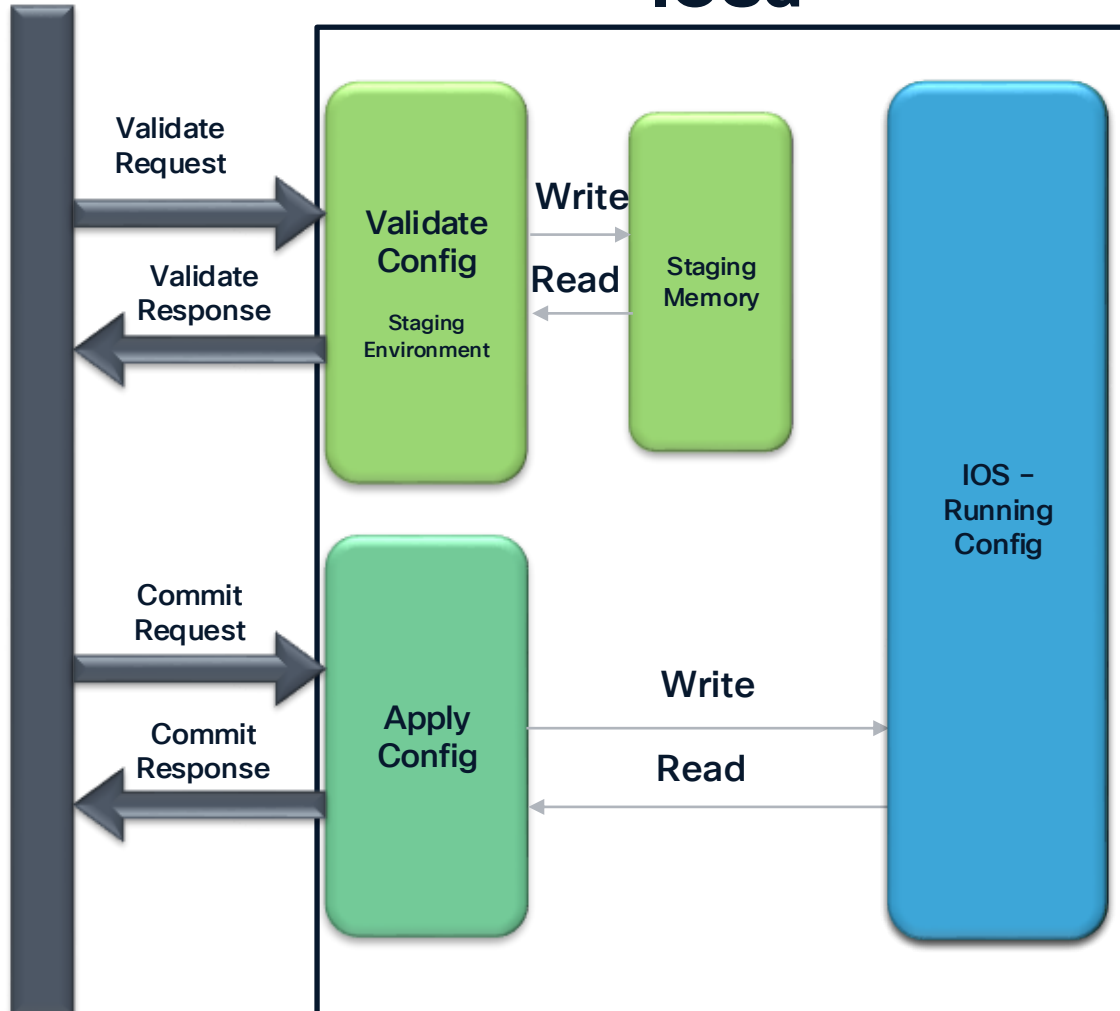
Attend the ACR Session  
Wednesday to learn more!

1:45 PM – 2:45 PM

**BRKDEV-2188**

Atomic Config Replace on  
Catalyst: A New Era of Network  
Management

## IOSd



### Validate Configuration

System verifies configuration  
integrity before applying.  
Catches potential  
errors early



### Atomic Transaction

Apply configuration changes  
as a single transaction. All  
changes succeed,  
or none do



### Config Rollback

If issues arise, revert to last  
known good configuration  
immediately

# ACR: Comment, Diff/Preview, Rescue/Rollback

New YANG data models for these features have been introduced to support additional useability options

1. Config Diff: Show the difference between the running and candidate so we know what is being applied
2. Config Comment: Add context to the config change operation with a comment
3. Config Rollback: Mark the config as rescue and ability to roll back by ID

## **(1) Config Diff & Preview**

Cisco-IOS-XE-cli-preview-rpc.yang

```
rpc candidate-preview {  
  description  
    "Generate a preview of the contents of the candidate datastore as  
    configuration CLI text.";  
  input {  
    uses ios-xe-cli-preview:candidate-preview;  
  }  
  output {  
    uses ios-xe-cli-preview:candidate-preview-output;  
  }  
}
```

## **(3) Rescue Config & Rollback Config:**

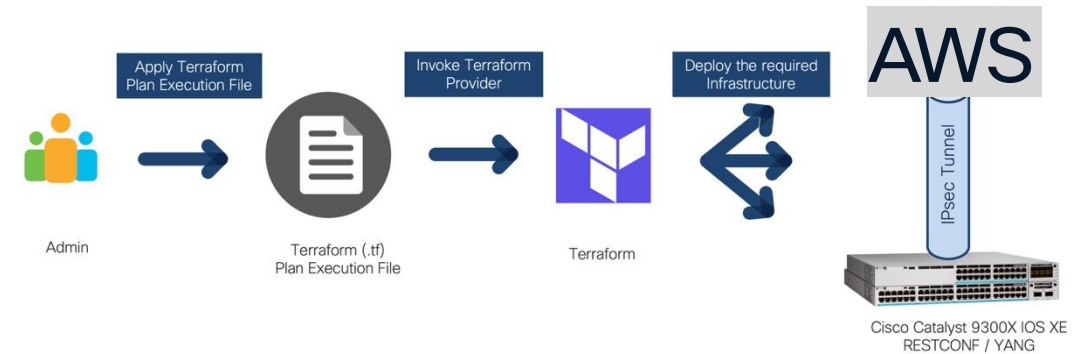
```
<rescue-config-save xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-rescue-config-rpc">  
</rescue-config-save>
```

## **(2) Comment Example:**

```
<config>  
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">  
    <ip>  
      <access-list>  
        <standard xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-acl">  
          <name>Test-ACL-1 </name>  
        </standard>  
      </access-list>  
    </ip>  
  </native>  
</config>  
<rollback-comment xmlns="http://tail-f.com/ns/netconf/rollback">  
  ACL created  
</rollback-comment>
```

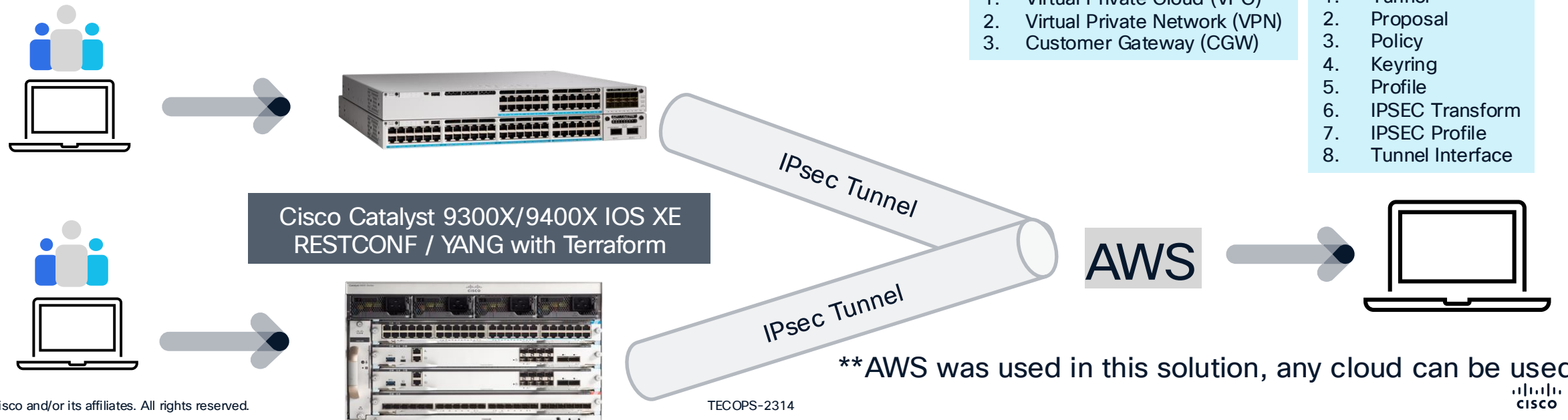
# IPsec tunnel + cloud automation with Terraform

1. Terraform configures the IPsec tunnel between the 9300X and the cloud service where the internal resources are available
2. Terraform also manages the cloud-native resources including certificate key management and IP subnetting
3. Connections between VPC, VPN, CGW and device certificates, tunnels, and interfaces are created

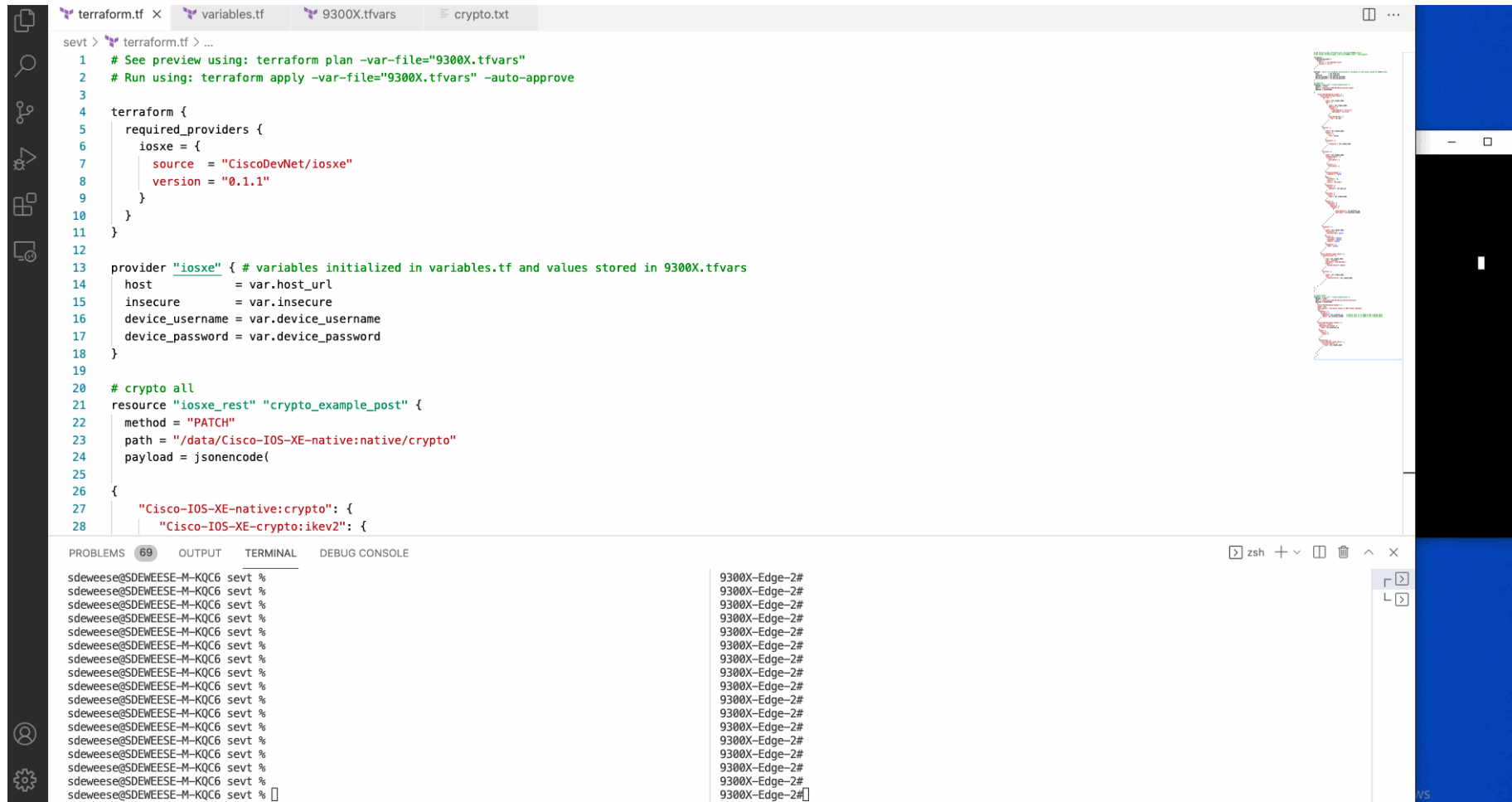


1. Virtual Private Cloud (VPC)
2. Virtual Private Network (VPN)
3. Customer Gateway (CGW)

1. Tunnel
2. Proposal
3. Policy
4. Keyring
5. Profile
6. IPSEC Transform
7. IPSEC Profile
8. Tunnel Interface



# Terraform + Crypto IPsec Demo



The screenshot shows a VS Code editor with four tabs: `terraform.tf`, `variables.tf`, `9300X.tfvars`, and `crypto.txt`. The `terraform.tf` file is active, displaying the following configuration:

```
1 # See preview using: terraform plan -var-file="9300X.tfvars"
2 # Run using: terraform apply -var-file="9300X.tfvars" -auto-approve
3
4 terraform {
5   required_providers {
6     iosxe = {
7       source = "CiscoDevNet/iosxe"
8       version = "0.1.1"
9     }
10  }
11 }
12
13 provider "iosxe" { # variables initialized in variables.tf and values stored in 9300X.tfvars
14   host = var.host_url
15   insecure = var.insecure
16   device_username = var.device_username
17   device_password = var.device_password
18 }
19
20 # crypto all
21 resource "iosxe_rest" "crypto_example_post" {
22   method = "PATCH"
23   path = "/data/Cisco-IOS-XE-native:native/crypto"
24   payload = jsonencode(
25   {
26     "Cisco-IOS-XE-native:crypto": {
27       "Cisco-IOS-XE-crypto:ikev2": {
```

The terminal window at the bottom shows the output of the `terraform apply` command, displaying a series of status messages for the `9300X-Edge-2#` device. The messages include the device name, the user `sevt`, and the command `%`.



# How can I replicate this?



## Configure an IPsec tunnel using Terraform

```
terraform-ipsec-tunnel-to-cloud / 9300X.tf

Code Blame 441 lines (427 loc) · 11.5 KB Code 55% faster with GitHub Copilot

36 # a. Add Crypto Config to Cisco IOS XE Device for Tunnel 1
37 resource "iosxe_rest" "crypto_example_post" {
38   depends_on = [ aws_vpn_connection.main ]
39   method = "PATCH"
40   path = "/data/Cisco-IOS-XE-native:native/crypto"
41   payload = jsonencode(
42
43   {
44     "Cisco-IOS-XE-native:crypto": {
45       "Cisco-IOS-XE-native:crypto:ikev2": {
46         "keyring": [
47           {
48             "name": var.crypto_name,
49             "peer": [
50               {
51                 "name": var.crypto_name,
52                 "address": {
53                   "ipv4": {
54                     "ipv4-address": "0.0.0.0",
55                     "ipv4-mask": "0.0.0.0"
56                   }
57               },
58               "pre-shared-key": {
59                 "key": var.psk
60               }
61             ]
62           }
63         ]
64       }
65     }
66   }
```

| terraform-ipsec-tunnel-to-cloud Public                         |                     |               | Pin | Unwatch 1 |
|----------------------------------------------------------------|---------------------|---------------|-----|-----------|
| main 1 Branch 0 Tags                                           |                     |               |     |           |
| Go to file                                                     |                     |               |     |           |
| Add file Code                                                  |                     |               |     |           |
| sdeweese Create variables.tf 9ee0a62 · 10 months ago 4 Commits |                     |               |     |           |
| 9300X.tf                                                       | Create 9300X.tf     | 10 months ago |     |           |
| aws.tf                                                         | Create aws.tf       | 10 months ago |     |           |
| outputs.tf                                                     | Create outputs.tf   | 10 months ago |     |           |
| variables.tf                                                   | Create variables.tf | 10 months ago |     |           |

```
terraform-ipsec-tunnel-to-cloud / aws.tf

Code Blame 83 lines (70 loc) · 2.9 KB Code 55% faster with GitHub Copilot

34 # 1b: Attach VPN/VPGW to VPC
35 # https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpn_gateway_attachment
36 resource "aws_vpn_gateway_attachment" "vpn_attachment" {
37   vpc_id = var.aws_vpc
38   vpn_gateway_id = aws_vpn_gateway.vpn_gw.id
39 }
40
41 # 1c: Enable Route Propagation
42 # https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpn_gateway_route_propagationdata "aws_route_tables" "rts" {
43 resource "aws_vpn_gateway_route_propagation" "example" {
44   vpn_gateway_id = aws_vpn_gateway.vpn_gw.id
45   route_table_id = var.aws_route
46 }
47
```

<https://github.com/sdeweese/terraform-ipsec-tunnel-to-cloud>

# Generate Python Script

YANG Suite / NETCONF / YANG set "c9300-default-yangset" / Modules

NETCONF

YANG Set: c9300-default-yangset

Module(s): Cisco-IOS-XE-native

NETCONF Operation: get-config

Device: C9300

Edit Device

Open Device Window

YANG Tree

Replays

RPC Options...

Build RPC

Run RPC(s)

Nodes

- Cisco-IOS-XE-native
  - default
  - bfd
  - version
  - stackwi
  - boot-st
  - boot

Load Replay...

Save as New Replay...

Edit Replay Information...

Save Changes

Generate Python script

Value

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" >
  <get-config>
    <source>
      <running/>
    </source>
    <filter>
      <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native" >
        <hostname/>
      </native>
    </filter>
  </get-config>
</rpc>
```

# Demo: Generate Python from NETCONF

Access YANG Suite >  
Protocols > NETCONF

Select YANG model then  
Build RPC payload

Select Replays

Select Generate Python  
script

The screenshot displays the Cisco YANG Suite interface. On the left, a sidebar shows the navigation menu with 'Admin', 'Setup', 'Explore', 'Protocols', and 'Help'. The 'Protocols' section is expanded, showing 'NETCONF'. The main area is titled 'YANG Suite / NETCONF / YANG set "c9300-default-yangset" / Modules'. It shows the 'c9300-default-yangset' YANG set selected, with the 'Module(s)' dropdown set to 'Cisco-IOS-XE-native'. The 'NETCONF Operation' is set to 'edit-config', and the 'Device' is 'C9300'. A 'Replays' button is highlighted with a blue box, and a dropdown menu is open showing options: 'Load Replay...', 'Save as New Replay...', 'Edit Replay Information...', 'Save Changes', and 'Generate Python script'. The 'Generate Python script' option is selected. The 'YANG Tree' on the left shows a hierarchy of nodes, with 'ios-vlan:vlan-list' selected. The 'Value' field shows '28' and the 'Operation' is 'create'. The right pane displays the generated XML RPC payload and the corresponding Python script. The XML payload is:

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <edit-config>
    <target>
      <running/>
    </target>
    <config>
      <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
        <vlan>
          <vlan-list xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-vlan">
            <id>28</id>
          </vlan-list>
        </vlan>
      </native>
    </config>
  </edit-config>
</rpc>
```

The Python script is:

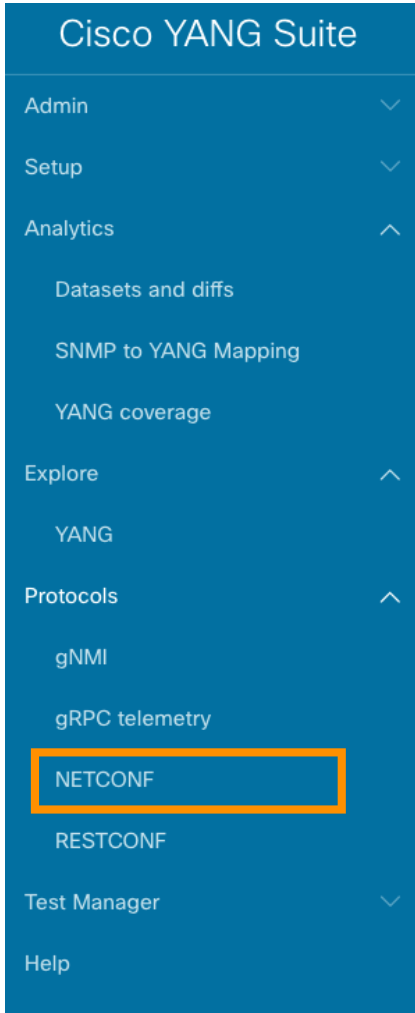
```
#!/usr/bin/env python
import lxml.etree as et
from argparse import ArgumentParser
from ncclient import manager
from ncclient.operations import RPCError

payload = [
    <get xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
      <filter>
        <lag-oper-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-lacp-oper">
          </lag-oper-data>
        </filter>
      </get>
    ]

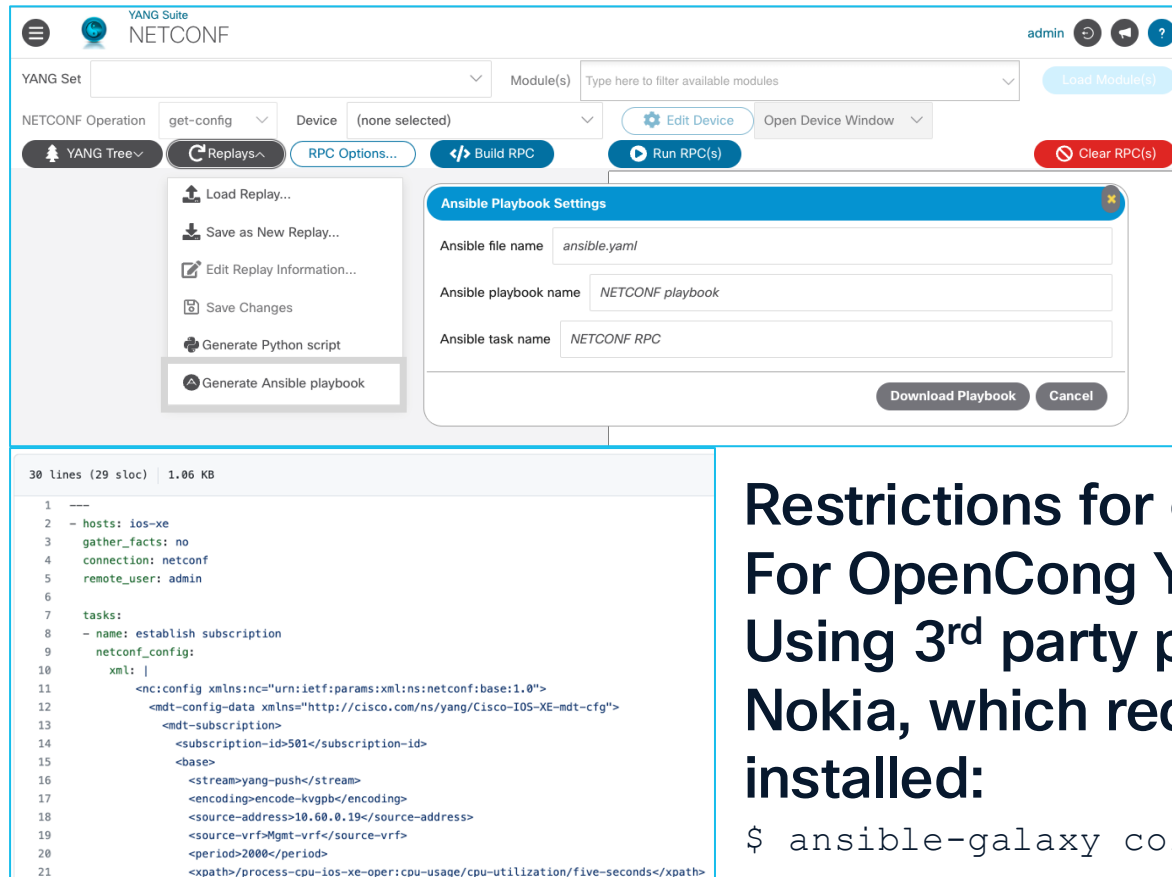
if __name__ == '__main__':
    parser = ArgumentParser(description='Usage:')
    # script arguments
    parser.add_argument('-a', '--host', type=str, required=True,
                        help="Device IP address or Hostname")
    parser.add_argument('-u', '--username', type=str, required=True,
                        help="Device Username (netconf agent username)")
    parser.add_argument('-p', '--password', type=str, required=True,
                        help="Device Password (netconf agent password)")
    parser.add_argument('--port', type=int, default=830,
                        help="Netconf agent port")
    args = parser.parse_args()
```

# YANG Suite + Ansible

## using NETCONF, RESTCONF & gNMI OpenConfig



Quickly and easily generate Ansible playbook for deployments to be used with the inventory, similar to the “Generate Python script” button.



For further coverage see:

<https://github.com/jeremycohoe/ansible-gnmi>

**Restrictions for gNMI**  
**For OpenCong YANG only**  
**Using 3<sup>rd</sup> party plugin to Ansible from**  
**Nokia, which requires collection to be**  
**installed:**

```
$ ansible-galaxy collection install nokia.grpc
```



# NETCONF + Ansible Update Interface Description Demo

Cisco YANG Suite

Admin Setup Analytics Explore Protocols Test Manager Help

YANG Suite / Help  
Help: Welcome to Cisco YANG Suite!

admin

## Welcome to Cisco YANG Suite!

YANG Suite is a set of tools related to YANG models ([RFC 6020](#), [RFC 7950](#)) and related technologies such as NETCONF ([RFC 6241](#)).

It provides a modular infrastructure which various YANG application plugins can be used.

Please check [developer.cisco.com/yangsuite](https://developer.cisco.com/yangsuite) for information, learning labs, and announcements.

## Docker-based Installation

The `yangsuite/docker/start_yang_suite.sh` script performs the following:

- Prompts for username, password, and email which will be the superuser to yangsuite.
- Gives the choice of creating test SSL/TLS certificate and key.
- Creates an environment file needed for the yangsuite docker container install.
- Runs docker-compose up.

## YANG Suite Documentation

Search docs

- [Welcome to Cisco YANG Suite!](#)
- [Device Profiles](#)
  - [Managing device profiles](#)
  - [Setting up YANG Suite TLS/SSL](#)
- [File Manager](#)
  - [Constructing and populating a YANG module repository](#)
  - [Uploading YANG files from the local filesystem to a YANG repository](#)
  - [Downloading YANG files via NETCONF from a device to a YANG repository](#)
  - [Copying YANG modules via SCP from a server to a YANG repository](#)
  - [Importing YANG modules from a Git repository to a YANG repository](#)
  - [Defining a YANG module set](#)
  - [Managing YANG module files in YANG Suite](#)
- [Working with YANG Models](#)
  - [Exploring YANG Models](#)
- [YANG Suite Analytics](#)
  - [Datasets](#)
  - [Mapping SNMP OIDs to YANG XPathS](#)
- [Using gNMI with YANG Suite](#)
- [YANG Suite gRPC telemetry receivers](#)
  - [Telemetry over gRPC Clear Channel](#)
  - [Configuring gRPC Telemetry Receivers](#)
  - [Telemetry over gRPC Secure Channel](#)
- [Using NETCONF with YANG Suite](#)
  - [Using NETCONF RPCs](#)
  - [Working with NETCONF Notification Streams](#)
  - [Using Replays for repeated workflows](#)
  - [Locking and unlocking datastores](#)
- [RESTCONF in YANG Suite](#)
  - [YANG Suite RESTCONF](#)
- [Test management with YANG Suite](#)
  - [Using Test Manager to define and execute tests](#)
  - [Generating YANG model Tests](#)
  - [Managing and editing replay files](#)

# Ansible NETCONF Example

## hosts.txt

```
[cat9300]
10.60.0.19
```

```
[ISR_4k]
10.12.253.43
```

```
[cat9800]
10.12.252.223
```

```
[ios-xe:children]
cat9300_4
ISR_4k
cat9800
```

```
[ios-xe:vars]
ansible_connection=netconf
ansible_network_os=ios
ansible_password=your_password
```

## ansible.cfg

```
[defaults]
inventory = ./hosts
host_key_checking = False
roles_path = ./
remote_user = demotme
```

## Add\_subscription.yaml

```
---
- hosts: cat9300
  gather_facts: no
  connection: netconf
  remote_user: demotme

  tasks:
    - name: establish subscription
      netconf_config:
        xml: |
          <config>
            <mdt-config-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-mdt-cfg">
              <mdt-subscription>
                <subscription-id>501</subscription-id>
                <base>
                  <stream>yang-push</stream>
                  <encoding>encode-kvgpb</encoding>
                  <source-address>10.60.0.19</source-address>
                  <source-vrf>Mgmt-vrf</source-vrf>
                  <period>2000</period>
                  <xpath>/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization/five-
seconds</xpath>
                </base>
                <mdt-receivers>
                  <address>10.12.252.224</address>
                  <port>57500</port>
                  <protocol>grpc-tcp</protocol>
                </mdt-receivers>
              </mdt-subscription>
            </mdt-config-data>
          </config>
```

```
- hosts: ISR_4k
  gather_facts: no
  connection: netconf
  remote_user: demotme

  tasks:
    - name: establish subscription
      netconf_config:
        xml: |
          <config>
            <mdt-config-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-mdt-cfg">
              <mdt-subscription>
                <subscription-id>501</subscription-id>
                <base>
                  <stream>yang-push</stream>
                  <encoding>encode-kvgpb</encoding>
                  <period>2000</period>
                  <xpath>/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization/five-seconds</xpath>
                </base>
                <mdt-receivers>
                  <address>10.12.252.224</address>
                  <port>57500</port>
                  <protocol>grpc-tcp</protocol>
                </mdt-receivers>
              </mdt-subscription>
            </mdt-config-data>
          </config>
```

```
- hosts: cat9800
  gather_facts: no
  connection: netconf
  remote_user: demotme

  tasks:
    - name: establish subscription
      netconf_config:
        xml: |
          <config>
            <mdt-config-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-mdt-cfg">
              <mdt-subscription>
                <subscription-id>501</subscription-id>
                <base>
                  <stream>yang-push</stream>
                  <encoding>encode-kvgpb</encoding>
                  <source-address>10.12.252.223</source-address>
                  <period>2000</period>
                  <xpath>/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization/five-
seconds</xpath>
                </base>
                <mdt-receivers>
                  <address>10.12.252.224</address>
                  <port>57500</port>
                  <protocol>grpc-tcp</protocol>
                </mdt-receivers>
              </mdt-subscription>
            </mdt-config-data>
          </config>
```

# Running the playbook

## add\_subscription

```
ubuntu@ubuntu18:~/ansible$ ansible-playbook add_subscription.yaml

PLAY [cat9300_4] *****

TASK [establish subscription] *****
changed: [10.60.0.19]

PLAY [ISR_4k] *****

TASK [establish subscription] *****
changed: [10.12.253.43]

PLAY [cat9800] *****

TASK [establish subscription] *****
changed: [10.12.252.223]

PLAY RECAP *****
10.12.252.223      : ok=1    changed=1    unreachable=0    failed=0
10.12.253.43      : ok=1    changed=1    unreachable=0    failed=0
10.60.0.19        : ok=1    changed=1    unreachable=0    failed=0
```

## delete\_subscription

```
ubuntu@ubuntu18:~/ansible$ ansible-playbook delete_subscription.yaml

PLAY [ios-xe] *****

TASK [delete subscription] *****
changed: [10.12.253.43]
changed: [10.12.252.223]
changed: [10.60.0.19]

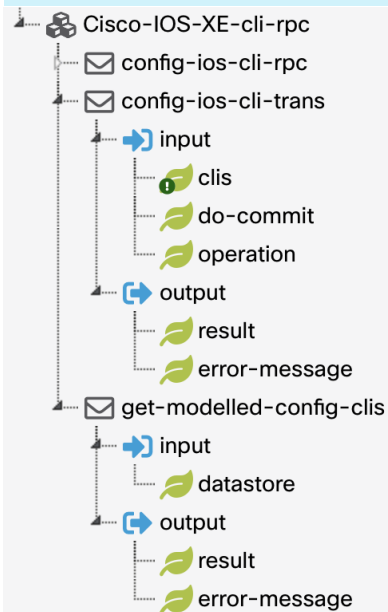
PLAY RECAP *****
10.12.252.223      : ok=1    changed=1    unreachable=0    failed=0
10.12.253.43      : ok=1    changed=1    unreachable=0    failed=0
10.60.0.19        : ok=1    changed=1    unreachable=0    failed=0
```

## list of files

```
ubuntu@ubuntu18:~/ansible$ ls
add_subscription.yaml  ansible.cfg  delete_subscription.yaml  hosts
```

# Cisco-IOS-XE-CLI-RPC.yang

- This YANG model allows working with CLI over YANG Interfaces
- Sending the “get-modelled-config-clis” RPC returns the modelled running-config in CLI format
- This CLI can be used as the template and sent back to the device or Jinja



RPC:

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">  
  <get-modelled-config-clis xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-cli-rpc"/>  
</rpc>
```

```
Sending:  
#246  
<nc:rpc xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:afff"  
  <get-modelled-config-clis xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-cli-rpc"/>  
</nc:rpc>
```

##

Received message from host

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns:nc="urn:ietf:params:  
<result xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-cli-rpc">version 17.14  
memory free low-watermark processor 130582  
service timestamps debug datetime msec  
service timestamps log datetime msec  
service call-home  
no service tcp-small-servers  
no service udp-small-servers  
hostname JCOH0E-C9300-2  
control-plane  
  service-policy input system-cpp-policy  
!  
clock summer-time PDST recurring  
clock timezone pacific -8 0  
login on-success log  
license boot level network-advantage addon dna-advantage  
transceiver type all  
monitoring  
!  
iox  
call-home  
  contact-email-addr sch-smart-licensing@cisco.com  
  profile CiscoTAC-1  
  active  
  destination transport-method http
```



# Device Monitoring

How can I monitor all  
5000 new switches reliably,  
efficiently and at scale?





# Why It's Time for a Change?

## Challenges of SNMP Monitoring



Lack of Real-Time and  
Granular Monitoring



Scalability and Performance  
Bottlenecks



Security Risks

# Webex API telemetry integrations

Goal: Notify me when device on my network comes online

1. ZTP triggers Embedded Event Manager (EEM)
2. The EEM actions copy & run a script into the guestshell
3. The script send a message in a pre-defined Webex space
4. All user in the Webex space are notified of the device coming online

```
print ("*** Starting notification on device! ***")
print ("** delete any old files **")
cli_command = "delete /force flash:guest-share/wxt.*"
cli.execute(cli_command)
print ("** copy files **")
cli_command = "copy http://128.107.223.248/wxt.tgz flash:guest-share/"
cli.execute(cli_command)
cli_command = "copy http://128.107.223.248/wxt.sh flash:guest-share/"
cli.execute(cli_command)
#cli_command = "copy http://128.107.223.248/wxt.tgz flash:guest-share/"
#cli.execute(cli_command)
#cli_command = "copy http://128.107.223.248/wxt.sh flash:guest-share/"
#cli.execute(cli_command)
print ("** create alias exec **")
# Create alias notification to run the EEM applet with:
cli.configure(["alias exec wxtnotification event manager run notification", "exit"])
# Enable notification EEM applet and script
print ("*** eem applet ***")
eem_notification_commands = ['no event manager applet notification',
                             'event manager applet notification',
                             'event none maxrun 600',
                             'action 0001 cli command "enable"',
                             'action 0002 cli command "guestshell enable"',
                             'action 0030 cli command "guestshell run /usr/bin/bash /bootflash/guest-share/wxt.sh"']
results = configure(eem_notification_commands)
print ("*** call that eem applet ***")
cli_command = "wxtnotification"
cli.execute(cli_command)
print ("*** Successfully configured notification on device! ***")
```

<https://github.com/jeremycohoe/IOSXE-Zero-Touch-Provisioning/blob/main/ztp-xelab.py>

```
cat9300x-pod27a#dir flash:guest-share
Directory of flash:guest-share/

638979  -rw-          641  Jan 6 2026 12:08:15 -08:00  wxt.sh
638978  -rw-         17873  Jan 6 2026 12:05:03 -08:00  downloaded_script.py

11353194496 bytes total (5338558464 bytes free)
cat9300x-pod27a#more fl
cat9300x-pod27a#more flash:gu
cat9300x-pod27a#more flash:guest-share/w
cat9300x-pod27a#more flash:guest-share/wxt.sh
#!/bin/bash

mkdir -p ~/wxt/
cd /home/guestshell/wxt/

/usr/bin/tar xvfz /bootflash/guest-share/wxt.tgz -C /home/guestshell/wxt/

sudo /usr/bin/pip3 install /home/guestshell/wxt/charset_normalizer-2.0.12-py3-none-any.whl
sudo /usr/bin/pip3 install /home/guestshell/wxt/idna-3.6-py3-none-any.whl
sudo /usr/bin/pip3 install /home/guestshell/wxt/urllib3-1.26.18-py2.py3-none-any.whl
sudo /usr/bin/pip3 install /home/guestshell/wxt/certifi-2023.11.17-py3-none-any.whl
sudo /usr/bin/pip3 install /home/guestshell/wxt/requests-2.27.1-py2.py3-none-any.whl

echo "Sending notification"
/usr/bin/python3 /home/guestshell/wxt/notification.py
```

```
auto@podadmin2-ubuntu:/var/www/html/wxt$ cat notification.py
import requests
import socket
from cli import configure, cli
import cli

url = "https://api.ciscospark.com/v1/messages"

room_id = "Y2l2Y29zcGFyYXovL3VzL1JPT00vOTg0ZmJyZDZlZWZjMjM0MmVjLWUwMTk4NGQ3ODU0MjA5MjUz"
bearer = "ZDRkZTc5NDM4MTNkNC00NDZlNFhMjE1MjY0ODM5NjIwZGQyZjZlNGJlM0RkPF84_1eb65fdf-9643-417f-9974-ad72cae0e10f"

#message = "This is a test message from the PANDA SourceM POD 21 yo yo yo"
#message = "C9300: a switch {} ".format(socket.gethostname()) + " has rebooted"
#print(socket.gethostname())

# Fixed: Use cli.execute() instead of cli.clip() which doesn't exist
uptime = cli.execute('show version | i uptime')
module = cli.execute('show module')
version = cli.execute('show version | i Cisco IOS XE Software, Version')

print(uptime)

payload = {
    "roomId": room_id,
    "text": "!!! ZTP Success !!!\n %s %s %s" % (uptime, version, module)
}

headers = {
    "Authorization": "Bearer %s " % bearer
}

response = requests.post(url, headers=headers, data=payload.json())
print(response)
```

PANDA Bot 1/6/26, 12:27 PM

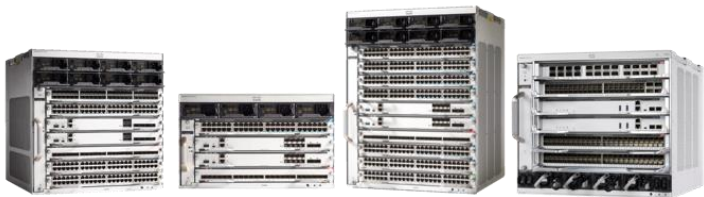
!!! ZTP Success !!!

cat9300x-pod01a uptime is 26 minutes

Cisco IOS XE Software, Version 17.18.01

| Switch | Ports | Model       | Serial No.  | MAC address    | Hw Ver. | Sw Ver.  |
|--------|-------|-------------|-------------|----------------|---------|----------|
| 1      | 78    | C9300X-48HX | FOC2727Y739 | 6c03.b5c1.9500 | V02     | 17.18.01 |

# Telemetry Data Flow



Cisco IOS XE Devices



**Collector/Receiver**  
Decodes to text



**Storage**  
Time Series Database



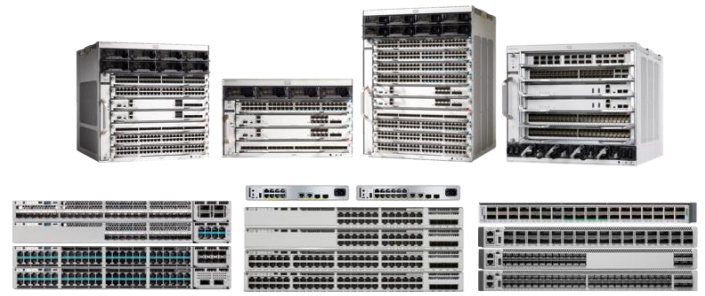
**Monitoring  
and Visualizations**



[https://hub.docker.com/r/jeremycohoe/tig\\_mdt](https://hub.docker.com/r/jeremycohoe/tig_mdt)    <https://github.com/jeremycohoe/cisco-ios-xe-mdt>  
[https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b\\_1715\\_programmability\\_cg/m\\_1715\\_prog\\_ietf\\_telemetry.html](https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1715/b_1715_programmability_cg/m_1715_prog_ietf_telemetry.html)



# Collecting data in Splunk



Cisco IOS XE Devices



**telegraf**<sup>TM</sup>

OpenTelemetry

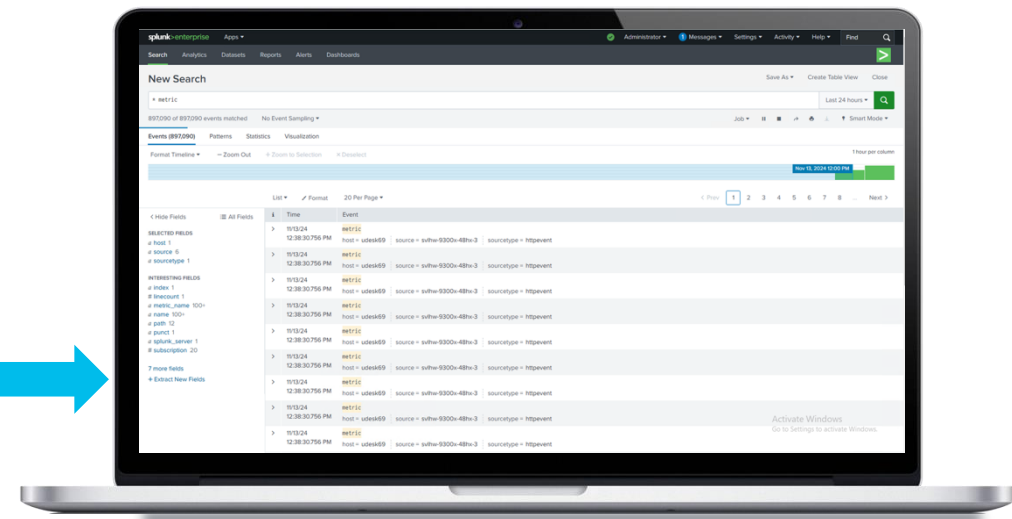
<https://opentelemetry.io/>

Collector/Receiver  
Decodes to text

Storage  
Time Series Database  
  
Monitoring  
and Visualizations

**splunk**>

a **CISCO** company



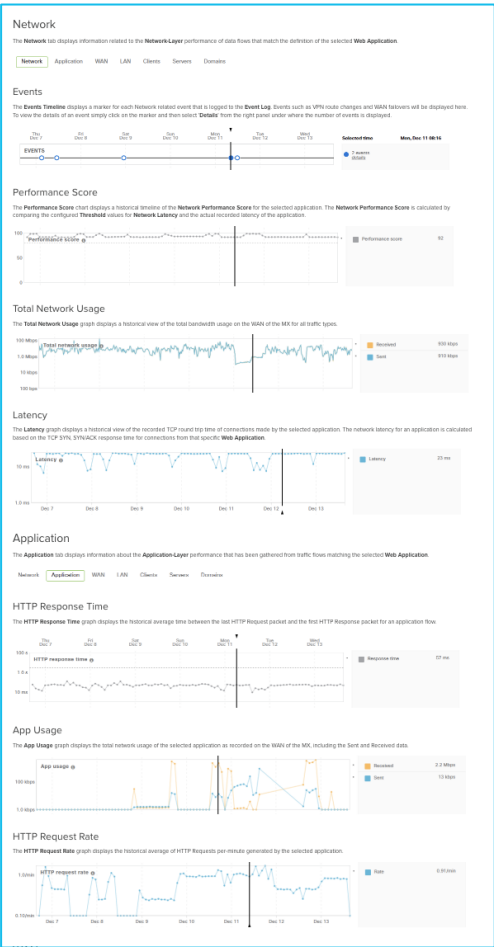


# Cisco-on-Cisco Telemetry Integrations ?

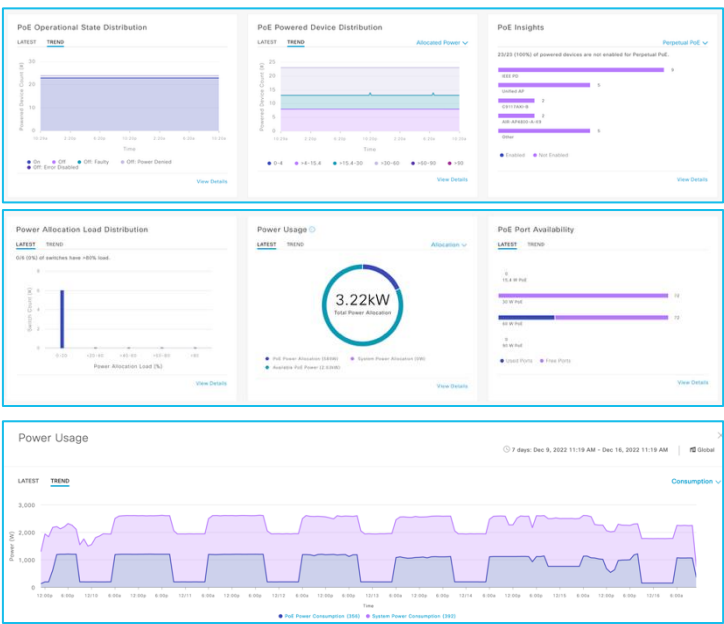
Q: How does Cisco consume telemetry from IOS XE ?

A: The Cisco software controllers also use the gRPC & NETCONF telemetry interfaces to collect data

## Meraki Dashboard



## Catalyst Center



Select a data type below to filter the preceding table details.

| Top Locations (P90 Count) | Top Switch Names (P90 Count) | Top Switch Types (P90 Count) | Top Powered Device Types (P90 Count) |
|---------------------------|------------------------------|------------------------------|--------------------------------------|
| San Jose, CA              | San Jose, CA                 | San Jose, CA                 | San Jose, CA                         |
| San Francisco, CA         | San Francisco, CA            | San Francisco, CA            | San Francisco, CA                    |
| Austin, TX                | Austin, TX                   | Austin, TX                   | Austin, TX                           |
| Bangalore, IN             | Bangalore, IN                | Bangalore, IN                | Bangalore, IN                        |
| Nevada, NV                | Nevada, NV                   | Nevada, NV                   | Nevada, NV                           |
| Las Vegas, NV             | Las Vegas, NV                | Las Vegas, NV                | Las Vegas, NV                        |
| Other networks            | Other networks               | Other networks               | Other networks                       |

Current view selected: San Jose, CA, 2023-10-10 PM 1:30 PM | Operational State: On

PD Table (23)

| Identifier | Powered Device Model | Powered Device Type | Connected Switch     | Switch Interface         | IEEE Compliant | Location                     | Allocated Power |
|------------|----------------------|---------------------|----------------------|--------------------------|----------------|------------------------------|-----------------|
| 1          | IEEE PD              | IEEE PD             | VN21-8000001-0000001 | TenGigabitEthernet1/0/22 | Yes            | Global/Vancouver/Canada/VN21 | 15.4W           |
| 2          | IEEE PD              | IEEE PD             | VN21-8000001-0000001 | TenGigabitEthernet1/0/22 | Yes            | Global/Vancouver/Canada/VN21 | 8.3W            |

## Catalyst SD-WAN

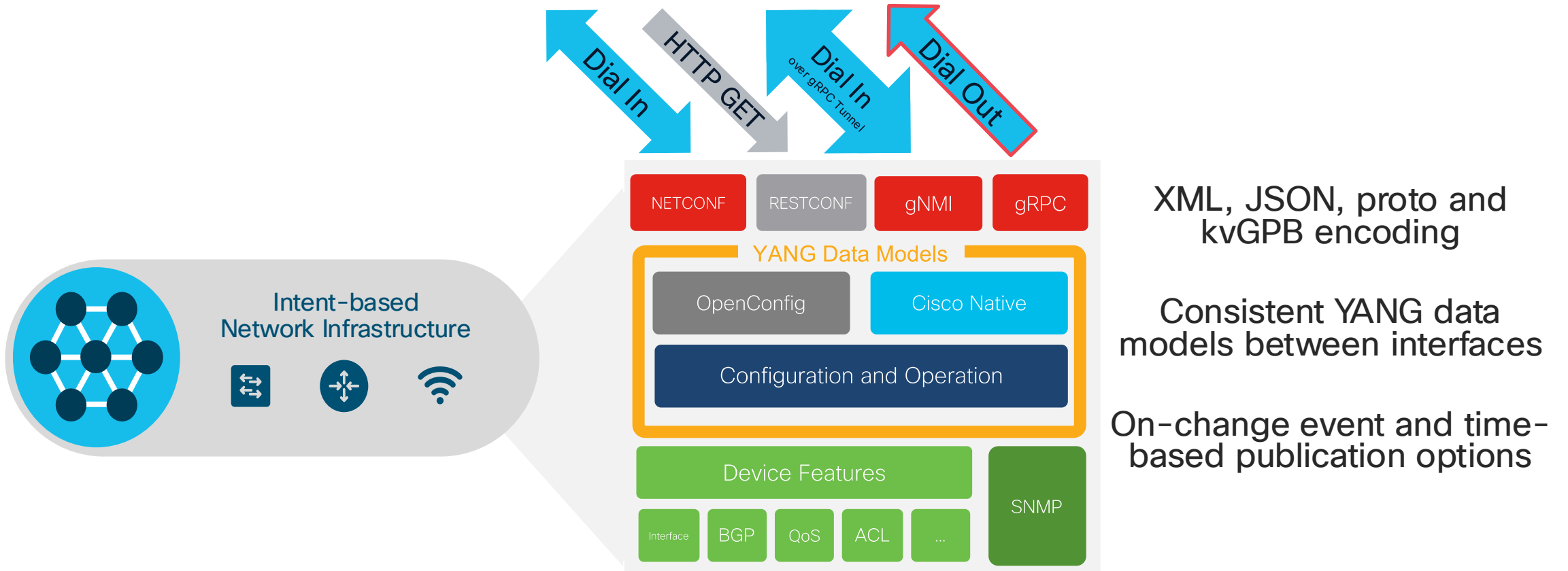


# Model Driven Telemetry Interfaces



- ↔ Dial In: Collector establishes a connection to the device then subscribes to telemetry (pub/sub)
- ← Dial Out: Telemetry is pushed from the device to the collector based off configuration (push)

## Publication / Subscription



# Model Driven Telemetry Interface Comparison

|                        | NETCONF                                 | gRPC (Dial-Out)              | gNMI                                                                                           |
|------------------------|-----------------------------------------|------------------------------|------------------------------------------------------------------------------------------------|
| Minimum IOS XE Version | 16.6 (2017)                             | 16.10 (2018)                 | Dial-In: 16.12 (2019)<br>gRPC tunnel (2023)                                                    |
| Telemetry Direction    | Dial-In,<br>IOS XE is server            | Dial-Out<br>IOS XE is client | Dial-In<br>IOS XE is server<br>Dial-Out<br>gRPC Tunnel                                         |
| Configuration          | Dynamic<br>per session                  | Static<br>per configuration  | Dynamic<br>per session                                                                         |
| Telemetry Collector    | Client                                  | Server                       | Client                                                                                         |
| Encoding               | XML                                     | KV GPB                       | JSON_IETF + PROTO                                                                              |
| Security               | SSH + PKI<br>certificate or<br>password | mTLS or plain-text           | mTLS certificates<br><small>mTLS cert only or<br/>mTLS cert + user/pass authentication</small> |
| Transport Protocol     | SSH                                     | HTTP2                        | HTTP2                                                                                          |
| Data Models            | YANG                                    | YANG                         | YANG                                                                                           |

Network architecture, security posture and policy, YANG data modules, tools and language preferences, and standards, and software version, are some considerations when leveraging the various MDT interfaces

- NETCONF for RFP/compliance use only – not really seen in use in production
- gRPC for most enterprises: the NetOps configures the telemetry to push to the server
- gNMI for leading edge: NetDevOps accesses the network device

# Cisco Telemetry Data Broker (Telegraf)

Cisco Telemetry Broker provides many benefits include brokering, filtering, and transforming data. It provides the ability to replicate telemetry data.



| PRODUCTS                                                             | UNIT LIST PRICE                                        | STATUS    | QUANTITY    | ACTION |
|----------------------------------------------------------------------|--------------------------------------------------------|-----------|-------------|--------|
| <b>Virtual License</b>                                               |                                                        |           |             |        |
| Cisco Telemetry Broker Essential License - 100GB/Day TB-ESS-100GB SA | 810.00<br>Per 100GB/Day/12 Month                       | Added     | 20<br>100GB | Delete |
| <b>Support - Virtual License</b>                                     |                                                        |           |             |        |
| Enhanced Support for Cisco Telemetry Broker SVS-TB-SUP-E             | 202.50<br>Per Each/Month                               | Added     | 1<br>Each   | Delete |
| Premium Support for Cisco Telemetry Broker SVS-TB-SUP-P              | Take an action to see the List Price<br>Per Each/Month | Not Added | 1<br>Each   | Swap   |

## ► Brokering Data:

The ability to route and replicate telemetry data from a source location to multiple destination consumers.

Quickly onboard new telemetry-based tools!

## ► Filtering Data:

The ability to filter data that is being replicated to consumers for fine grain control over what consumers are able to see and analyze.

Save money sending data to expensive tools!

## ► Transforming Data:

The ability to transform data protocols from the exporter to the consumer's protocol of choice.

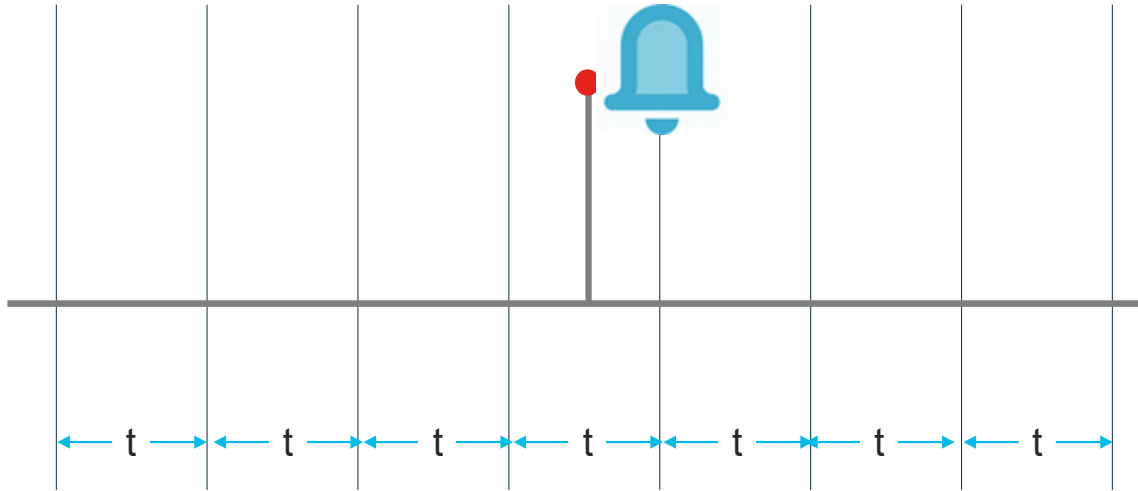
Enable tools to consume multiple data formats!

<https://cs.co/telemetrybroker> aka <https://www.cisco.com/c/en/us/products/security/telemetry-broker/index.html>  
<https://blogs.cisco.com/security/taking-full-control-of-your-telemetry-with-the-intelligent-telemetry-plane>



# Publication options

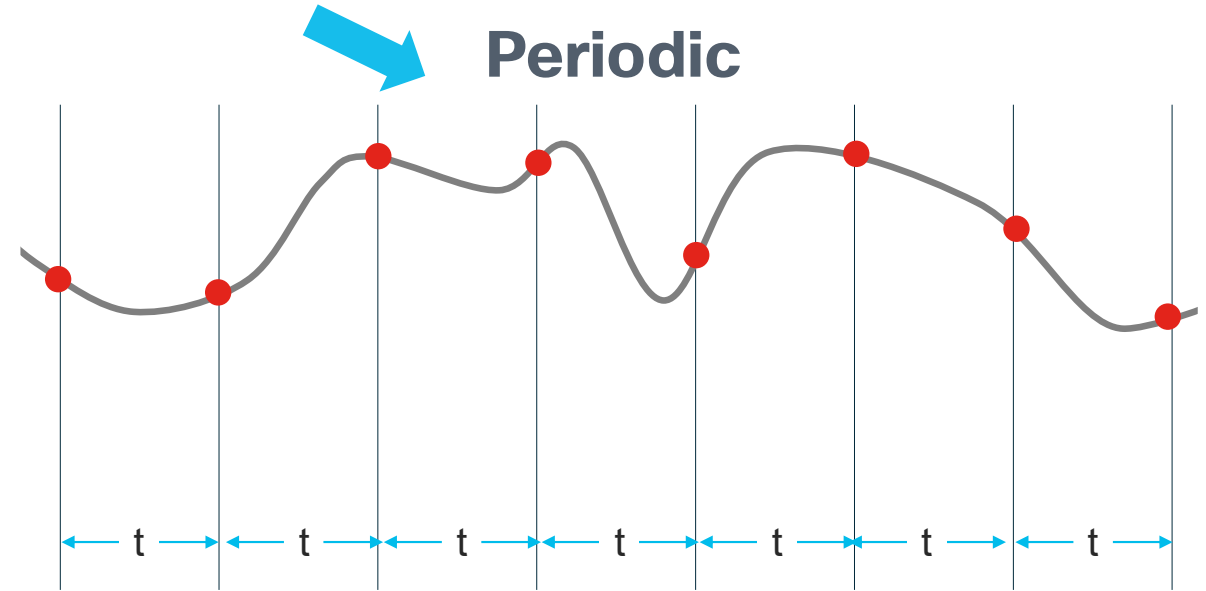
## On-Change



Feature Model “On-Change” Notifications  
Event Notifications (failed login, optic fault, etc)  
State and Configuration

We'll focus on periodic in this session because it is the most common!

## Periodic

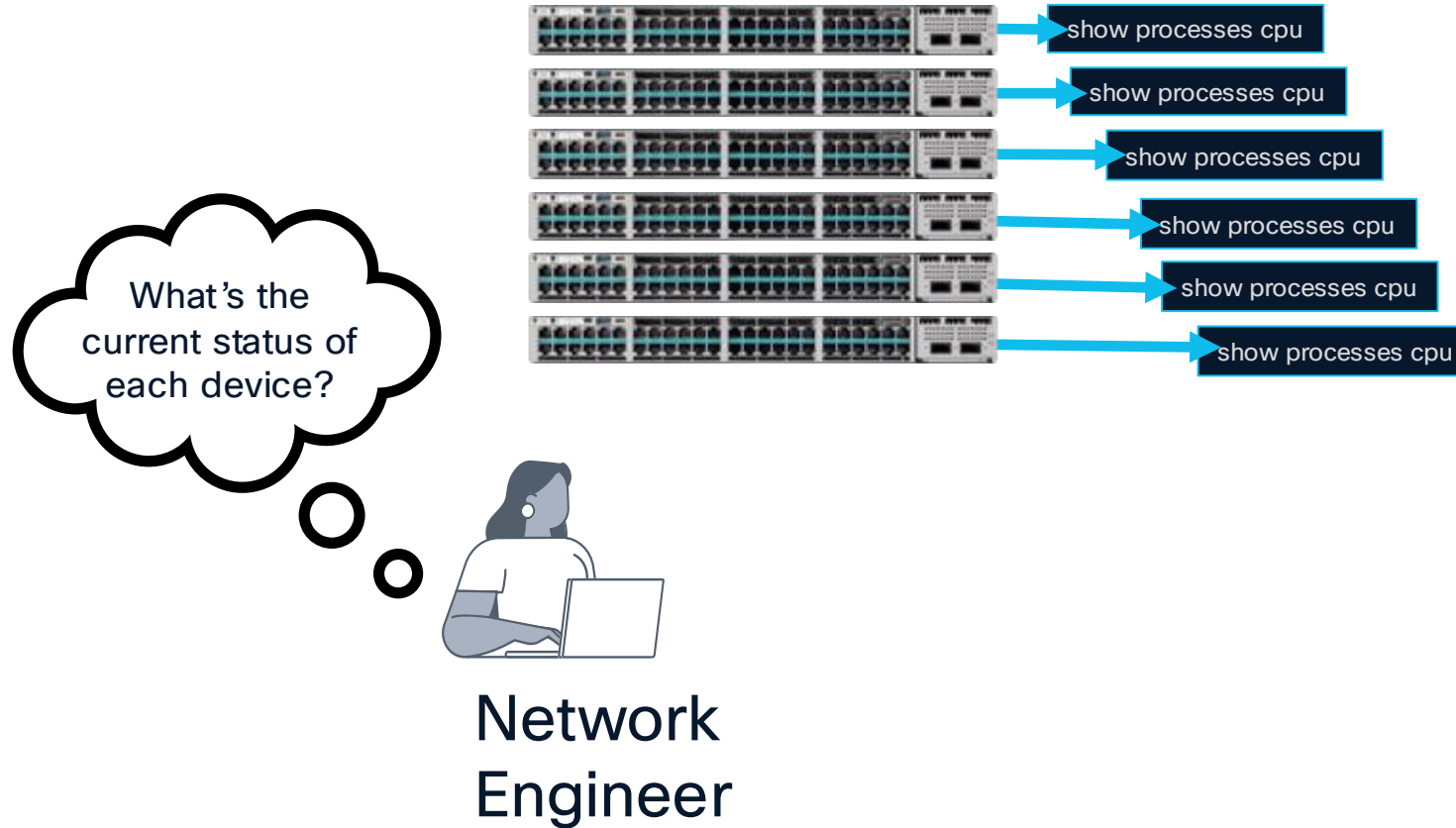


Feature Model “Periodic” Notifications  
Time based publication  
Minimum interval 100 centiseconds (1s)

OpenConfig YANG with GNXI, not NETCONF

# Using Telemetry to Track Devices

Show CPU on each device



# Now, let's work more efficiently

Cisco devices use structured data called YANG models

## CLI

```
cat9300x-pod29a#show processes cpu
CPU utilization for five seconds: 0%/0%; one minute: 0%; five minutes: 0%
PID Runtime(ms) Invoked uSecs 5Sec 1Min 5Min TTY Process
1 0 19 0 0.00% 0.00% 0.00% 0 Chunk Manager
2 429509 2209233 194 0.00% 0.00% 0.00% 0 Load Meter
3 0 2 0 0.00% 0.00% 0.00% 0 PKI Trustpool
4 378 17 22235 0.00% 0.00% 0.00% 0 RF Slave Main Th
5 0 1 0 0.00% 0.00% 0.00% 0 Retransmission o
6 0 1 0 0.00% 0.00% 0.00% 0 IPC ISSU Dispatc
7 0 1 0 0.00% 0.00% 0.00% 0 RO Notify Timers
8 1920 368205 5 0.00% 0.00% 0.00% 0 VIDB BACKGD MGR
9 4460802 1502068 2969 0.00% 0.03% 0.00% 0 Check heaps
10 27588 184104 149 0.00% 0.00% 0.00% 0 Pool Manager
11 0 1 0 0.00% 0.00% 0.00% 0 DiscardQ Backgro
12 0 2 0 0.00% 0.00% 0.00% 0 Timers
13 11 1623 6 0.00% 0.00% 0.00% 0 WATCH_AFS
14 6955 472437 14 0.00% 0.00% 0.00% 0 DB Lock Manager
15 69695 11046159 6 0.00% 0.00% 0.00% 0 GraphIt
16 0 3 0 0.00% 0.00% 0.00% 0 DB Notification
```

CPU data

## YANG Model

YANG Suite / Exploring YANG / YANG set "IOS XE Switching" / Modules

Explore YANG Models

Select a YANG set: IOS XE Switching Select YANG module(s): Cisco-IOS-XE-process-cpu-oper

Icon legend Search XPath's Search nodes Expand all nodes

Cisco-IOS-XE-process-cpu-oper

cpu-usage

cpu-utilization

five-seconds

five-seconds-intr

one-minute

five-minutes

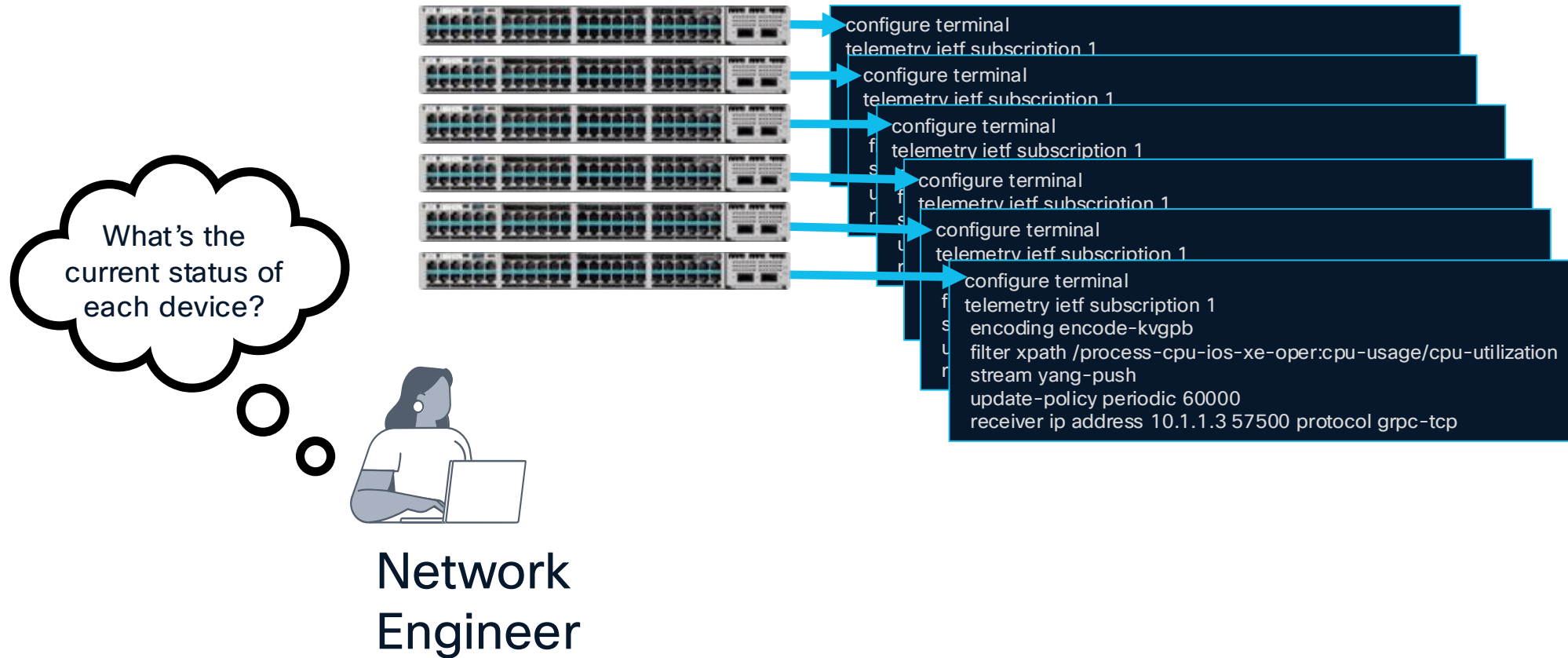
cpu-usage-processes

Node Properties

|                |                                                        |
|----------------|--------------------------------------------------------|
| Name           | cpu-utilization                                        |
| Nodetype       | container                                              |
| Description    | Data nodes for Total CPU Utilization Statistics.       |
| Module         | Cisco-IOS-XE-process-cpu-oper                          |
| Revision       | 2017-02-07                                             |
| Xpath          | /cpu-usage/cpu-utilization                             |
| Prefix         | process-cpu-ios-xe-oper                                |
| Namespace      | http://cisco.com/ns/yang/Cisco-IOS-XE-process-cpu-oper |
| Access         | read-only                                              |
| Operations     | • "get"                                                |
| Schema Node Id | /cpu-usage/cpu-utilization                             |

# Using Telemetry to Track Devices

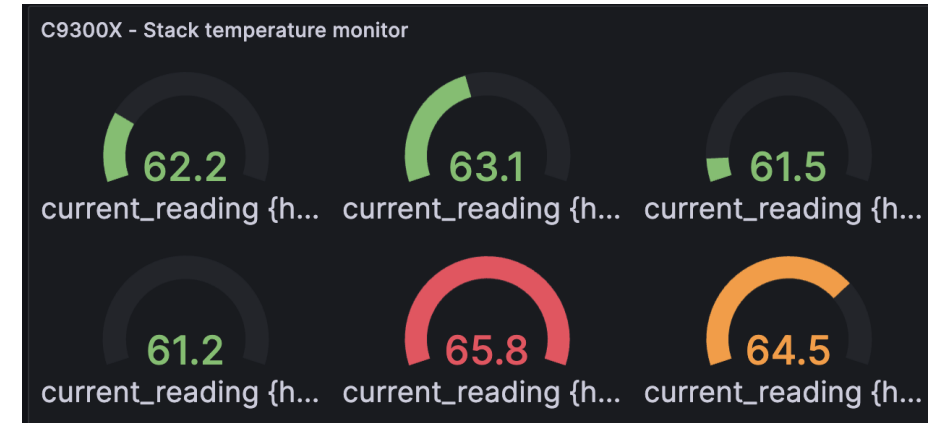
## Configure Telemetry Subscription One Time



# Monitor devices through a single pane of glass



We need to track the status of all these devices at scale



Telemetry from a Cisco device allows us to know its current status

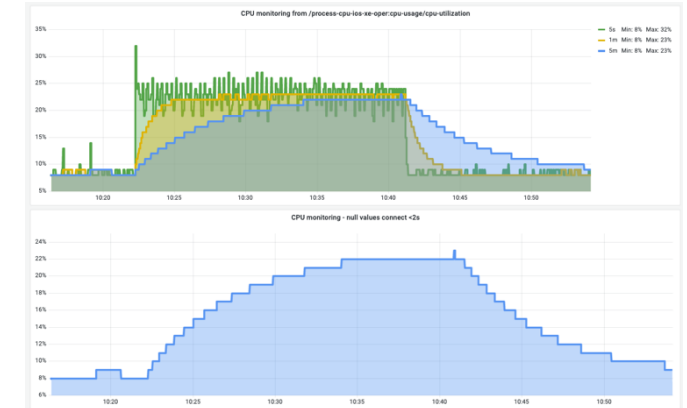
We can collect this data at scale!



# Model Driven Telemetry: usage comparison

60-minute collection sample with 60-second update interval

| Interface | CPU Impact | PCAP file size/data size (MB) | Data byte Rate | Data bit rate | Average Packet Rate (sec) | Average Packet Size (bytes) |
|-----------|------------|-------------------------------|----------------|---------------|---------------------------|-----------------------------|
| gNMI      | +3%        | 23 MB                         | 6 kBps         | 53 kbps       | 5                         | 1180                        |
| gRPC      | +3%        | 69 MB                         | 19 kBps        | 155 kbps      | 58                        | 333                         |
| NETCONF   | +2%        | 83 MB                         | 23 kBps        | 185 kbps      | 29                        | 780                         |
| RESTCONF  | +4%        | 200 MB                        | 35 kBps        | 281 kbps      | 37                        | 945                         |
| SNMP *    | +6%        | 120 / 87                      | 24 kBps        | 197 kbps      | 90                        | 273                         |



17 xpaths collected at 60 second update interval

```

/arp-ios-xe-oper:arp-data
/cdp-ios-xe-oper:cdp-neighbor-details
/environment-ios-xe-oper:environment-sensors
/if:interfaces-state
/interfaces-ios-xe-oper:interfaces/interface
/ios:native
/ldp-ios-xe-oper:ldp-entries
/matm-ios-xe-oper:matm-oper-data
/mdt-oper:mdt-oper-data/mdt-subscriptions
/memory-ios-xe-oper:memory-statistics/memory-statistic
/oc-if:interfaces/interface/state/counters
/oc-platform:components
/oc-sys:system
/platform-ios-xe-oper:components
/poe-ios-xe-oper:poe-oper-data/poe-switch
/process-cpu-ios-xe-oper:cpu-usage/cpu-utilization
/process-memory-ios-xe-oper:memory-usage-processes
  
```

+ Device-hardware-oper + Switch-stack-oper + more ?



This demonstrates that even when SNMP is only measuring Interfaces the load is still significantly higher than YANG which is measuring significantly more YANG data

# Model Metadata with YANG Suite

## filter xpath / PREFIX : XPATH

The CLI config to enable MDT requires the “filter xpath” which defines which YANG data to publish

Use YANG Suite to find the prefix and xpath from the YANG models to use in the configuration

Configuration Guide:

[https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1610/b\\_1610\\_programmability\\_cg/model\\_driven\\_telemetry.html](https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1610/b_1610_programmability_cg/model_driven_telemetry.html)

The screenshot shows the YANG Suite interface for exploring YANG models. The 'Node Properties' panel on the right displays the following information for the selected node 'cpu-utilization':

| Node Properties |                                                        |
|-----------------|--------------------------------------------------------|
| Name            | cpu-utilization                                        |
| Nodetype        | container                                              |
| Description     | Data nodes for Total CPU Utilization Statistics.       |
| Module          | Cisco-IOS-XE-process-cpu-oper                          |
| Revision        | 2017-02-07                                             |
| Xpath           | /cpu-usage/cpu-utilization                             |
| Prefix          | process-cpu-ios-xe-oper                                |
| Namespace       | http://cisco.com/ns/yang/Cisco-IOS-XE-process-cpu-oper |
| Access          | read-only                                              |
| Operations      | • "get"                                                |
| Schema Node Id  | /cpu-usage/cpu-utilization                             |

Arrows from the 'Xpath' and 'Prefix' fields point to a text box at the bottom of the slide.

filter xpath /process-cpu-ios-xe-oper:cpu-usage/cpu-utilization

# Configure a gRPC Telemetry Subscription

Configuring telemetry subscriptions like the following to collect CPU data over time

On Cisco IOS XE Device:

```
configure terminal
telemetry ietf subscription 1
encoding encode-kvgpb
filter xpath /process-cpu-ios-xe-oper cpu-usage/cpu-utilization
stream yang-push
update-policy periodic 60000
receiver ip address 10.1.1.3 57500 protocol grpc-tcp
```

See more examples at: <https://github.com/jeremycohoe/cisco-ios-xe-mdt>

# Monitoring today for each network device

## CLI

```
cat9300x-pod29a#show processes cpu
CPU utilization for five seconds: 0%/0%; one minute: 0%; five minutes: 0%
PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min  TTY Process
  1         0         19         0  0.00%  0.00%  0.00%  0 Chunk Manager
  2    429509    2209233      194  0.00%  0.00%  0.00%  0 Load Meter
  3         0         2         0  0.00%  0.00%  0.00%  0 PKI Trustpool
  4     378         17    22235  0.00%  0.00%  0.00%  0 RF Slave Main Th
  5         0         1         0  0.00%  0.00%  0.00%  0 Retransmission o
  6         0         1         0  0.00%  0.00%  0.00%  0 IPC ISSU Dispatc
  7         0         1         0  0.00%  0.00%  0.00%  0 RO Notify Timers
  8     1920    368205         5  0.00%  0.00%  0.00%  0 VIDB BACKGD MGR
  9   4460802   1502068    2969  0.00%  0.03%  0.00%  0 Check heaps
 10     27588    184104     149  0.00%  0.00%  0.00%  0 Pool Manager
 11         0         1         0  0.00%  0.00%  0.00%  0 DiscardQ Backgro
 12         0         2         0  0.00%  0.00%  0.00%  0 Timers
 13        11     1623         6  0.00%  0.00%  0.00%  0 WATCH_AFS
 14     6955    472437        14  0.00%  0.00%  0.00%  0 DB Lock Manager
 15    69695   11046159         6  0.00%  0.00%  0.00%  0 GraphIt
 16         0         3         0  0.00%  0.00%  0.00%  0 DB Notification
```

x6 devices

# Splunk integration

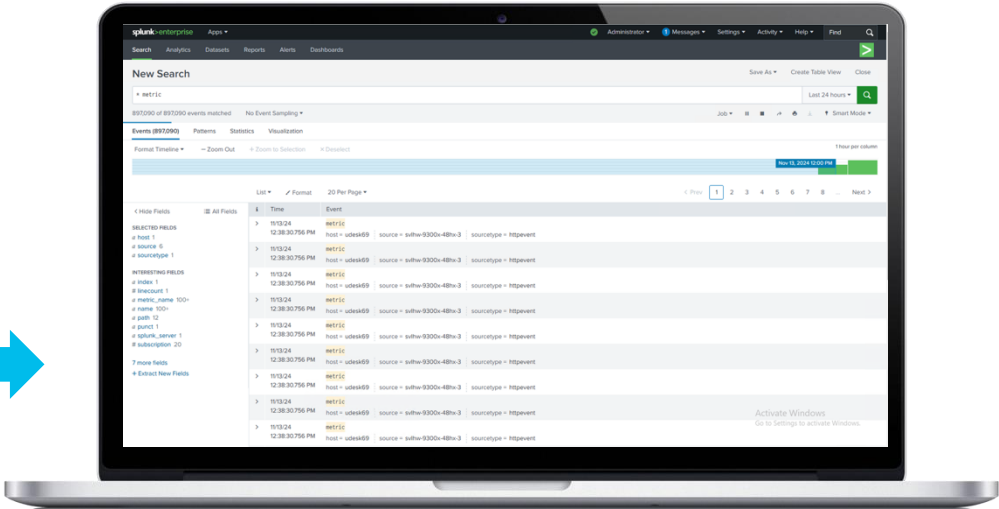
Cisco IOS XE Devices



Collector/Receiver  
Decodes to text

```
# Output to Splunk
[[outputs.http]]
url = "https://172.26.202.150:8088/services/collector"
insecure_skip_verify = true
data_format = "splunkmetric"
splunkmetrichec_routing = true
[outputs.http.headers]
Content-Type = "application/json"
Authorization = "Splunk 3042d3d2-c140-48e7-a7c7-2f60f4c54b05"
X-Splunk-Request-Channel = "3042d3d2-c140-48e7-a7c7-2f60f4c54b05"
```

Storage  
Time Series Database  
Monitoring  
and Visualizations





# What's the Return on Investment (ROI)?

# Enhanced Energy Metering

## What it is

Enables energy meter for Switch System Power and PoE switchport power consumption  
Platforms: C9200, C9300, C9400

## What it does

System Energy is power consumed by the system for a specific duration, measured in unit of Watt Second. Macro-metered window size of 3-hours with 12 x 15-minute micro-meters

Gives the visibility within the windows of consumption that can be further analyzed and considered for carbon intensity reporting, density-based usage analysis and so on.

## What it receives

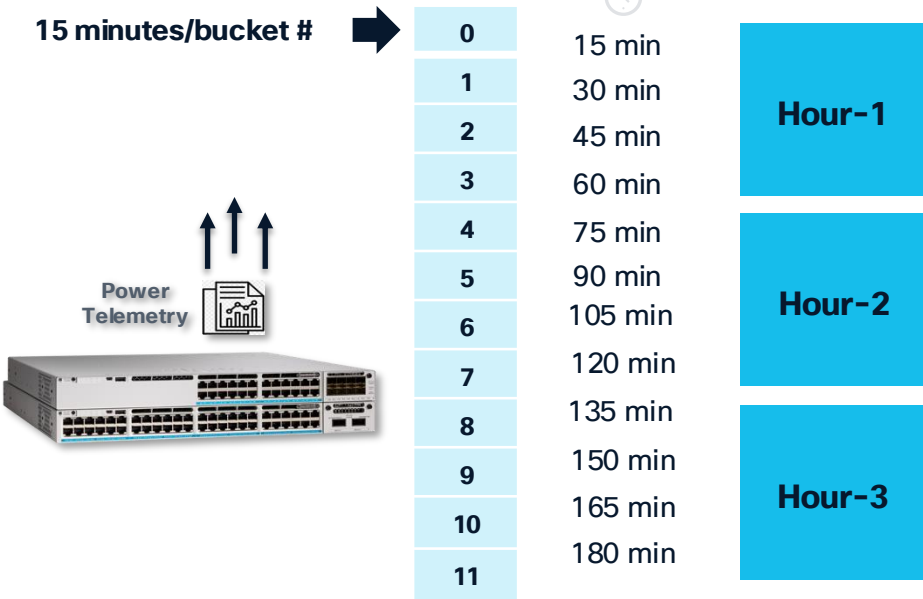
Ledger on Energy consumption for System Power and PoE Port with a bucketized data of 15 minutes each.

### System Meter

| Meter start time 2024-10-22 10:36:34 PST |           |                              |                          |
|------------------------------------------|-----------|------------------------------|--------------------------|
| Energy Data For Last 180 Minute          |           |                              |                          |
| Mod                                      | Model No  | System Energy (MilliWattSec) | System Meter Update Time |
| 1                                        | C9300-24H | 234894600                    | 2024-10-22 11:32:39 PST  |

### PoE Port Meter

| SB-Salone1-C9324H#sho power inline meter |                         |                                     |                                                 |
|------------------------------------------|-------------------------|-------------------------------------|-------------------------------------------------|
| Module                                   | Available (Watts)       | Used (Watts)                        | Remaining (Watts)                               |
| 1                                        | 860.0                   | 120.0                               | 740.0                                           |
| Interface                                | Meter Update Time       | Hourly Metered Value (MilliWattSec) | Metered Energy in MilliWattSec (15min Buckets)  |
| Gi1/0/1                                  | 2024-10-22 11:21:09 PST | 0                                   | 0-0-0-0-0-0-0-0-0-0-0-0                         |
| Gi1/0/2                                  | 2024-10-22 11:21:09 PST | 10507380                            | 1465368-3553224-3558320-1930468-0-0-0-0-0-0-0-0 |





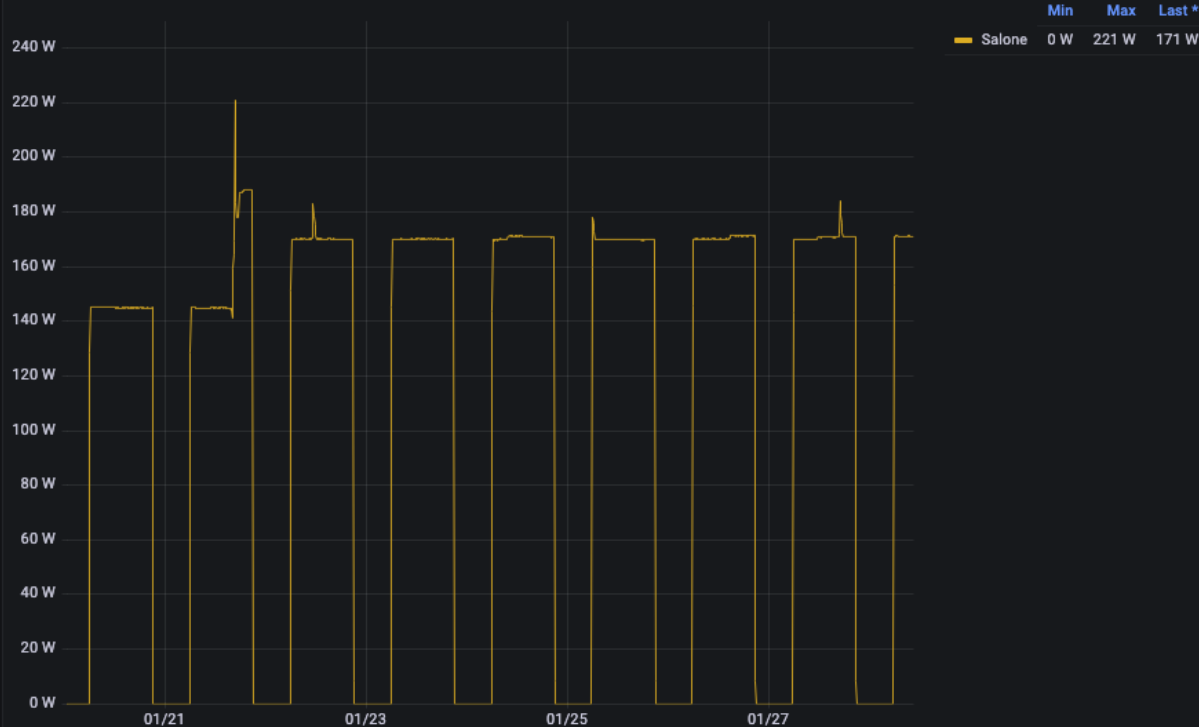
Interface All switch SB-Salone1-C9324H ▾



Total Power Allocated - POE + System



Realtime POE Power Consumption



Per interface power consumption



# Power Telemetry

<https://github.com/YangModels/yang/blob/main/vendor/cisco/xr/17101/Cisco-IOS-XE-poe-oper.yang>

# telemetry configuration

```
no telemetry ietf subscription 69001
telemetry ietf subscription 69001
filter xpath /poe-ios-xe-oper:poe-oper-data/poe-port-detail
receiver ip address 10.85.134.66 57508 protocol grpc-tcp
source-address 10.85.134.70
source-vrf Mgmt-vrf
stream yang-push
update-policy periodic 3000
encoding encode-kvgpb
```

```
no telemetry ietf subscription 69002
telemetry ietf subscription 69002
filter xpath /poe-ios-xe-oper:poe-oper-data/poe-switch
receiver ip address 10.85.134.66 57508 protocol grpc-tcp
source-address 10.85.134.70
source-vrf Mgmt-vrf
stream yang-push
update-policy periodic 3000
encoding encode-kvgpb
```

```
no telemetry ietf subscription 69003
telemetry ietf subscription 69003
filter xpath /poe-ios-xe-oper:poe-oper-data/poe-stack
receiver ip address 10.85.134.66 57508 protocol grpc-tcp
source-address 10.85.134.70
source-vrf Mgmt-vrf
stream yang-push
update-policy periodic 3000
encoding encode-kvgpb
```



# YANG data model

|                                                                                                        |                       |                                                                      |
|--------------------------------------------------------------------------------------------------------|-----------------------|----------------------------------------------------------------------|
|  poe-port-detail    | <b>Name</b>           | poe-port-detail                                                      |
|  intf-name          | <b>Nodetype</b>       | list                                                                 |
|  power-used         | <b>Description</b>    | List of PoE interfaces, keyed by interface name                      |
|  pd-class           | <b>Module</b>         | Cisco-IOS-XE-poe-oper                                                |
|  device-detected    | <b>Revision</b>       | 2022-07-01                                                           |
|  device-name        | <b>Xpath</b>          | /poe-oper-data/poe-port-detail                                       |
|  police             | <b>Prefix</b>         | poe-ios-xe-oper                                                      |
|  power-admin-max    | <b>Namespace</b>      | http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper                       |
|  power-from-pse     | <b>Schema Node Id</b> | /poe-oper-data/poe-port-detail                                       |
|  power-to-pd        | <b>Keys</b>           | <ul style="list-style-type: none"><li>• "intf-name"</li></ul>        |
|  poe-switch         | <b>Access</b>         | read-only                                                            |
|  switch-num         | <b>Operations</b>     | <ul style="list-style-type: none"><li>• "get"</li></ul>              |
|  power-budget       | <b>Name</b>           | poe-switch                                                           |
|  power-allocated    | <b>Nodetype</b>       | list                                                                 |
|  low-port-priority  | <b>Description</b>    | List of PoE switches, keyed by switch number                         |
|  high-port-priority | <b>Module</b>         | Cisco-IOS-XE-poe-oper                                                |
|  switch-priority    | <b>Revision</b>       | 2022-07-01                                                           |
|  port-one-status    | <b>Xpath</b>          | /poe-oper-data/poe-switch                                            |
|  port-two-status    | <b>Prefix</b>         | poe-ios-xe-oper                                                      |
|                                                                                                        | <b>Namespace</b>      | http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper                       |
|                                                                                                        | <b>Schema Node Id</b> | /poe-oper-data/poe-switch                                            |
|                                                                                                        | <b>Keys</b>           | <ul style="list-style-type: none"><li>• "switch-num"</li></ul>       |
|                                                                                                        | <b>Access</b>         | read-only                                                            |
|                                                                                                        | <b>Operations</b>     | <ul style="list-style-type: none"><li>• "get"</li></ul>              |
|  poe-stack          | <b>Name</b>           | poe-stack                                                            |
|  power-stack-name  | <b>Nodetype</b>       | list                                                                 |
|  mode             | <b>Description</b>    | List of PoE stacks, keyed by stack name                              |
|  topolgy          | <b>Module</b>         | Cisco-IOS-XE-poe-oper                                                |
|  total-power      | <b>Revision</b>       | 2022-11-01                                                           |
|  rsvd-power       | <b>Xpath</b>          | /poe-oper-data/poe-stack                                             |
|  alloc-power      | <b>Prefix</b>         | poe-ios-xe-oper                                                      |
|  unused-power     | <b>Namespace</b>      | http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper                       |
|  num-sw           | <b>Schema Node Id</b> | /poe-oper-data/poe-stack                                             |
|  num-ps           | <b>Keys</b>           | <ul style="list-style-type: none"><li>• "power-stack-name"</li></ul> |
|                                                                                                        | <b>Access</b>         | read-only                                                            |
|                                                                                                        | <b>Operations</b>     | <ul style="list-style-type: none"><li>• "get"</li></ul>              |

## Same prefix, three Xpaths





# Data Explorer

S

Graph

CUSTOMIZE

Local

SAVE AS



| table | _measurement                                      | _field                  | _value             | _start                    | _stop                     | _time                        | host            | path            |
|-------|---------------------------------------------------|-------------------------|--------------------|---------------------------|---------------------------|------------------------------|-----------------|-----------------|
| last  | group<br>string                                   | group<br>string         | no group<br>string | group<br>dateTime:RFC3339 | group<br>dateTime:RFC3339 | no group<br>dateTime:RFC3339 | group<br>string | group<br>string |
| 0     | Cisco-IOS-XE-environment-oper:environment-sensors | environment_sensor/name | Power Supply C     | 2025-05-28T00:50:19.234Z  | 2025-05-28T01:05:19.234Z  | 2025-05-28T00:53:20.000Z     | udesk69         | Cisco-IOS-XE-   |
| 0     | Cisco-IOS-XE-environment-oper:environment-sensors | environment_sensor/name | Power Supply C     | 2025-05-28T00:50:19.234Z  | 2025-05-28T01:05:19.234Z  | 2025-05-28T00:58:20.000Z     | udesk69         | Cisco-IOS-XE-   |

1 2

Query 1 (0.03s)

+

View Raw Data



Past 15m

SCRIPT EDITOR

SUBMIT

FROM

Search buckets

SB-Bucket

\_monitoring

\_tasks

+ Create Bucket

Filter

source

1

Search source tag values

C9350-48U\_Stack\_Edge

SB-C9524Q-Core-SVL

sv1hw-9300x-48hx-3

sv1hw-93

Filter

subscription

Search subscription tag value

79006

79007

79008

79009

79010

79011

79012

79013

79014

Filter

\_measurement

1

Search \_measurement tag va

Cisco-IOS-XE-envIRONM...

Cisco-IOS-XE-interfac...

Cisco-IOS-XE-memory-o...

Cisco-IOS-XE-platform...

Cisco-IOS-XE-poe-oper...

Cisco-IOS-XE-poe-oper...

Cisco-IOS-XE-poe-oper...

TECOPS-2314

Filter

\_field

Search \_field tag values

environment\_sensor/hi...

environment\_sensor/hi...

environment\_sensor/lo...

environment\_sensor/lo...

environment\_sensor/lo...

environment\_sensor/na...

environment\_sensor/se...

Filter

host

Search host tag values

udesk69

WINDOW PERIOD

CUSTOM

AUTO

auto (10s)

Fill missing values

AGGREGATE FUNCTION

CUSTOM

AUTO

mean

median

last



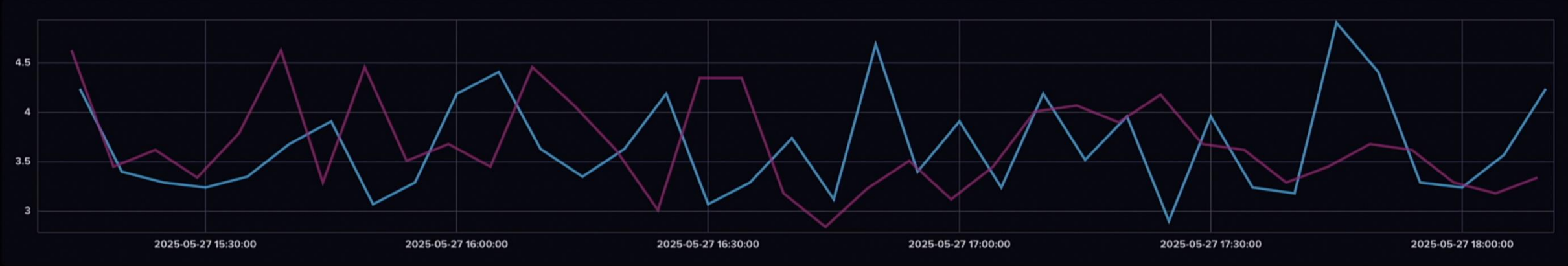
# Data Explorer

Band

CUSTOMIZE

Local

SAVE AS



Query 1 (0.02s)

View Raw Data

Past 3h

SCRIPT EDITOR

SUBMIT

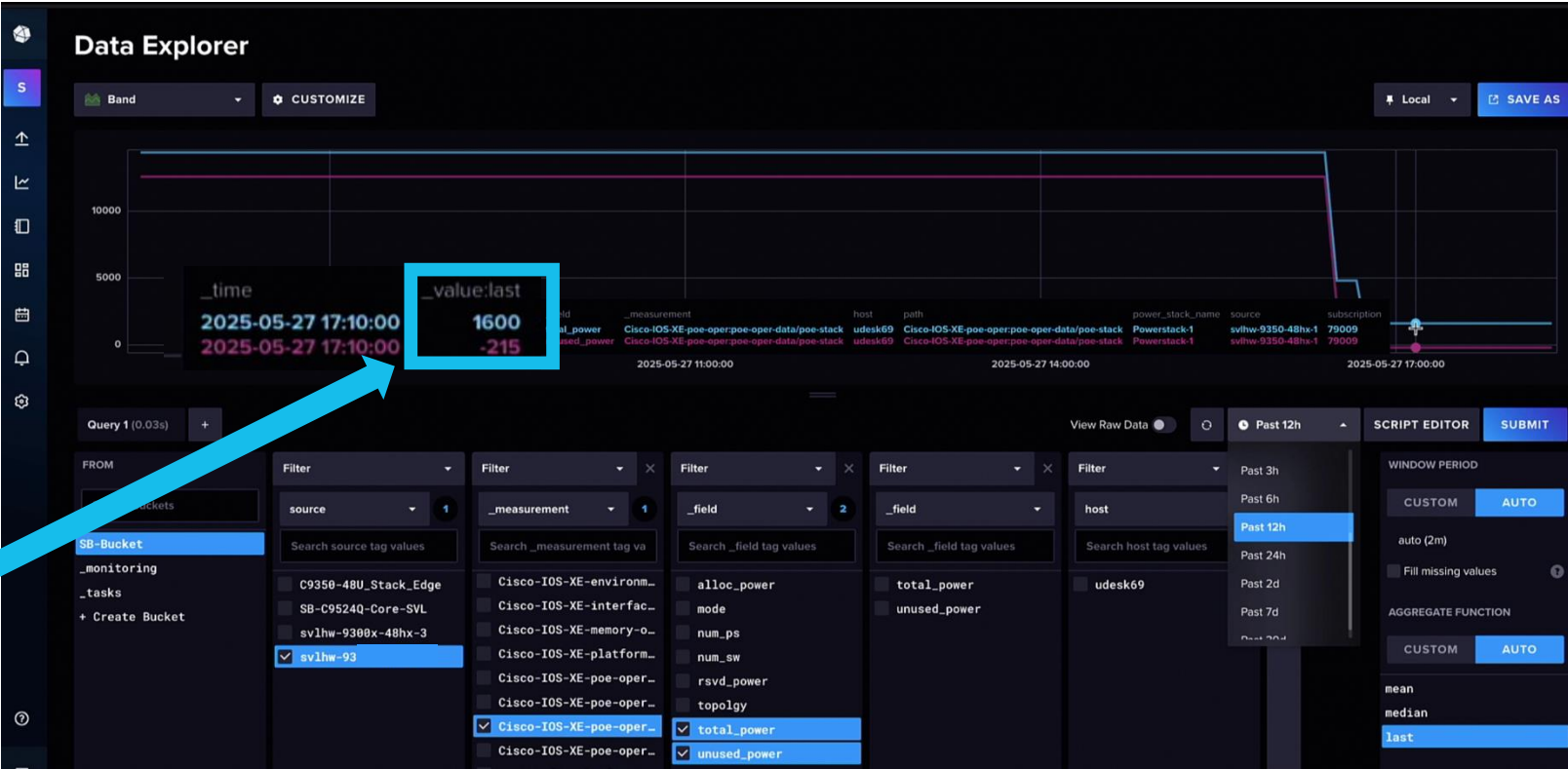
|                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>FROM</div> <div>Search buckets</div> <div>SB-Bucket</div> <div>_monitoring</div> <div>_tasks</div> <div>+ Create Bucket</div> | <div>Filter</div> <div>source</div> <div>3</div> <div>Search source tag values</div> <div><div><input checked="" type="checkbox"/> C9350-48U-Stack-Edge</div><div><input type="checkbox"/> SB-C9524Q-Core-SVL</div><div><input checked="" type="checkbox"/> sv1hw-9300x-48hx-3</div><div><input checked="" type="checkbox"/> sv1hw-93</div></div> | <div>Filter</div> <div>subscription</div> <div>69007</div> <div>69008</div> <div>69009</div> <div>69010</div> <div>69011</div> <div>69012</div> <div>69013</div> <div>69014</div> <div>69015</div> | <div>Filter</div> <div>_measurement</div> <div>1</div> <div>Search _measurement tag values</div> <div><div><input type="checkbox"/> Cisco-IOS-XE-interrac...</div><div><input type="checkbox"/> Cisco-IOS-XE-memory-o...</div><div><input type="checkbox"/> Cisco-IOS-XE-platform...</div><div><input type="checkbox"/> Cisco-IOS-XE-platform...</div><div><input type="checkbox"/> Cisco-IOS-XE-poe-oper...</div><div><input checked="" type="checkbox"/> Cisco-IOS-XE-poe-oper_...</div><div><input type="checkbox"/> Cisco-IOS-XE-poe-oper...</div><div><input type="checkbox"/> Cisco-IOS-XE-switchpo...</div></div> | <div>Filter</div> <div>_field</div> <div>2</div> <div>Search _field tag values</div> <div><div><input type="checkbox"/> meter-energy-value</div><div><input type="checkbox"/> module_id</div><div><input type="checkbox"/> number_of_buckets</div><div><input type="checkbox"/> oper_police</div><div><input checked="" type="checkbox"/> oper_power</div><div><input type="checkbox"/> oper_priority</div><div><input type="checkbox"/> oper_state</div><div><input type="checkbox"/> over_current_counter</div><div><input type="checkbox"/> pd_class</div></div> | <div>Filter</div> <div>_field</div> <div>1</div> <div>Search _field tag values</div> <div><div><input type="checkbox"/> device_detected</div><div><input type="checkbox"/> value/string</div><div><input type="checkbox"/> device_name</div><div><input checked="" type="checkbox"/> oper_power</div></div> | <div>Filter</div> <div>intf_name</div> <div>1</div> <div>Search intf_name</div> <div><div><input checked="" type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div><div><input type="checkbox"/> TenGigabitEt</div></div> | <div>WINDOW PERIOD</div> <div>CUSTOM</div> <div>AUTO</div> <div>auto (1m)</div> <div><input type="checkbox"/> Fill missing values</div> <div>AGGREGATE FUNCTION</div> <div>CUSTOM</div> <div>AUTO</div> <div>mean</div> <div>median</div> <div>last</div> |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

CLI

```
show power detail
```

| Power Summary<br>(in Watts) | Allocated | Consumed | Maximum<br>Available |
|-----------------------------|-----------|----------|----------------------|
| System Power                | 1440      | 518      | 1470                 |
| POE Power                   | 345       | 67       | 130                  |
| Total                       | 1785      | 585      | 1600                 |

Telemetry





# Auto-off StackPower PSU



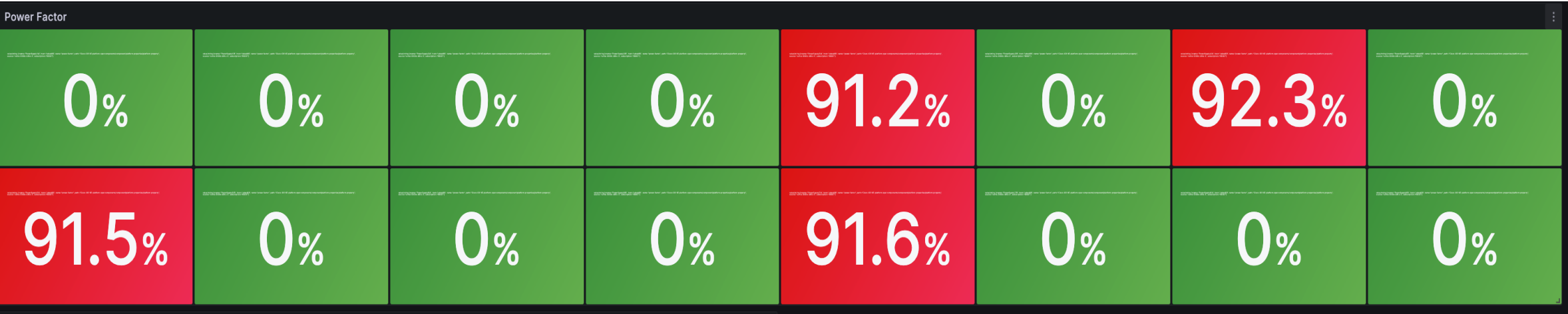
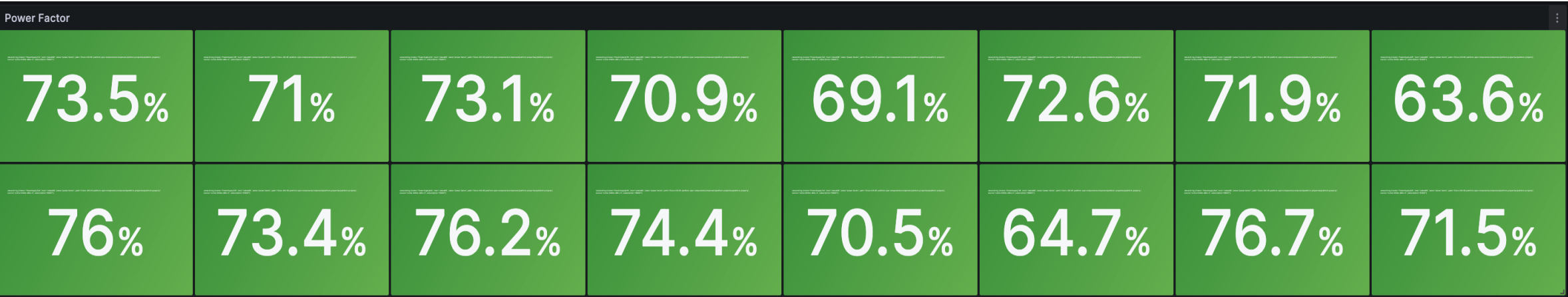
# StackPower Power draw with auto-off

|                                   |      |      |      |      |      |      |    |
|-----------------------------------|------|------|------|------|------|------|----|
| Stack Power Supplies Output Power |      |      |      |      |      |      |    |
| 91                                | 88.8 | 90.3 | 86.8 | 85.3 | 94.8 | 83.3 | 48 |
| 86.8                              | 89.5 | 93   | 89.3 | 90.3 | 51.5 | 98   | 84 |

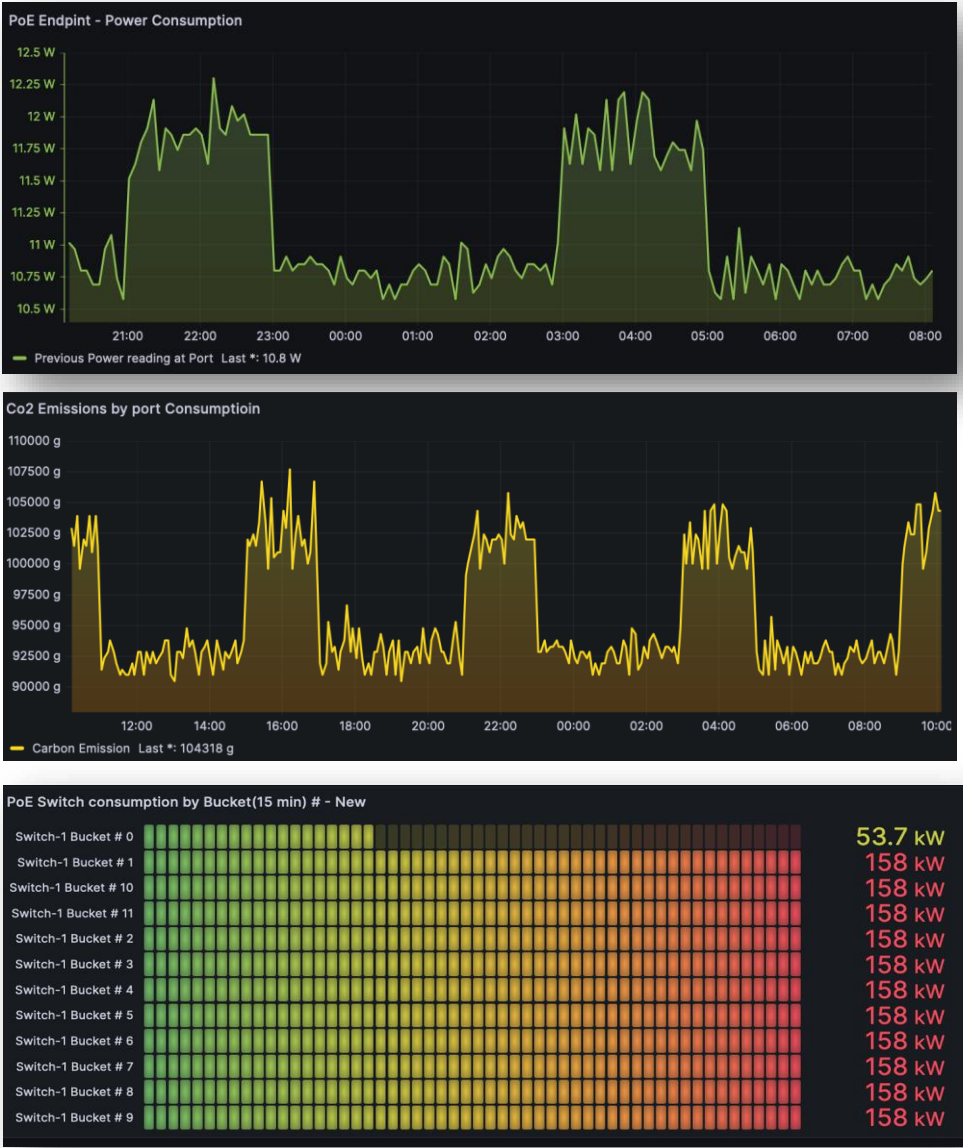
|                                            |   |   |   |     |   |     |   |
|--------------------------------------------|---|---|---|-----|---|-----|---|
| Stack Power Supplies Input vs Output Power |   |   |   |     |   |     |   |
| 0                                          | 0 | 0 | 0 | 362 | 0 | 331 | 0 |
| 329                                        | 0 | 0 | 0 | 370 | 0 | 0   | 0 |



# StackPower Efficiency with auto-off



# What can we do with Power Telemetry



Up to **25%**  Energy usage **reduction** on POE switchports with IOT endpoints turned off at TOD (Time of a Day)

How is this applicable to Cisco controller-based solutions for Wireless Access Points?

# Catalyst 9800 WLC Calendar Template scheduling

To enable **power save mode** on Cisco Catalyst Access Points  
<https://github.com/jeremycohoe/c9800-ap-power-save/>

The screenshot shows the Cisco Catalyst 9800-40 Wireless Controller configuration interface. The left sidebar contains navigation links: Dashboard, Monitoring, Configuration, Administration, Licensing, and Troubleshooting. The main content area is titled 'Configuration > Tags & Profiles > AP Join'. The 'Edit AP Join Profile' window is open, showing the 'Calendar Profile - Power Profile Map' section. This section includes a table with columns: Calendar, Recurrence, Start Time, End Time, and Power Profile. The 'Off Work' calendar is selected, with a daily recurrence from 00:00:00 to 23:30:00, linked to the 'Off Work Hours' power profile. Below this, the 'Power Profile Detailed' section shows a table of power-saving actions for various interfaces, all set to 'Disabled'.

| Sequence | Interface | Interface ID    | Parameter | Parameter Value |
|----------|-----------|-----------------|-----------|-----------------|
| 0        | Radio     | 6 GHz           | State     | Disabled        |
| 1        | Radio     | Secondary 5 GHz | State     | Disabled        |
| 2        | Radio     | 2.4 GHz         | State     | Disabled        |
| 3        | USB       | USB 0           | State     | Disabled        |
| 4        | Radio     | 5 GHz           | State     | Disabled        |

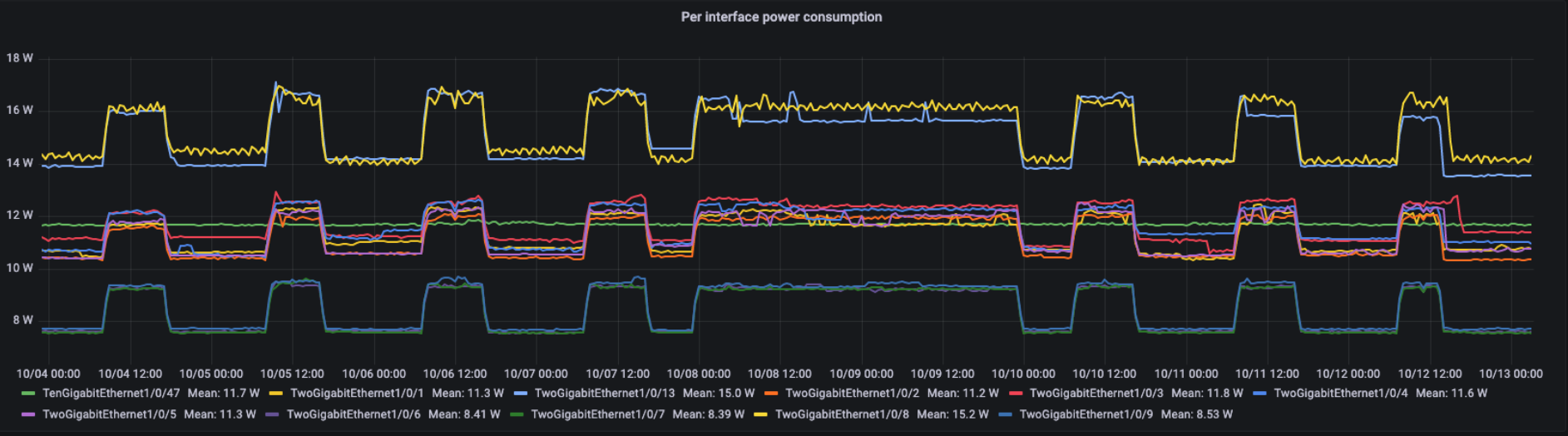
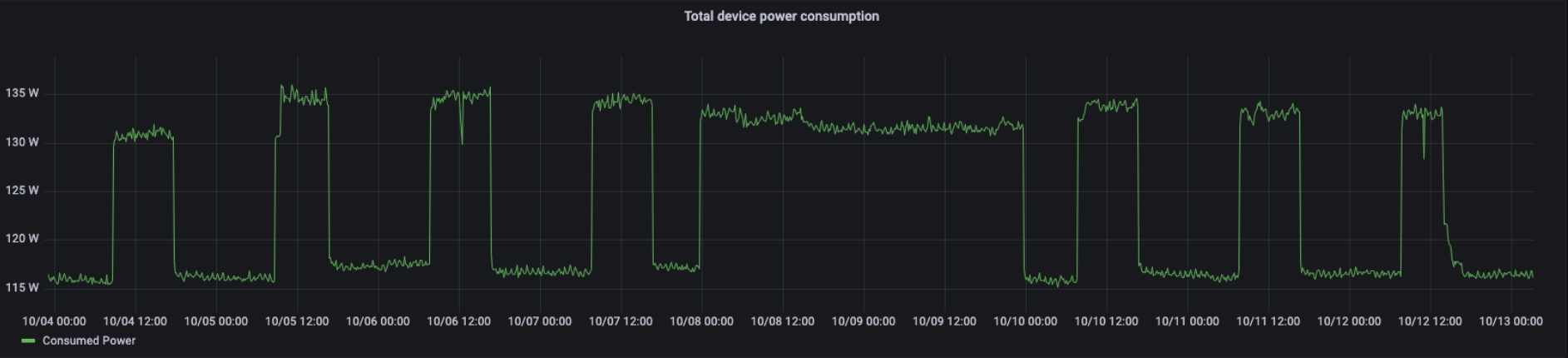
wireless profile power "Off Work Hours"  
0 radio 6ghz state shutdown  
1 radio secondary-5ghz state shutdown  
2 usb 0 state disable  
3 radio 5ghz state shutdown

wireless profile calendar-profile name "Off Work 5PM to Midnight"  
day monday  
day tuesday  
day wednesday  
day thursday  
day friday  
recurrence weekly  
start 17:00:00 end 23:59:59

wireless profile calendar-profile name "Off Work Midnight to 8AM"  
day monday  
day tuesday  
day wednesday  
day thursday  
day friday  
recurrence weekly  
start 00:00:00 end 08:00:00

ap profile default-ap-profile  
calendar-profile "Workday 5pm to Midnight"  
action power-saving-mode power-profile "Off Work Hours"  
calendar-profile "Workday Midnight to 8am"  
action power-saving-mode power-profile "Off Work Hours"

Interface TenGigabitEthernet1/0/47 + TwoGigabitEthernet1/0/1 + TwoGigabitEth... switch SJC14F1-WTME-C9K-48UXM



| POE Interface |                          |                 |                 |                   |             |  |
|---------------|--------------------------|-----------------|-----------------|-------------------|-------------|--|
| device        | intf_name                | power_admin_max | max_power_drawn | power_consumption | power_to_pd |  |
| C9130AXE-B    | TenGigabitEthernet1/0/47 | 60              | 13.3            | 11.6              | 33.3        |  |
| CW9164I-B     | TwoGigabitEthernet1/0/1  | 60              | 15.6            | 11.9              | 31.4        |  |
| C9136I-B      | TwoGigabitEthernet1/0/13 | 60              | 21.3            | 17.0              | 41.9        |  |
| C9136I-B      | TwoGigabitEthernet1/0/16 | 60              | 6.25            | 0.950             | 41.9        |  |
| CW9164I-B     | TwoGigabitEthernet1/0/2  | 60              | 13.3            | 12.5              | 31.4        |  |
|               | Ethernet1/0/3            |                 |                 | 12.6              | 34.1        |  |



# Monitoring & Automation Impact

Up to **25%↓** of Energy usage **reduction** on POE switchports with IOT endpoints turned off ~ 9hrs/Day

Minimum of **12%↓** Energy usage **reduction** for controller-based Cisco Catalyst Access Points

**100 %↑** Visibility on POE Power Consumption

\*based on use case and data presented in demos

# What's changed with power in the 17.15 release?

## Previously

- Instantaneous power only

## Today

- Instantaneous power
- Power over time
  - Smart Buildings use case:
    - 48 PoE ports create too much data, now we can use buckets & can collect data at less frequent intervals

# Enhanced Energy Metering

## Energy Metering System Power

```
Meter start time 2024-10-22 10:36:34 PST
Energy Data For Last 180 Minute
```

| Mod | Model No  | System Energy<br>(MilliWattSec) | System Meter Update<br>Time |
|-----|-----------|---------------------------------|-----------------------------|
| 1   | C9300-24H | 234894600                       | 2024-10-22 11:32:39 PST     |

## Energy Metering PoE Port Power

```
SB-Salone1-C9324H#sho power inline meter
```

| Module | Available<br>(Watts) | Used<br>(Watts) | Remaining<br>(Watts) |
|--------|----------------------|-----------------|----------------------|
| 1      | 860.0                | 120.0           | 740.0                |

| Interface | Meter Update<br>Time    | Hourly Metered<br>Value (MilliWattSec) | Metered Energy in MilliWattSec (15min Buckets)      |
|-----------|-------------------------|----------------------------------------|-----------------------------------------------------|
| Gi1/0/1   | 2024-10-22 11:21:09 PST | 0                                      | 0-0-0-0-0-0-0-0-0-0-0-0-0-0-0-0                     |
| Gi1/0/2   | 2024-10-22 11:21:09 PST | 10507380                               | 1465368-3553224-3558320-1930468-0-0-0-0-0-0-0-0-0-0 |
| Gi1/0/3   | 2024-10-22 11:21:09 PST | 0                                      | 0-0-0-0-0-0-0-0-0-0-0-0-0-0-0-0                     |
| Gi1/0/4   | 2024-10-22 11:21:09 PST | 0                                      | 0-0-0-0-0-0-0-0-0-0-0-0-0-0-0-0                     |
| Gi1/0/5   | 2024-10-22 11:21:09 PST | 0                                      | 0-0-0-0-0-0-0-0-0-0-0-0-0-0-0-0                     |
| Gi1/0/6   | 2024-10-22 11:21:09 PST | 0                                      | 0-0-0-0-0-0-0-0-0-0-0-0-0-0-0-0                     |

```
SB-Salone1-C9324H#sho ver | i time
SB-Salone1-C9324H uptime is 45 minutes
Uptime for this control processor is 46 minutes
```

# Enhanced Energy Metering

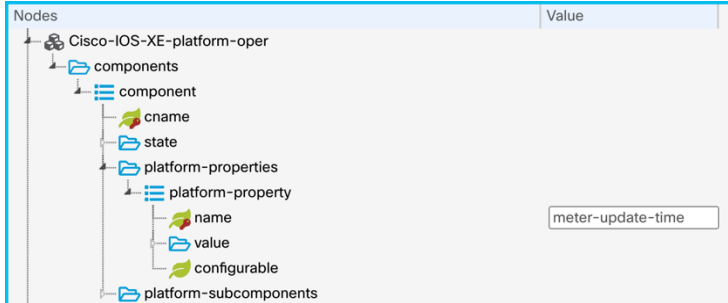
## Energy Metering - System Power

|                |                                                       |
|----------------|-------------------------------------------------------|
| Name           | cname                                                 |
| Nodetype       | leaf                                                  |
| Datatype       | string                                                |
| Description    | References component name                             |
| Module         | Cisco-IOS-XE-platform-oper                            |
| Revision       | 2023-03-01                                            |
| Xpath          | /components/component/cname                           |
| Prefix         | platform-ios-xe-oper                                  |
| Namespace      | http://cisco.com/ns/yang/Cisco-IOS-XE-platform-oper   |
| Schema Node Id | /components/component/cname                           |
| Key            | true                                                  |
| Access         | read-only                                             |
| Operations     | <ul style="list-style-type: none"><li>"get"</li></ul> |

## Energy Metering - PoE Port Power

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| Name            | power-consumption                                     |
| Nodetype        | leaf                                                  |
| Datatype        | decimal64                                             |
| Description     | Actual Power consumed by power device                 |
| Module          | Cisco-IOS-XE-poe-oper                                 |
| Revision        | 2024-03-01                                            |
| Xpath           | /poe-oper-data/poe-port-detail/power-consumption      |
| Prefix          | poe-ios-xe-oper                                       |
| Namespace       | http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper        |
| Schema Node Id  | /poe-oper-data/poe-port-detail/power-consumption      |
| Units           | Watts                                                 |
| Min             | -92233720368547758.08                                 |
| Max             | 92233720368547758.07                                  |
| Fraction Digits | 2                                                     |
| Access          | read-only                                             |
| Operations      | <ul style="list-style-type: none"><li>"get"</li></ul> |

# Energy Metering using the API



Show same 2 things as CLI  
Get system energy + time

## NETCONF GET operation

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <get>
    <filter>
      <components xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-platform-oper">
        <component>
          <platform-properties>
            <platform-property>
              <name>meter-update-time</name>
            </platform-property>
          </platform-properties>
        </component>
      </components>
    </filter>
  </get>
</rpc>
```

## Response

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
  xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:820e9cb4-fc8d-4f3d-bad9-4b09bab635e2">
  <data>
    <components xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-platform-oper">
      <component>
        <platform-properties>
          <platform-property>
            <name>meter-update-time</name>
            <value>
              <string>1730240928747</string>
            </value>
            <configurable>false</configurable>
          </platform-property>
        </platform-properties>
      </component>
    </components>
  </data>
</rpc-reply>
```

Epoch time  
= Tuesday, October 29th



# PoE API

☰

 YANG Suite / Exploring YANG / YANG set "smartbuilding-salone1-default-yangset" / Modules

admin

⌂

🗨

?

Select a YANG set

smartbuilding-salone1-default-yangset

Select YANG module(s)

Cisco-IOS-XE-poe-oper

Load module(s)

Icon legend

Search XPaths

Search nodes

Expand all nodes

☒ Display schema nodes only ☐ Display all nodes

Cisco-IOS-XE-poe-oper

poe-oper-data

poe-port

poe-port-detail

intf-name

power-used

pd-class

device-detected

device-name

police

power-admin-max

power-from-pse

power-to-pd

power-consumption

max-power-drawn

oper-state

admin-state

oper-power

admin-police

oper-police

cutoff-power-police

power-negotiation-used

four-pair-poe-supported

four-pair-poe-enabled

four-pair-pd-arch

over-current-counter

short-current-counter

power-denied-counter

Node Properties

|                |                                                             |
|----------------|-------------------------------------------------------------|
| Name           | poe-port-detail                                             |
| Nodetype       | list                                                        |
| Description    | List of PoE interfaces, keyed by interface name             |
| Module         | Cisco-IOS-XE-poe-oper                                       |
| Revision       | 2024-07-01                                                  |
| Xpath          | /poe-oper-data/poe-port-detail                              |
| Prefix         | poe-ios-xe-oper                                             |
| Namespace      | http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper              |
| Schema Node Id | /poe-oper-data/poe-port-detail                              |
| Keys           | <ul style="list-style-type: none"><li>"intf-name"</li></ul> |
| Access         | read-only                                                   |
| Operations     | <ul style="list-style-type: none"><li>"get"</li></ul>       |

Reference URL: <https://tools.ietf.org/html/rfc6020#section-7.8>

7.8. The list Statement

The "list" statement is used to define an interior data node in the schema tree. A list node may exist in multiple instances in the data tree. Each such instance is known as a list entry. The "list" statement takes one argument, which is an identifier, followed by a block of substatements that holds detailed list information.

A list entry is uniquely identified by the values of the list's keys, if defined.

# PoE API (cont.) NETCONF GET

YANG Set

smartbuilding-salone1-default-yangset

Module(s)

Cisco-IOS-XE-poe-oper X

Load Module(s)

NETCONF Operation

get

Device

SB-salone2

Edit Device

Open Device Window

YANG Tree

Replays

RPC Options...

Build RPC

Run RPC(s)

Clear RPC(s)

Nodes

Value

Cisco-IOS-XE-poe-oper

poe-oper-data

poe-port

poe-port-detail

intf-name

power-used

pd-class

device-detected

device-name

police

power-admin-max

power-from-pse

power-to-pd

power-consumption

max-power-drawn

oper-state

admin-state

oper-power

admin-police

oper-police

cutoff-power-police

power-negotiation-used

four-pair-poe-supported

four-pair-poe-enabled

four-pair-pd-arch

over-current-counter

short-current-counter

power-denied-counter

conn-type

<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">  
<get>  
<filter>  
<poe-oper-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper">  
<poe-port-detail/>  
</poe-oper-data>  
</filter>  
</get>  
</rpc>

## NETCONF GET operation

### Response

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0">  
<data>  
<poe-oper-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-poe-oper">  
<poe-port-detail>  
<intf-name>GigabitEthernet1/0/2</intf-name>  
<power-used>30.0</power-used>  
<pd-class>pd-ieee4</pd-class>  
<device-detected>true</device-detected>  
<device-name>leee PD</device-name>  
<police>false</police>  
<power-admin-max>90.0</power-admin-max>  
<power-from-pse>30.0</power-from-pse>  
<power-to-pd>30.0</power-to-pd>  
<power-consumption>3.91</power-consumption>  
<max-power-drawn>5.08</max-power-drawn>  
</poe-port-detail>  
</poe-oper-data>  
</data>  
</rpc-reply>
```

# Subscription to the new platform data

```
telemetry ietf subscription 79007
encoding encode-kvgpb
filter xpath /platform-ios-xe-oper:components/component/platform-properties/platform-property
source-address 172.26.202.111
stream yang-push
update-policy periodic 300000
receiver ip address 172.26.202.69 57000 protocol grpc-tcp
```

```
telemetry ietf subscription 79008
encoding encode-kvgpb
filter xpath /poe-ios-xe-oper:poe-oper-data/poe-port-detail
source-address 172.26.202.111
stream yang-push
update-policy periodic 30000
receiver ip address 172.26.202.69 57000 protocol grpc-tcp
```

```
telemetry ietf subscription 79007
encoding encode-kvgpb
filter xpath /platform-ios-xe-oper:components/component/platform-properties/platform-property
source-address 172.26.202.111
stream yang-push
update-policy periodic 300000
receiver ip address 172.26.202.69 57000 protocol grpc-tcp
```

```
telemetry ietf subscription 79008
encoding encode-kvgpb
filter xpath /poe-oper-data/poe-port-detail
source-address 172.26.202.111
stream yang-push
update-policy periodic 30000
receiver ip address 172.26.202.69 57000 protocol grpc-tcp
```



# Device Optimization

How can I troubleshoot and manage image upgrades for all 5000 new switches reliably, efficiently and at scale?





# Livetools: YANG support for common networking operations

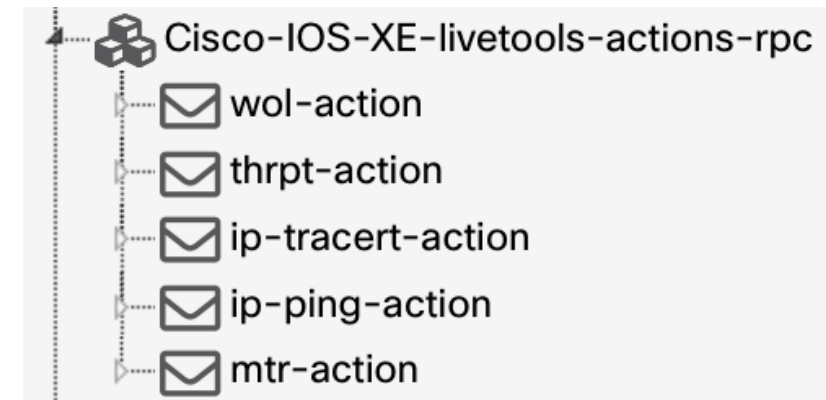
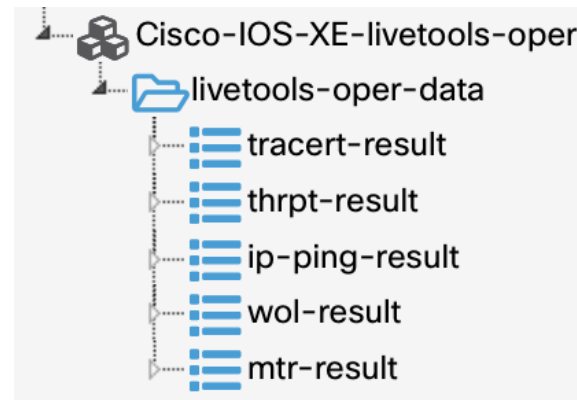
- |         |               |                |
|---------|---------------|----------------|
| 1. MTR  | 3. Throughput | 5. Wake-On-LAN |
| 2. Ping | 4. Traceroute |                |



# Cisco-IOS-XE-livetools: Ping, Traceroute, MTR, etc

- 5 common network management tasks including ping and traceroute are now available through the YANG API (mtr, ping, tracerout, throughput, WOL)
- They are called from the “RPC” model with variable inputs like IP address, etc
  - A “JobID” is returned to programmatically retrieve the results
- The Results are available by querying the “oper” model

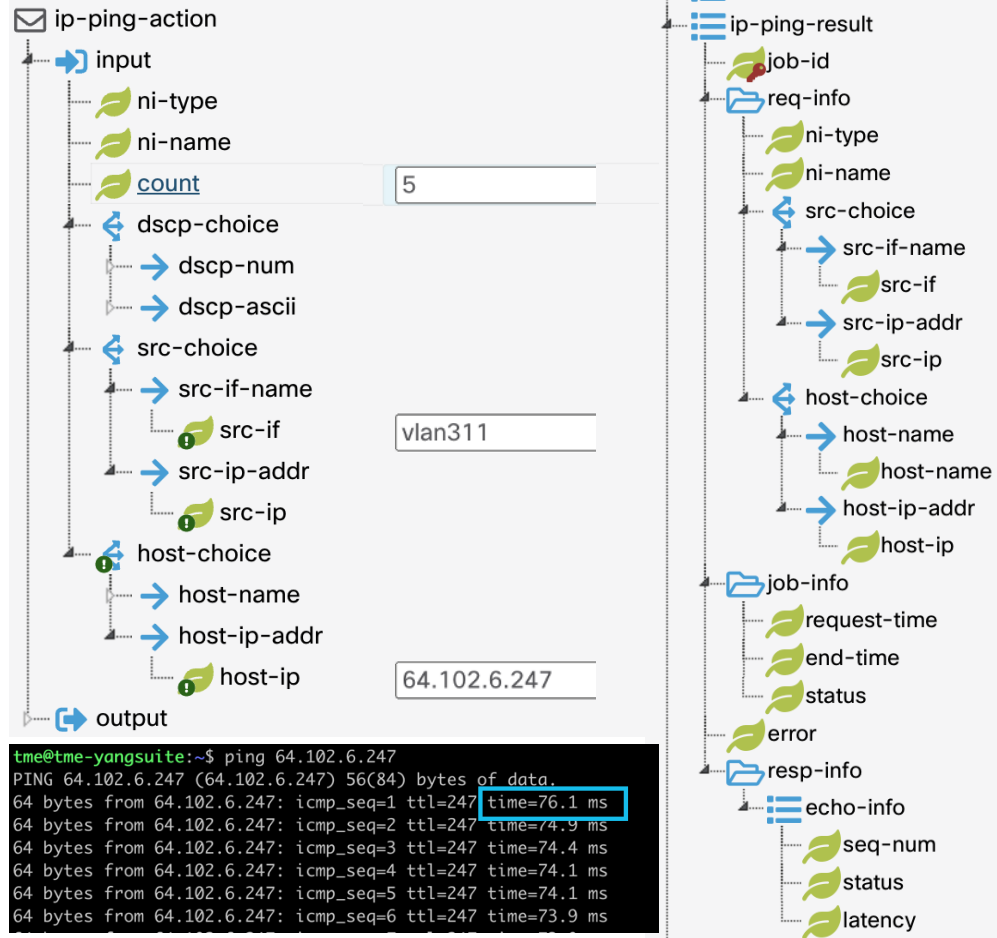
1. MTR
2. Ping
3. Throughput
4. Traceroute
5. Wake-On-LAN



These “live tools” enable additional programmatic network connectivity troubleshooting and validation

# Ping RPC

The ping RPC measures the network latency in milliseconds to the specified IP or hostname



## RPC TO EXEC PING

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <ip-ping-action xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-livetools-actions-rpc">
    <count>5</count>
    <src-if>vlan311</src-if>
    <host-ip>64.102.6.247</host-ip>
  </ip-ping-action>
</rpc>
```

## RPC TO GET PING RESULTS

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <get>
    <filter>
      <live-tools-oper-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-livetools-oper">
        <ip-ping-result>
          <job-id>1</job-id>
        </ip-ping-result>
      </live-tools-oper-data>
    </filter>
  </get>
</rpc>
```

## RESPONSE FROM API

```
<live-tools-oper-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-livetools-oper">
  <ip-ping-result>
    <job-id>1</job-id>
    <req-info>
      <ni-type>net-inst-default</ni-type>
      <ni-name/>
      <src-if>vlan311</src-if>
      <host-ip>64.102.6.247</host-ip>
    </req-info>
    <job-info>
      <request-time>2024-09-11T23:34:01.976+00:00</request-time>
      <end-time>2024-09-11T23:34:02.348+00:00</end-time>
      <status>job-completed</status>
    </job-info>
    <error>ip-ping-no-err</error>
    <resp-info>
      <echo-info>
        <seq-num>1</seq-num>
        <status>echo-success</status>
        <latency>75</latency>
      </echo-info>
      <echo-info>
        <seq-num>2</seq-num>
        <status>echo-success</status>
        <latency>74</latency>
      </echo-info>
    </resp-info>
  </ip-ping-result>
</live-tools-oper-data>
```

# App Hosting



# C9K, C9350 Application Hosting Throughput testing

Another example of AI tooling use:

Q: How much throughput can the Docker container on the C9K support ?

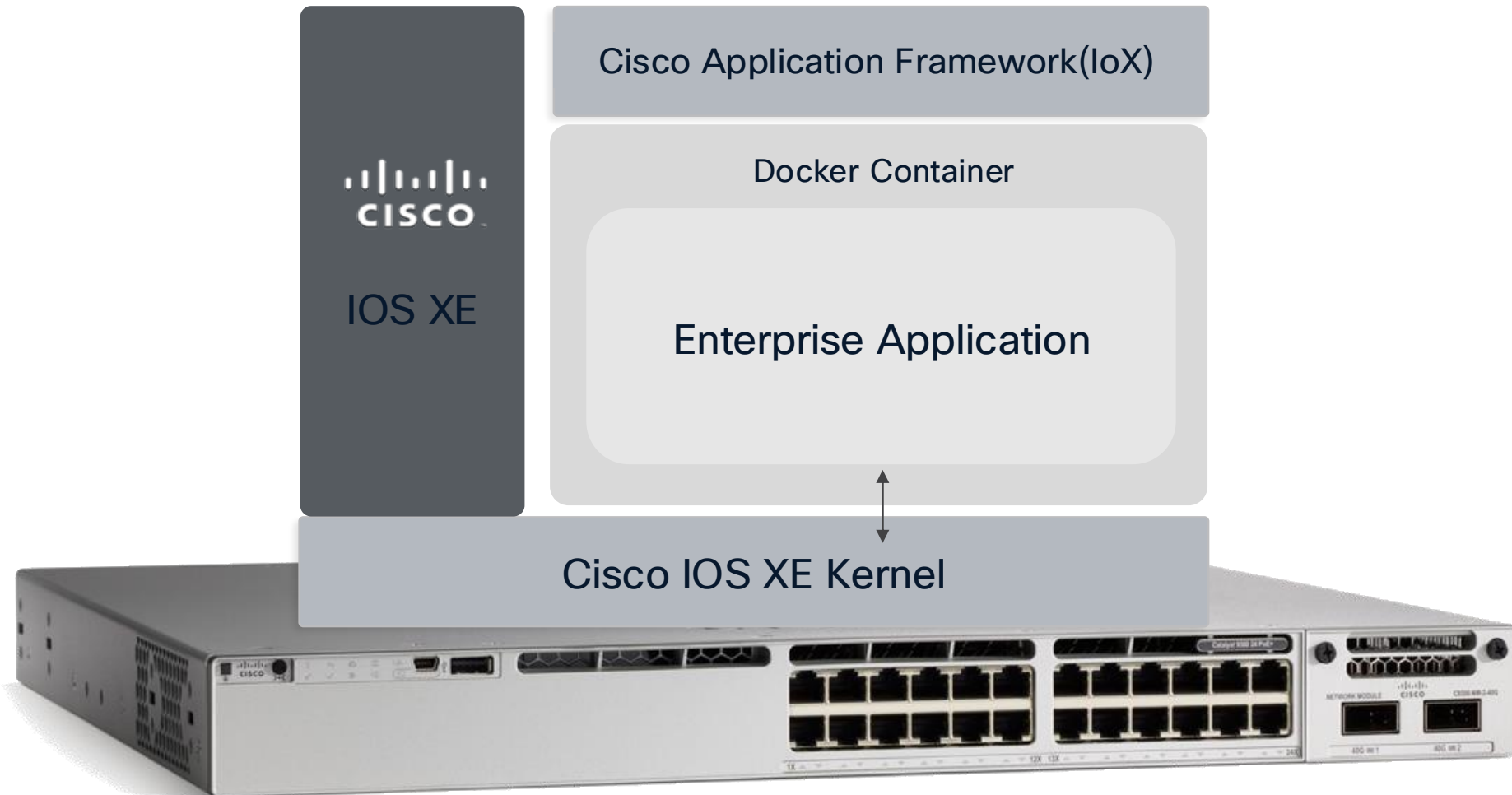
A: <https://github.com/jeremycohoe/cisco-ios-xe-application-hosting-throughput>

## 1. Executive Summary

| <div>Key Findings:</div> <ul style="list-style-type: none"><li><b>C9350</b> Best single-direction throughput: 4.24 Gbps forward, 9.35 Gbps reverse — fastest container RX processing</li><li><b>C9300X</b> Best bidirectional aggregate: 12.16 Gbps (8+8 streams) — 12% better than C9350 in bidir mode</li><li><b>C9300</b> Hardware-limited at 1G AppGig: 939 Mbps in all directions — but 98.9% line-rate efficiency, IOx stack is excellent</li><li>All platforms: Reverse (Container→Host) achieves near line-rate thanks to TSO/GSO offloads</li><li>All platforms: Forward (Host→Container) is bottlenecked by single-core softirq RX processing in container namespace</li><li>Optimization: <code>hold-queue 4096 out</code> on AppGig eliminates interface drops; <code>sysctl/privileged/jumbo</code> all blocked by IOx restrictions</li></ul> |            |            |           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|------------|-----------|
| METRIC                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | C9350      | C9300X     | C9300     |
| TCP Forward (best)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 4.24 Gbps  | 3.15 Gbps  | 942 Mbps  |
| TCP Reverse (best)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 9.35 Gbps  | 9.32 Gbps  | 935 Mbps  |
| Bidir Aggregate (best)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 10.85 Gbps | 12.16 Gbps | 1.67 Gbps |
| AppGig Link Speed                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 10 Gbps    | 10 Gbps    | 1 Gbps    |
| Link Efficiency (Fwd)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 42%        | 32%        | 94%       |

- Original Prompt: To create a Docker container and then export it to .tar format, and then copy to the USBFlash on the C9350 switch at 10.85.134.65 and turn on the Docker container as part of the AppHosting feature. The docker container should be a basic simple but full/complete version of Ubuntu and we will use this for running iperf and iperf3 similar tools to test the throughput into the container, you recommend tools and how to run the tools to validate speeds.
- Enhanced Prompt Request: What additional details are needed to discuss or clarify ?
- Enhanced Prompt: Included in the Github Repo

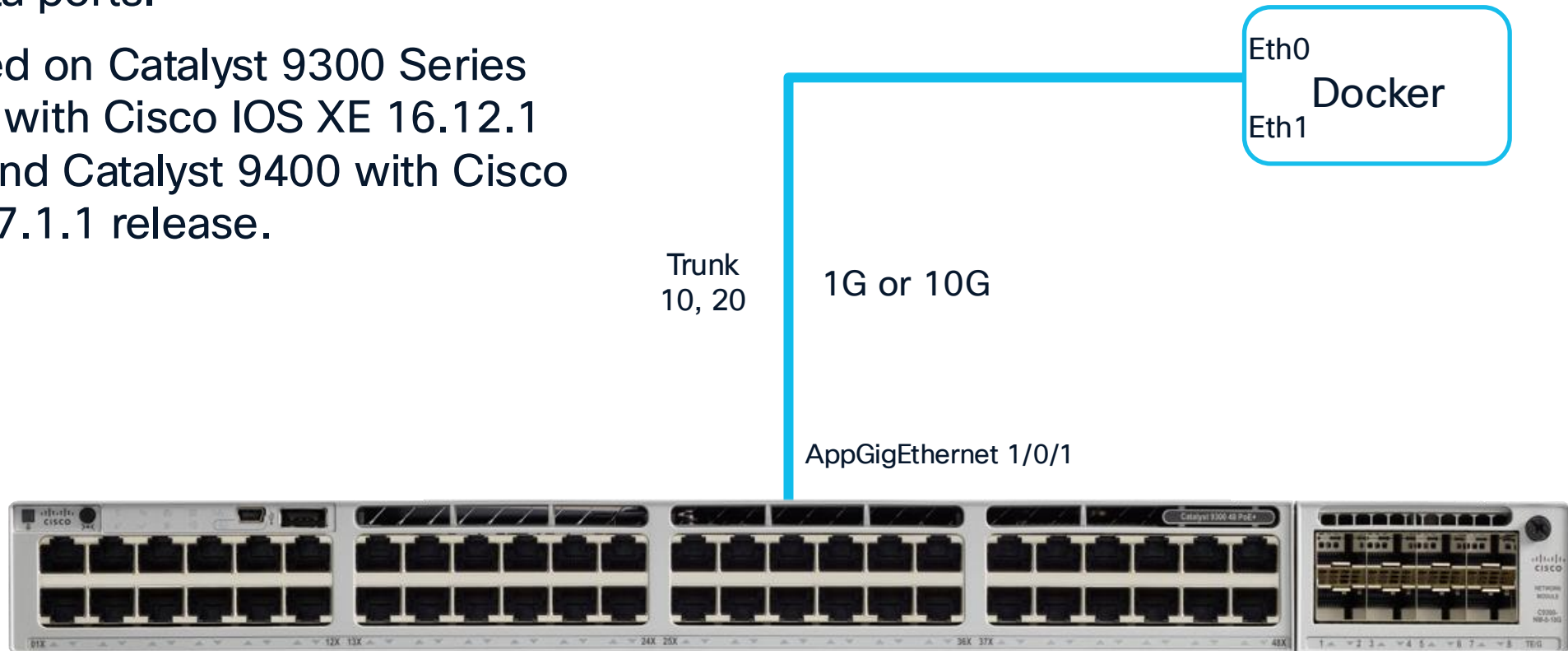
# Catalyst 9000 Application Hosting Infra



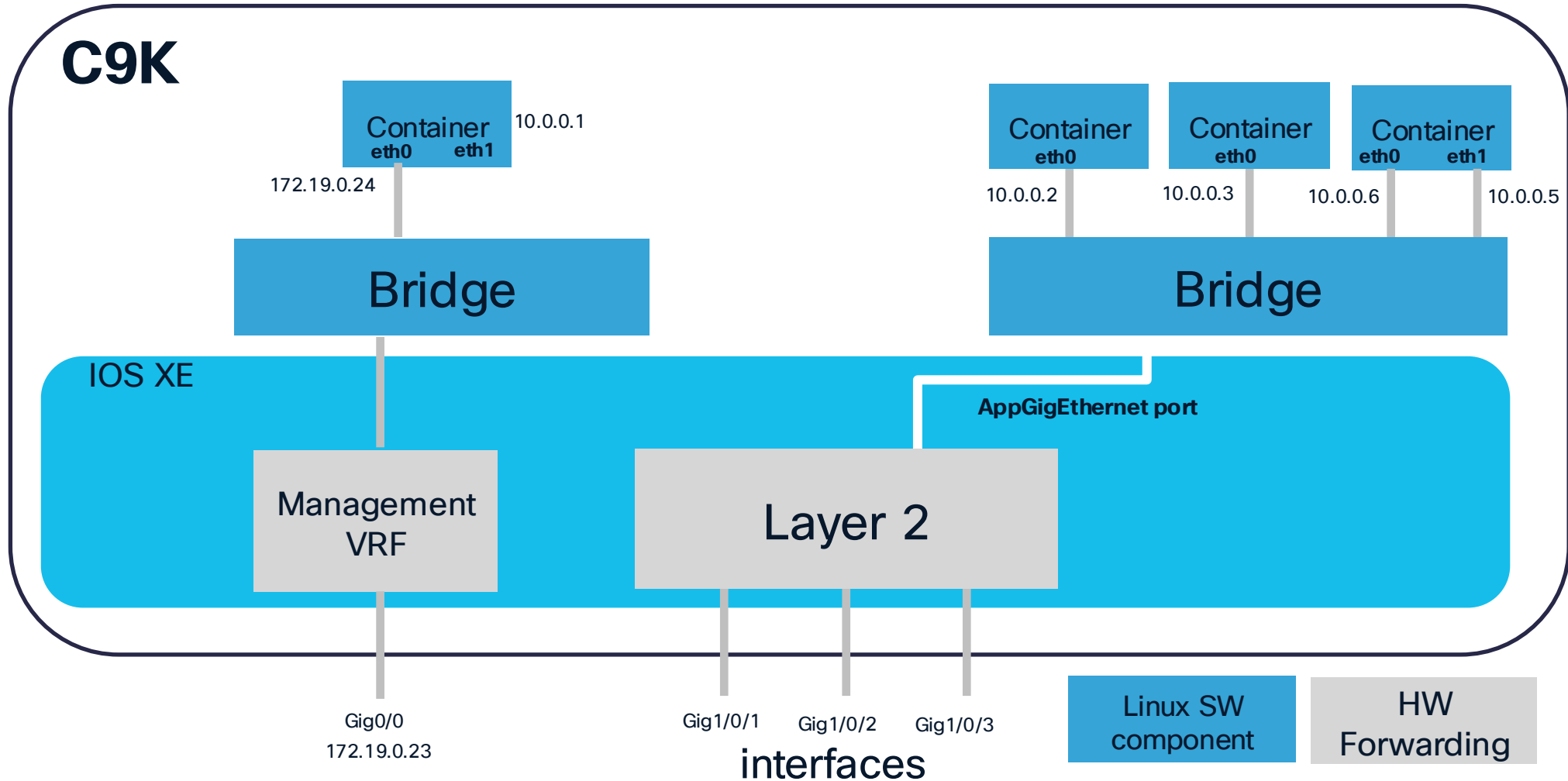


# AppGigEthernet Port

- What is AppGigEthernet Port ?
- An internal hardware data port which is hardware-switched to the front-panel data ports.
- Introduced on Catalyst 9300 Series switches with Cisco IOS XE 16.12.1 release and Catalyst 9400 with Cisco IOS XE 17.1.1 release.

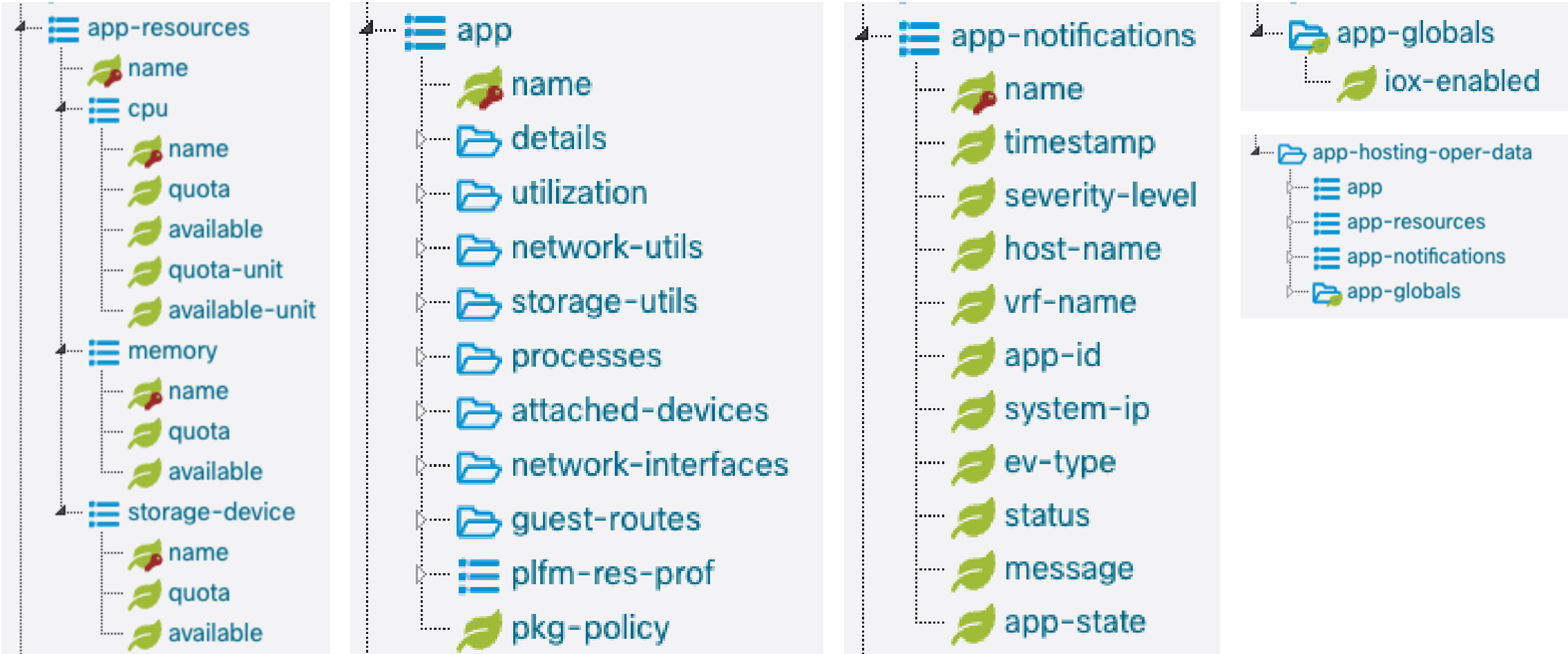


# Catalyst 9000 Containers Networking

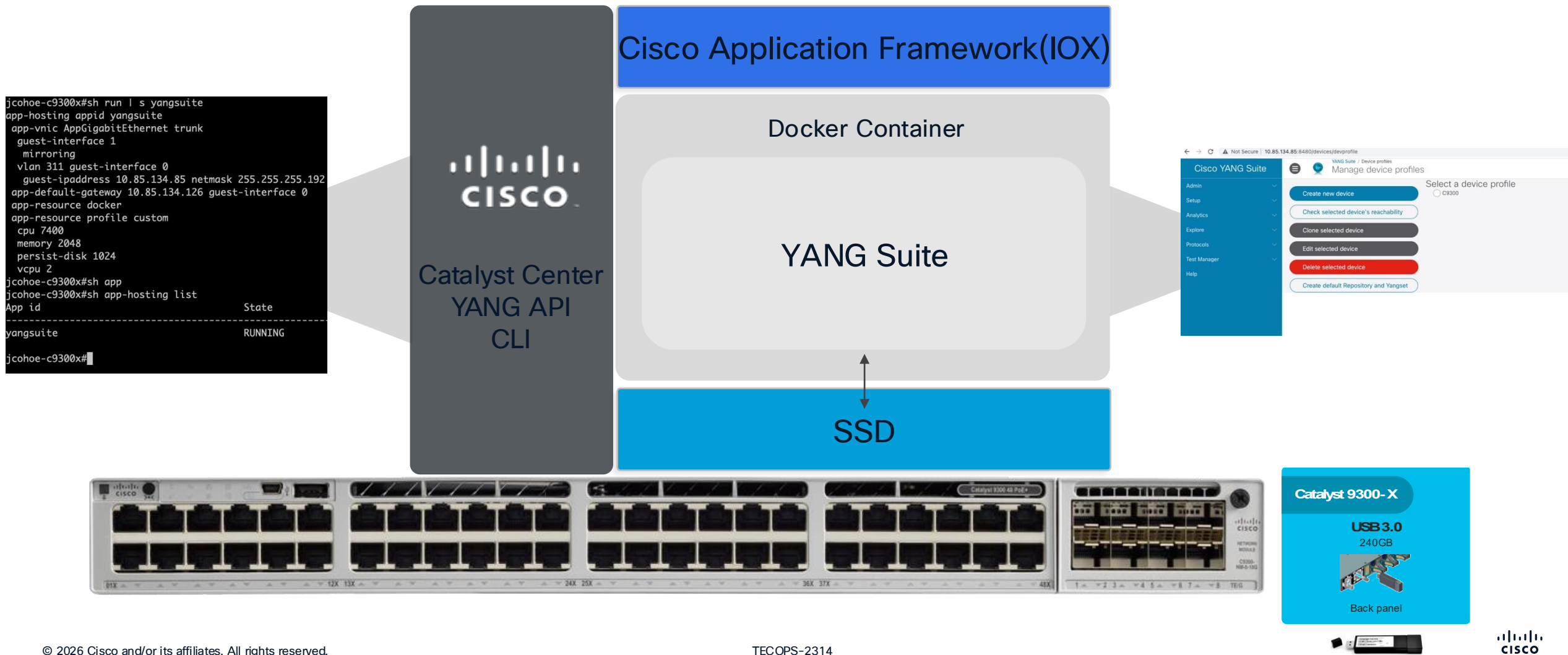


# App Hosting Operational Data

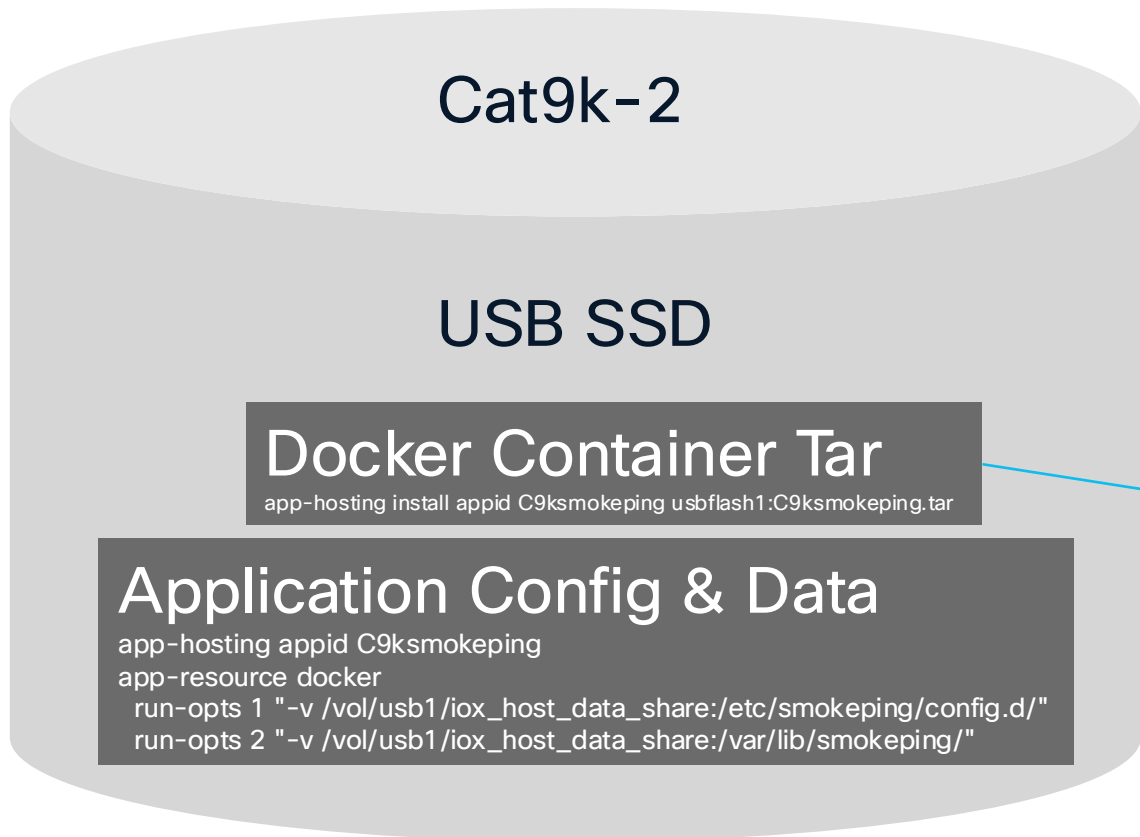
Like all other features, Application Hosting can be configured with the YANG API and operational data is available



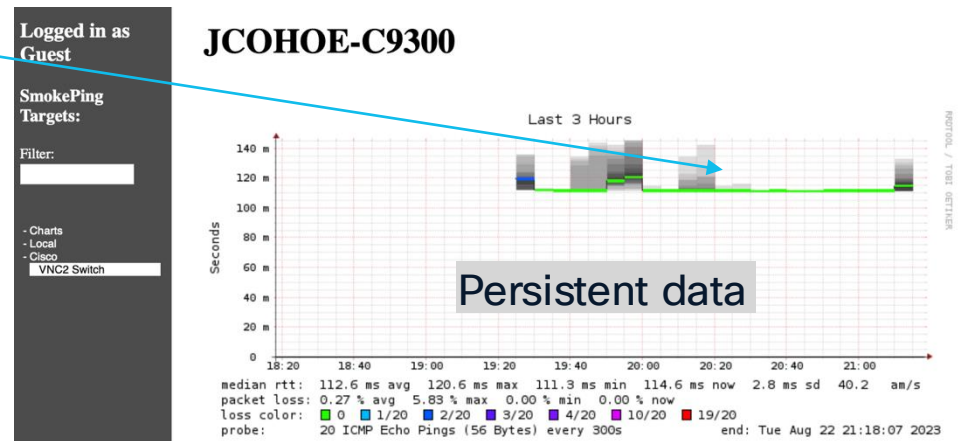
# Application Hosting of YANG Suite on the C9K



# Application Hosting of Smoke Ping app on C9K



1. The Docker Tar file and any persistent application files are placed onto the USBFlash1 SSD as part of day0 bootstrapping setup.
2. Copy using SCP/SFTP, tftp/http, or YANG API copy RPC with NETCONF or RESTCONF
3. Application data is saved to USB Flash and available after container or switch reload





# Configuring persistent data folders

## CLI Examples for Persistence:

app-hosting appid C9ksmokeping

app-resource docker

run-opts 1 "-v /vol/usb1/iox\_host\_data\_share:/etc/smokeping/config.d/"

run-opts 2 "-v /vol/usb1/iox\_host\_data\_share:/var/lib/smokeping/"

Switch USB Flash

:

Path inside container



run-opts 2 "-v /vol/usb1/iox\_host\_data\_share:/var/lib/smokeping/"

Same file inside and outside of container/switch USB disk

```
Cat9k-2#
Cat9k-2#app-hosting connect appid C9ksmokeping session
# ls -la /etc/smokeping/config.d/
total 52
drwxrwsrwx. 5 smokeping www-data 4096 Aug 22 19:18 .
drwxr-xr-x. 3 root root 4096 Aug 22 19:22 ..
-rw-rwSr--. 1 smokeping www-data 176 Aug 21 21:14 Alerts
-rw-rwSr--. 2 smokeping www-data 4096 Aug 22 19:18 Cisco
-rw-rwSr--. 1 smokeping www-data 236 Aug 21 21:14 Database
-rw-rwSr--. 1 smokeping www-data 490 Aug 21 21:14 General
drwxrwsr-x. 2 smokeping www-data 4096 Aug 21 21:16 Local
-rw-rwSr--. 1 smokeping www-data 874 Aug 21 21:14 Presentation
-rw-rwSr--. 1 smokeping www-data 138 Aug 21 21:14 Probes
-rw-rwSr--. 1 smokeping www-data 146 Aug 21 21:14 Slaves
-rw-rwSr--. 1 smokeping www-data 499 Aug 22 19:17 Targets
drwxrwsr-x. 2 smokeping www-data 4096 Aug 22 21:07 __sortercache
-rw-rwSr--. 1 smokeping www-data 225 Aug 21 21:14 pathnames
#
# exit
Cat9k-2#dir usbflash1:iox_host_data_share/
Directory of usbflash1:iox_host_data_share/

4456460 drwx      4096 Aug 22 2023 21:07:54 __sortercache
4456461 drwx      4096 Aug 22 2023 19:18:51 Cisco
4456456 -rw-      499 Aug 22 2023 19:17:42 Targets
4456458 drwx      4096 Aug 21 2023 21:16:05 Local
4456457 -rw-      225 Aug 21 2023 21:14:56 pathnames
4456453 -rw-      874 Aug 21 2023 21:14:55 Presentation
4456450 -rw-      176 Aug 21 2023 21:14:55 Alerts
4456455 -rw-      146 Aug 21 2023 21:14:55 Slaves
4456451 -rw-      236 Aug 21 2023 21:14:55 Database
4456452 -rw-      490 Aug 21 2023 21:14:55 General
4456454 -rw-      138 Aug 21 2023 21:14:55 Probes

117951578112 bytes total (109656801280 bytes free)
Cat9k-2#
```

<https://developer.cisco.com/docs/app-hosting/#!application-hosting-configuration/docker-runtime-options>

# Troubleshooting

# Troubleshooting

For each, refer to the programmability configuration guide for proper setup

<https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1718/b-1718-programmability-cg.html>

## NETCONF

- Ensure NETCONF is enabled
- Ensure port 830 is open
- Ensure the payload is correctly formatted using YANG Suite to generate payloads

```
jcohoe-c9300#debug netconf-yang ?
diagnostics  Debug netconf-yang diagnostics
level        Log level of netconf-yang processes
ssh          Debug netconf-yang ssh server
<cr>         <cr>
```

## RESTCONF / Terraform

Status codes and how to configure

200 – OK, API worked as expected

400 – Client Error

- Ensure RESTCONF is configured on device
- Ensure the formatting follows the expected:  
HTTPS:// IP / restconf / data / module : xpath
- If you are using HTTPS, ensure secure server is configured and configure a prox, if needed

500 – Server Error

- Ensure payload is correctly formatted using YANG Suite to generate payloads

```
jcohoe-c9300#debug restconf ?
level  Log level of restconf processes
<cr>   <cr>
```

## gNMI

- Ensure gNMI is enabled
- Ensure the port 50052 is open
- Ensure the payload is correctly formatted using YANG Suite to generate payloads
- For more in-depth logging, use “set platform software trace ?” to trace gnmi

```
jcohoe-c9300#set platform software trace gnmi switch active r0 gnmi ?
debug      Debug messages
emergency  Emergency possible message
error      Error messages
info       Informational messages
noise      Maximum possible message
notice     Notice messages
verbose    Verbose debug messages
warning    Warning messages
```



# Resources

# Cisco Catalyst Programmability Sessions at Cisco Live Amsterdam 2026

Monday Feb 9

**8:45 AM – 1:15 PM**

## **TECOPS-2314**

Automating All Things YANG, All the time - Programmability and Automation 101 with Cisco IOS XE

Tuesday Feb 10

**3:00 PM – 4:30 PM**

## **DEVNET-2295**

Become a Catalyst Image Upgrade Expert using Programmability

Wednesday Feb 11

**1:45 – 2:45 PM**

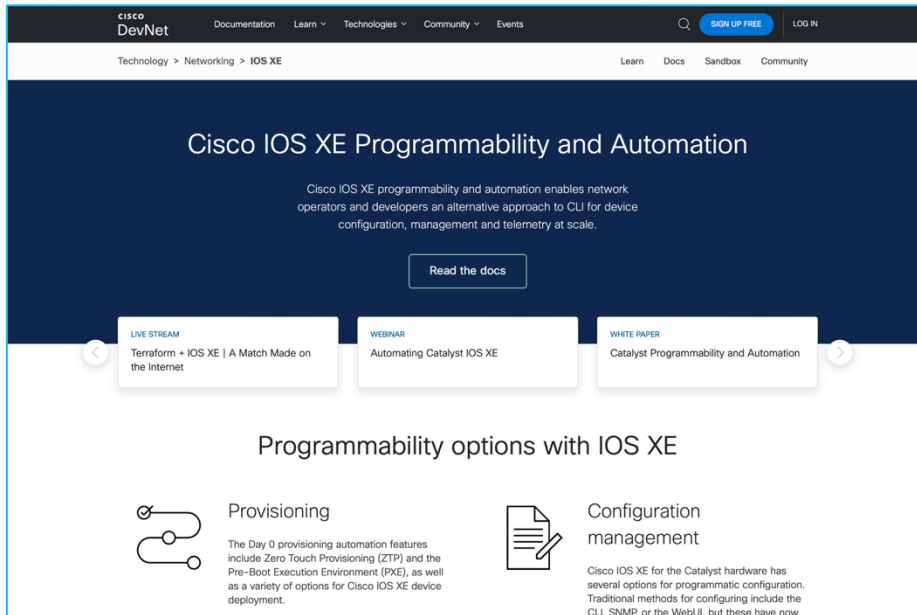
## **BRKDEV-2188**

Atomic Config Replace on Catalyst: A New Era in Network Management



# Programmability Website

The one-stop-shop for Cisco IOS XE Programmability resources including videos, white papers, labs and more!



- Community Forum
- IOS XE FAQ
- White Papers
- Code Exchange
- IOS XE Docs & Guide
- Learning Tracks and Labs
- Sandboxes
  - ... and more !



<https://developer.cisco.com/iosxe/>

# Cisco YANG Suite



## YANG API Testing and Validation Environment

Construct and test YANG based APIs over  
NETCONF, RESTCONF, gRPC and gNMI

IOS XE / IOS XR / NX OS platforms

On-Demand Learning Lab with YANG Suite in Docker  
<https://developer.cisco.com/learning/labs/intro-yangsuite/>

[developer.cisco.com/yangsuite](https://developer.cisco.com/yangsuite)

[github.com/CiscoDevNet/yangsuite](https://github.com/CiscoDevNet/yangsuite)



# API White Paper

Programmability and auto... ^



## Table of Contents

Programmability and automatio... -

Day 0: Provisioning automation

Day 1: Model-driven programmability

Day 2: Model-driven telemetry

Day N: Device optimization

Cisco IOS XE operational consistency

Yet Another Next Generation (Y... +

Day 1: Model-driven program... +

Tooling: Cisco YANG Suite +

Day 2: Model-driven telemetry +

Day N: Device optimization +

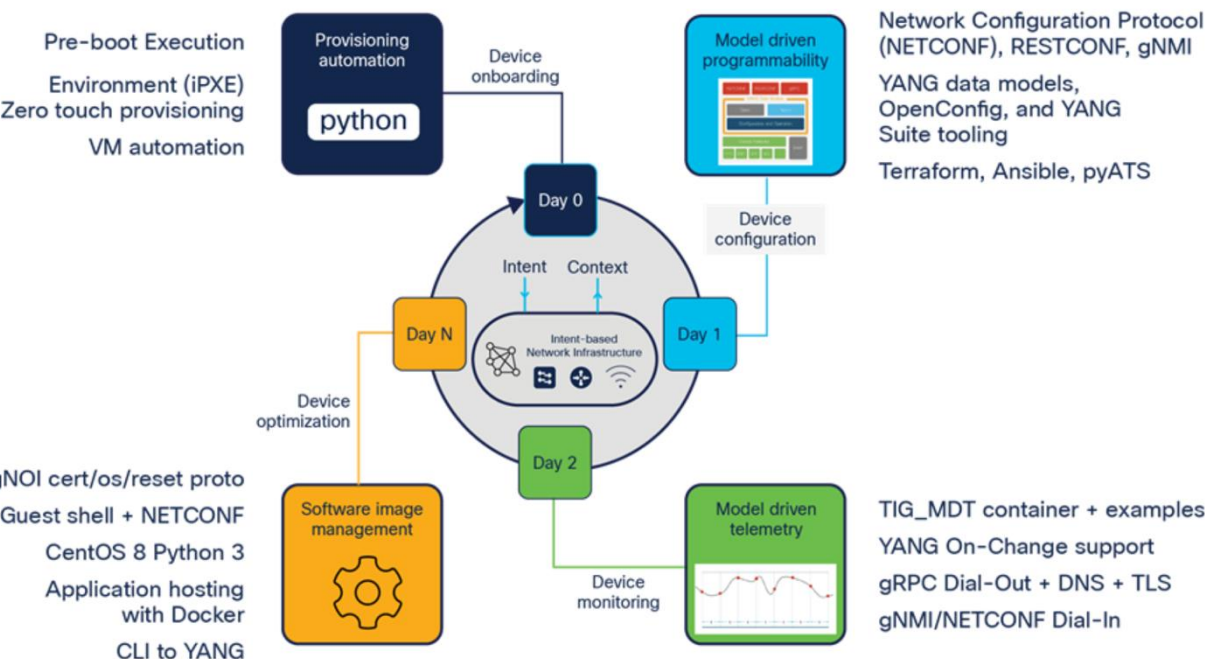
Conclusion

Additional resources +

Blogs

Products & Services / Switches / Campus LAN Switches - Access / Cisco Catalyst 9300 Series Switches /

## Catalyst Programmability and Automation



<http://cs.co/apiwp>



<http://cs.co/apiwppdf>

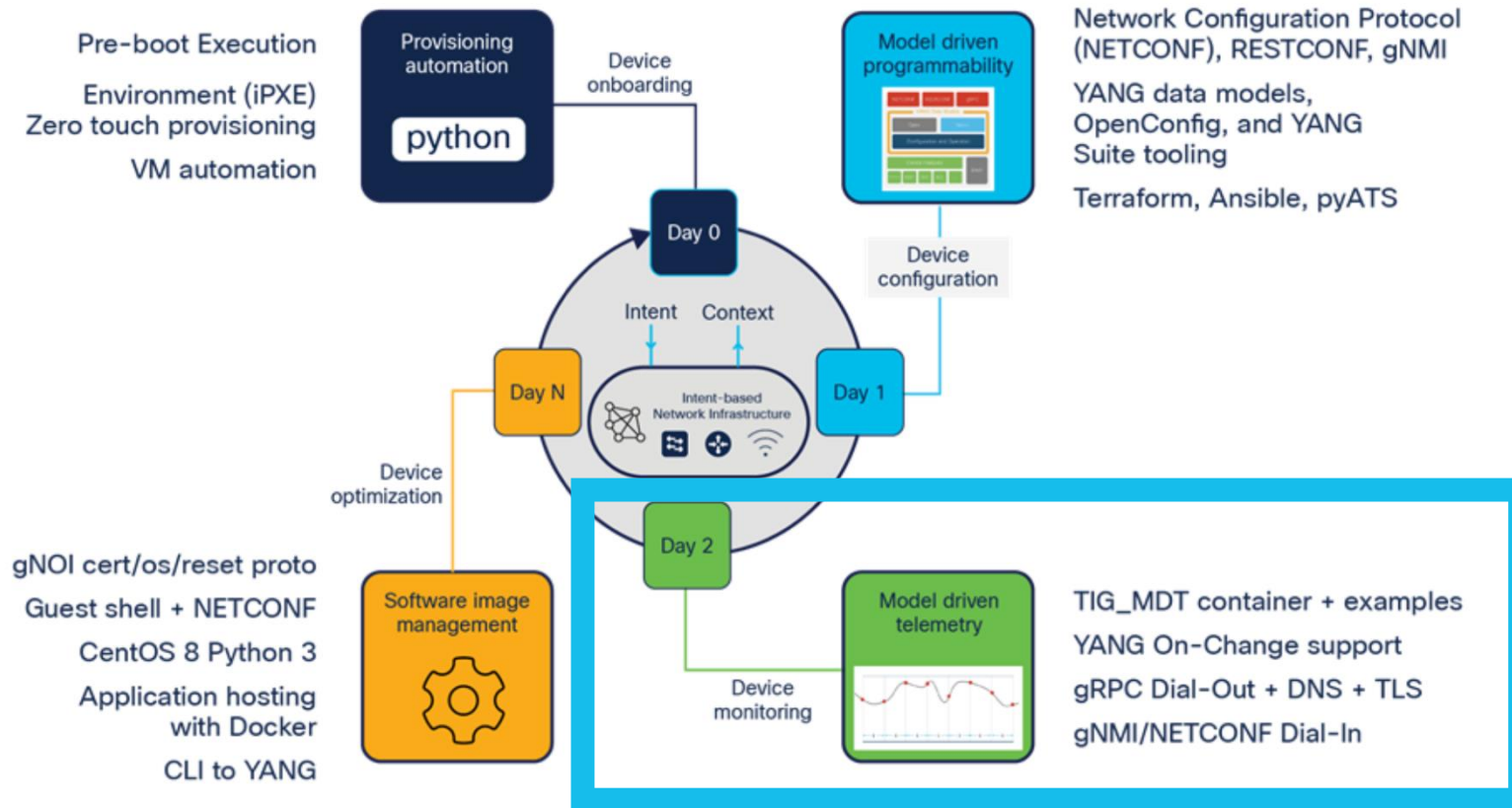
<https://www.cisco.com/c/en/us/products/collateral/switches/catalyst-9300-series-switches/nb-06-catalyst-programmability-automation-wp.html>

<https://www.cisco.com/c/en/us/products/collateral/switches/catalyst-9300-series-switches/nb-06-catalyst-programmability-automation-wp.pdf>

<https://www.youtube.com/watch?v=LdcK5PnPu2I>

# Model Drive Telemetry (MDT) White Paper

The Model Driven Telemetry White Paper includes examples, use cases and tooling related to telemetry. This paper is now available online and in PDF form!



View online: <https://www.cisco.com/c/en/us/products/collateral/switches/catalyst-9300-series-switches/model-driven-telemetry-wp.html>  
View as PDF: <https://www.cisco.com/c/en/us/products/collateral/switches/catalyst-9300-series-switches/model-driven-telemetry-wp.pdf>

Introduction to Cisco IOS ... ^ Q

Table of Contents

- Introduction to Cisco IOS XE
- Introduction to telemetry
- Benefits of model driven tele...
- Network monitoring challeng...
- Architecture and databases
- Dial-in and dial-out MDT +
- Publication notification optio...
- YANG data modeling language +
- Benchmarking and comparis... +
- Cisco controller solutions +
- Cloud solutions +
- Tooling +
- Dashboarding and validation +
- Configuration examples +
- Telemetry configuration man... +
- Troubleshooting and validati... +
- Best practices and lessons l... +
- Conclusion
- Resources

# dCloud Programmability

<https://dcloud.cisco.com>

“Cisco Catalyst 9000 IOS XE Programmability & Automation Lab v1”

<https://dcloud2.cisco.com/demo/catalyst-9000-ios-xe-programmability-automation-lab-v1>

## Use Cases:

### EVPN:

Ansible with CLI deployment of EVPN solutions  
EVPN management over RESTCONF/YANG with Postman  
Declarative EVPN fabric management with Terraform

## Tooling and Integrations

### YANG Suite

- NETCONF/RESTCONF/gNMI API
  - Ansible integration
- NETCONF/gNMI Dial-In Telemetry
- gRPC Dial-Out Telemetry receiver

### Telemetry

- TIG stack in Docker
- Grafana dashboard for device health

### Postman / RESTCONF

- EVPN fabric API calls

### Terraform/RESTCONF

- Declarative EVPN fabric management

### Ansible

- EVPN solution enablement using CLI

## Model Driven Telemetry

Telemetry configuration with CLI and YANG Suite  
Collection with TIG\_MDT container and tooling

## YANG Programmability

YANG Suite tooling and integrations to YANG API's  
Ansible integrations

## Ubuntu VM Details:

Syslog receiver from all switches  
TFTP config backup  
See slide

## Windows VM Details

VS Code

Terraform @ folder

Ansible @ folder

Chrome browser

YANG Suite, Grafana

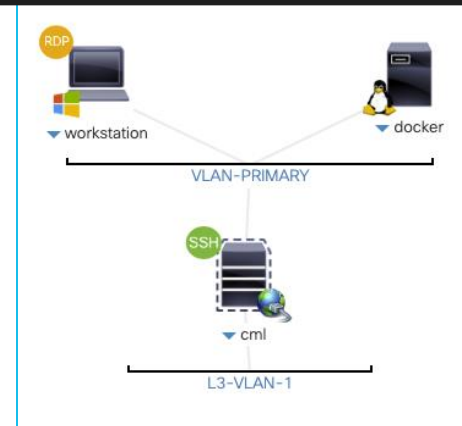
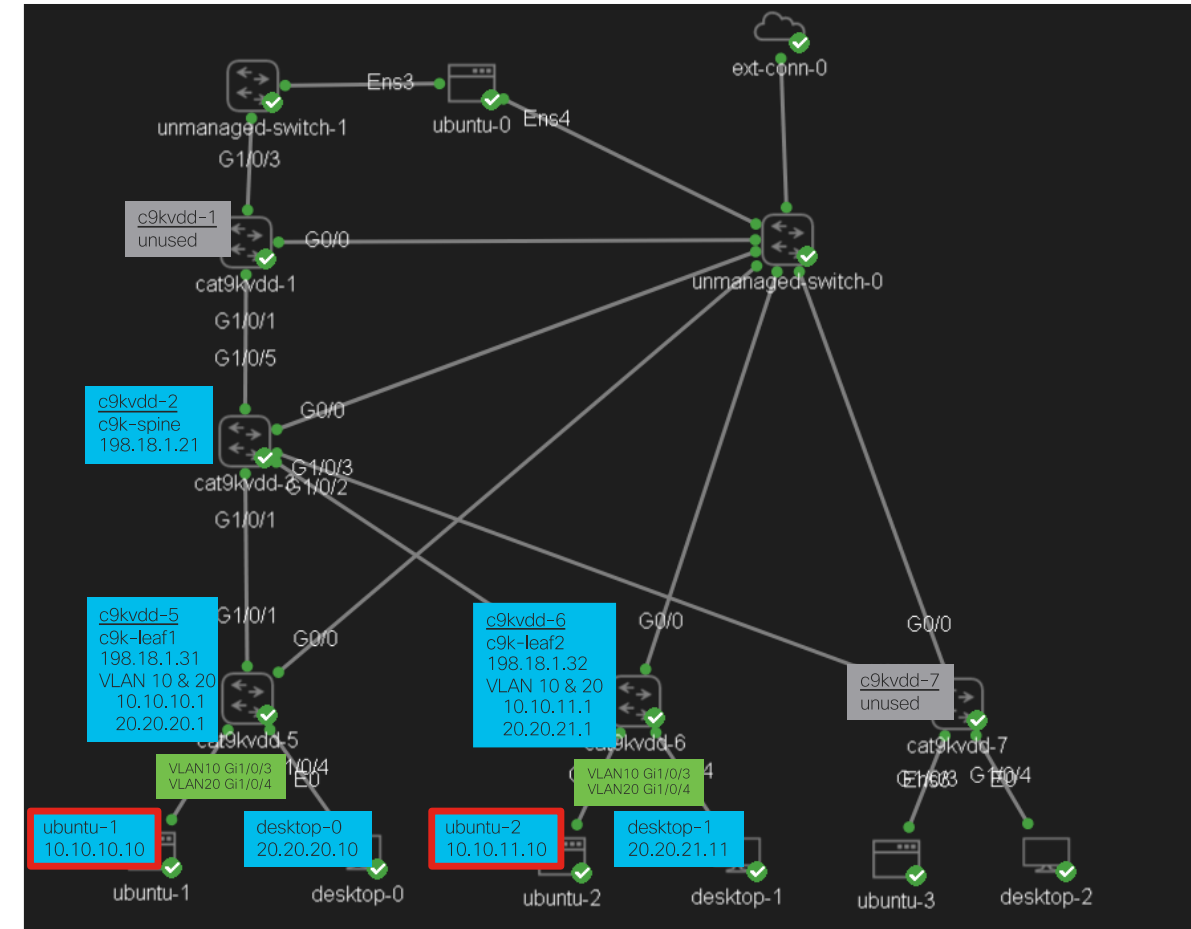
Bash/PS/Cmd shells

SSH into C9K or Ubuntu

Postman

Workspace for EVPN

## C9K VM's



|                         |                                |
|-------------------------|--------------------------------|
| VLAN1                   |                                |
| c9k-spine               | IP: 198.18.1.21<br>developer / |
| C1sco12345              |                                |
| c9k-leaf1               | IP: 198.18.1.31<br>developer / |
| C1sco12345              |                                |
| c9k-leaf2               | IP: 198.18.1.32<br>developer / |
| C1sco12345              |                                |
| c9kvdd-1 - unconfigured |                                |
| c9kvdd-7 - unconfigured |                                |



# DevNet Sandbox – overview for Campus and Enterprise

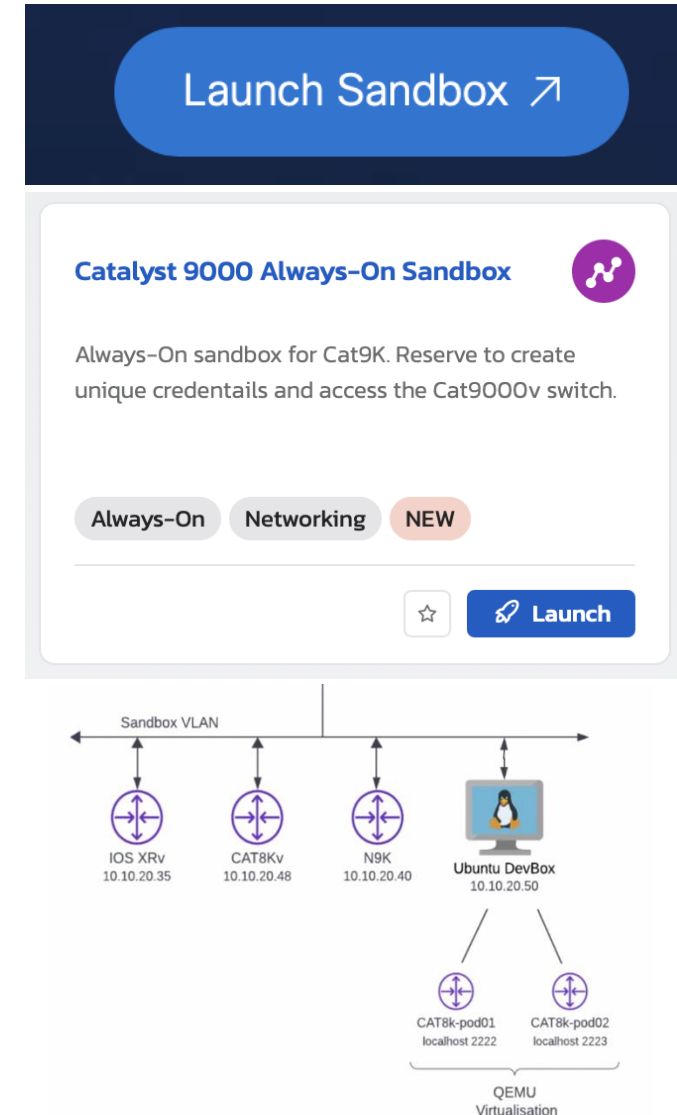
<https://developer.cisco.com/site/sandbox/>

1. **Reservable Physical**: C9200, C9300, C9300X including stacks  
About to go into production April 2025  
Usecases: Application Hosting, Power telemetry, etc
2. **Reservable Virtual**: C8KV Router + NX + XR + Ubuntu VM  
Usecases: Enterprise topology, dual-ZTP
3. **Always-On**: C9KV  
**Usecase: Virtual switch for basic config validation usecases**  
**DNS: devnetsandboxiosxec9k.cisco.com**  
**“Launch Sandbox” to get login credentials**
4. **Always-On**: C8KV  
DNS is devnetsandboxiosxe.cisco.com
5. **Reservable** Catalyst Center (Physical & Virtual)  
CML and C9KV, ISE for SDA

C9KV user is priv15 and has full permissions, there is reset automation for when you break it now too 😊

## **Additional enablement labs:**

1. YANG Suite “Learning Lab 2.0”  
Interactive guide with tool running in Docker container  
“YANG Suite as a service”
2. dCloud Programmability Lab  
EVPN topology with all programmability features enabled

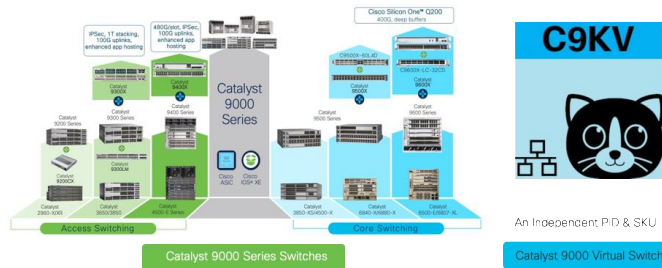


# C9KV Always-On with reservation

Sandbox access: [https://devnetsandbox.cisco.com/DevNet/catalog/Cat9k-Always-On\\_cat9k-always-on](https://devnetsandbox.cisco.com/DevNet/catalog/Cat9k-Always-On_cat9k-always-on)  
Hostname: devnetsandboxiosxec9k.cisco.com

## C9KV Sandbox Capabilities:

- Cisco IOS XE 17.15.1 Virtual Catalyst 9000 UADP 8 port switch
- “Always-On” outside of the Cisco Network DMZ in a colocation on hardware in VMWARE/OVA
- Accessible with reservation though Cisco’s Developer Enablement platform
- Reservable by anybody including customers, partners and external
- Enabled for **read-only usecases** with SSH/CLI and API: NETCONF/YANG, RESTCONF, gNMI
- Support for Model-Driven Telemetry and basic configuration changes through the API
- Supports 40 concurrent sessions



```
sdeweese@SDEWEESE-M-C20V ~ % ssh sdeweese@devnetsandboxiosxec9k.cisco.com  
(sdeweese@devnetsandboxiosxec9k.cisco.com) Password:
```

```
CAT9k_A0#  
CAT9k_A0#  
CAT9k_A0#  
CAT9k_A0#
```

```
CAT9k_A0#sh inv  
NAME: "Switch 1", DESCR: "Catalyst 9000 UADP 8 Port Virtual Switch"  
PID: C9KV-UADP-8P      , VID: V01  , SN: 98DVJUONW1X  
  
NAME: "Switch 1", DESCR: "Catalyst 9000 UADP 8 Port Virtual Switch"  
PID: C9KV-UADP-8P      , VID: V01  , SN: 98DVJUONW1X
```

```
CAT9k_A0#sh ver  
Cisco IOS XE Software, Version 17.15.01
```

# Let's stay connected!

Reach out via mailers or Webex Teams for help and support with your automation journey at:

**cisco-en-programmability@cisco.com**

YANG Suite (General/External): Anyone can join this space using <https://eurl.io/#MaW78CeIS>

C9KV Public - <https://eurl.io/#h6Pe-Qgmo>

Email & Webex Direct: [jcohoe@cisco.com](mailto:jcohoe@cisco.com) & [sdeweese@cisco.com](mailto:sdeweese@cisco.com) & [ashparek@cisco.com](mailto:ashparek@cisco.com)



**We aim to deliver a 5 star session  
Please submit your session survey !  
Your feedback and input is always highly valued**



**Complete your surveys** in the Cisco Events App.



**Complete** a minimum of 4 session surveys and the overall event survey to receive a unique Cisco Live t-shirt.  
(from 11:30 on Thursday, while supplies last)

# Continue your education



**Visit** the Cisco Showcase for related demos



**Book** your one-on-one Meet the Engineer meeting

**Visit** the Technical Solutions Clinics to discuss your technical questions



**Attend** the interactive education with DevNet, Capture the Flag, and Walk-in Labs



**Visit** the On-Demand Library for more sessions at [CiscoLive.com/On-Demand](https://CiscoLive.com/On-Demand)



# Questions



**Thank you**

**CISCO** Live !

